

讲义下载地址: <http://www.leizhang.tk/resources/AI.pdf>



LiVE Group
视觉智能与学习研究中心

机器学习

主讲: 张磊

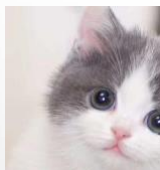
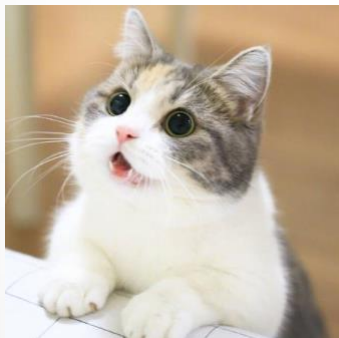
E-mail: leizhang@cqu.edu.cn
Lab Website: <http://www.leizhang.tk>



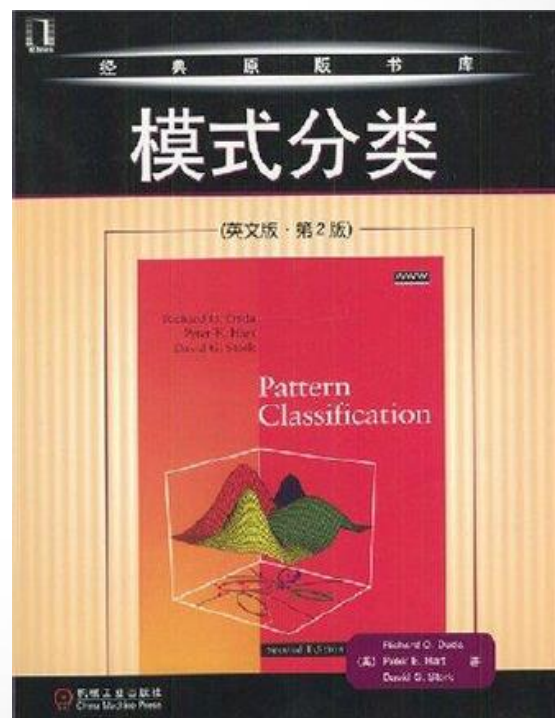
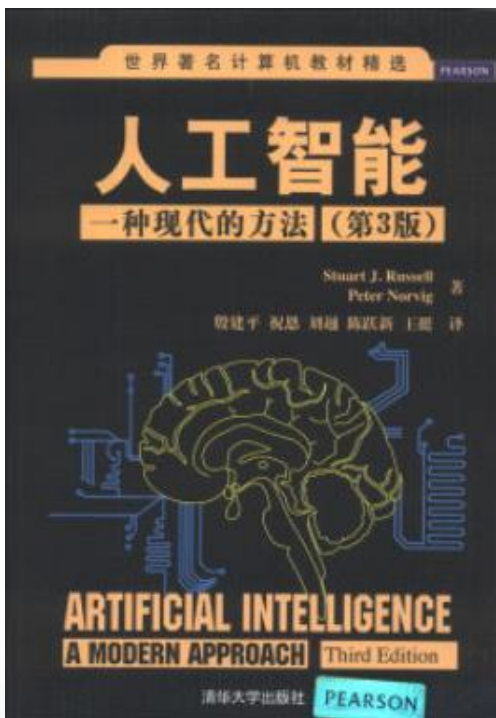
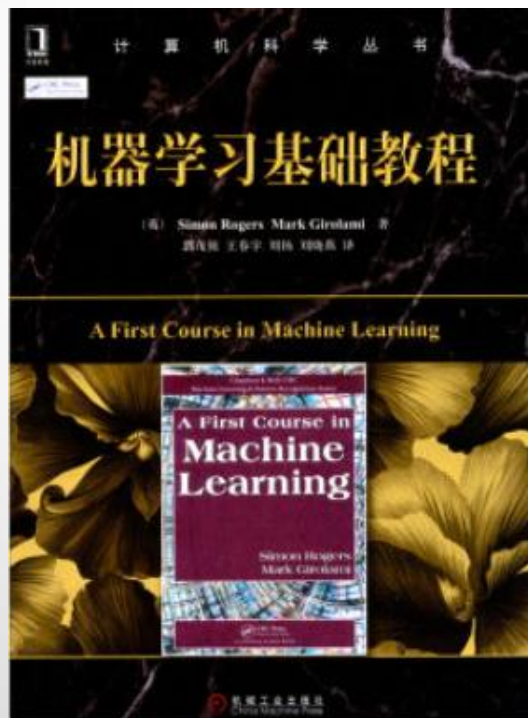


机器学习是什么？ What is Machine Learning?

看图识猫?



教材与参考书目





教材与参考书目

Simon Rogers, Mark Girolami编, 《机器学习基础教程》, 机械工业出版社

Stuart J. Russell, Peter Norvig编 《人工智能——一种现代的方法》第3版, 清华大学出版社,

Richard O. Duda, Peter E. Hart, David G. Stork编 《模式分类》第2版, 机械工业出版社, 中信出版社

Tom M. Mitchell编, 《机器学习》, 机械工业出版社



课时安排

学时：32学时（16次）

考试：理论笔试（闭卷）

学分：2.0

成绩（百分制）：70%考试+30%出勤



课程内容

基础概念：

第一章：绪论—人工智能概念

第二章：智能体

第三章：机器学习中的数学基础

理论方法：

第四章：线性建模（回归）

第五章：正则化学习

第六章：概率建模

第七章：贝叶斯学习

第八章：分类器学习

第九章：群体智能仿生优化

第十章：神经网络与深度学习

国家人工智能发展规划（国务院2017.7）

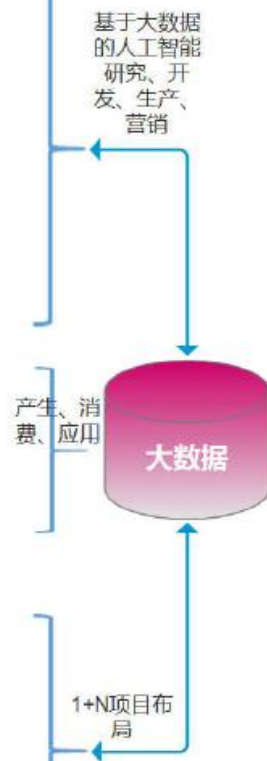
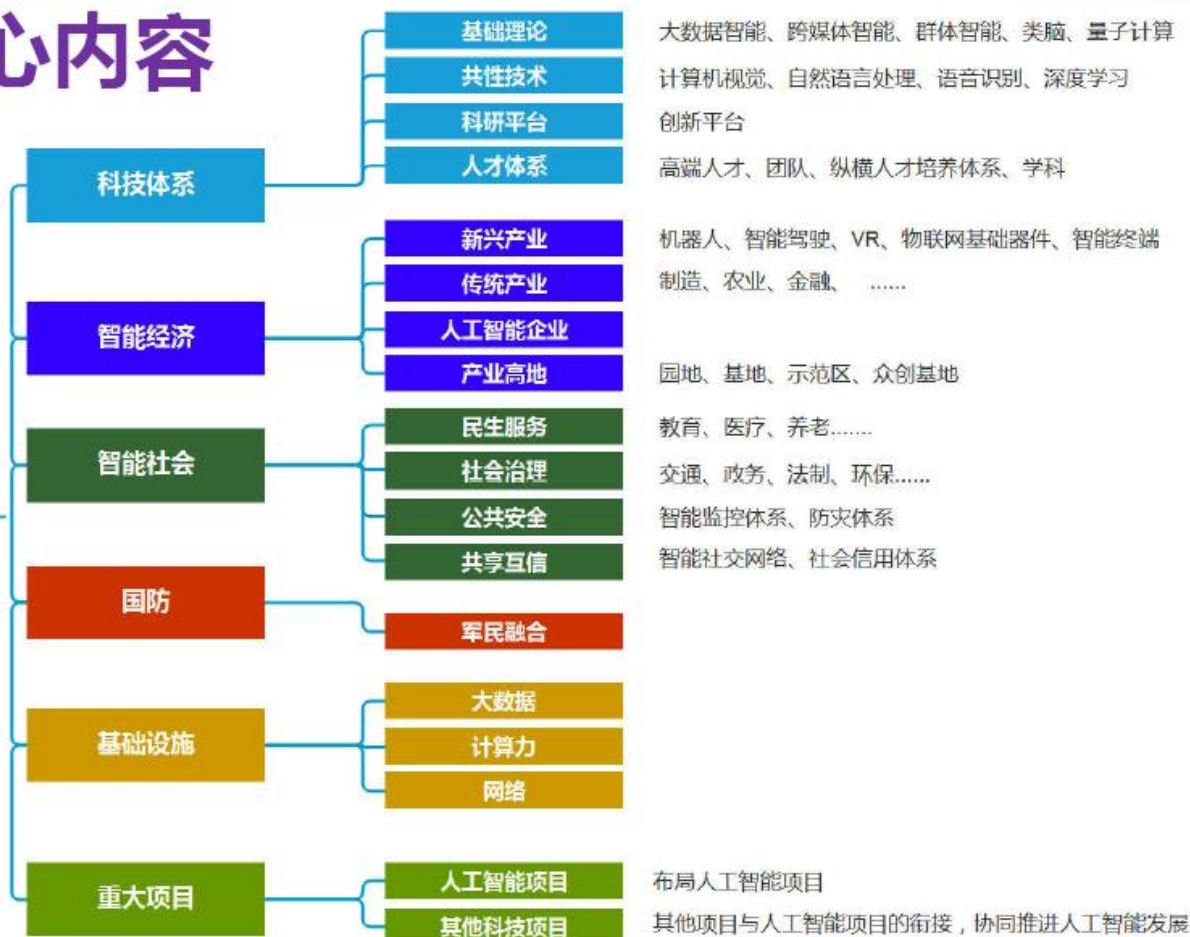




规划核心内容

新一代人工智能发展规划

2017





第一章：绪论

人工智能(Artificial Intelligence, AI) 1956(**达特茅斯会议**):

1950年“图灵测试”的提出，由Alan Turing(人工智能之父)
——测试者与被测试者在隔离开的情况下，向被测试者提出疑问



计算机科学之父
(1912-1954)

[1] **On computable numbers, with an application to the Entscheidungsproblem.** 1936

《论数字计算在决断难题中的应用》

[2] **Computing machinery and intelligence.** 1950. 《计算机与智能》

[3] **Can a machine think?** 1950, The world of mathematics. 《机器能思考吗？》

人工智能是最新兴的科学工程领域。机器学习为实现人工智能的必要手段，其目标和任务是试图理解和创造智能实体。

人工智能的大量粉丝：潜在的爱因斯坦们和爱迪生们，被大多数领域的科学家们评为“最想参与的研究领域”



第一章：绪论

什么是人工智能(Artificial Intelligence, AI)：

- ◆ 电影和小说中描绘的人工智能（主宰的可怕未来）塑造了大众对人工智能的想象，但这些都是虚构的。在现实生活中，人工智能基本上是在改善人类健康、安全和提升生产力等好的方面。
- ◆ 多数研究型大学和科技公司（苹果、谷歌、facebook、IBM、微软、BAT）也投入巨资，并将人工智能视为未来发展的关键。
- ◆ 好莱坞也将人工智能技术搬上荧幕（反乌托邦人工智能幻想故事）

人工智能定义：

一直以来，人工智能缺乏一个精确的、被普遍接受的定义。目前，一个通用的定义：人工智能是致力于让机器变得智能的活动，而智能就是使实体在其环境中有远见地、适当地实现功能性的能力。

第一章：绪论

什么是人工智能(Artificial Intelligence, AI)?

◆ 像人一样思考

机器要有“脑”，而且会自己思考、决策、求解、学习等。初级的人工智能，需要人类去“教”会机器如何思考，也就是利用计算机模型来研究机器的智能（智力），使机器的自动感知、推理、行为，从计算上成为可能（智能计算）。

利用计算机完成人更加擅长的事情，称为计算智能。



第一章：绪论

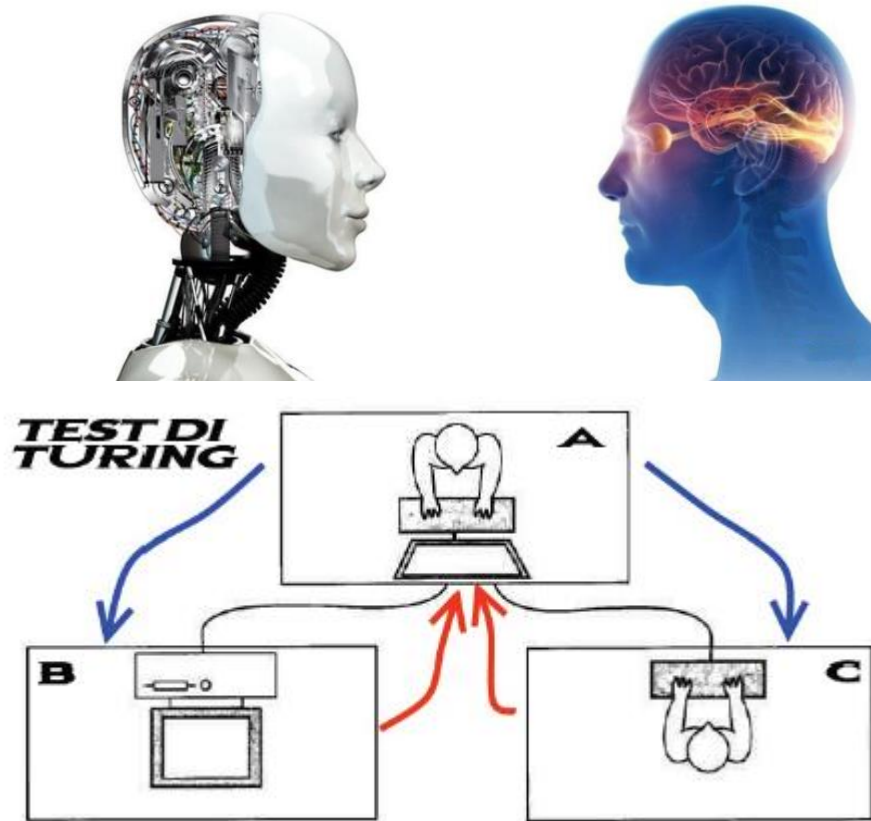


什么是人工智能(Artificial Intelligence, AI)?

◆ 像人一样行动（行为）

图灵测试：由Alan Turing1950提出，为人工智能提供了令人满意的可操作的定义。如果一位人类询问者提出一些问题后，**无法判断对方的回答是来自人还是计算机**，那么这台计算机通过测试（成功“骗”过人）。

60年以后，该测试仍然适合，值得称赞！



第一章：绪论

什么是人工智能(Artificial Intelligence, AI)?

◆ 像人一样行动（行为）

为了能“骗”过人类，还需要通过大量的计算数学模型和计算机编程工作。计算机需要具备以下能力：

知识表示与存储：用于计算机去“记忆”

机器学习：对新知识的认知和学习

自动推理：思考并回答问题和推出新结论

自然语言处理：“听、说”，用于外语交流

计算机视觉：“看”世界（感知）

机器嗅觉、触觉：“嗅”和“摸”（感知）

机器人学：操纵和移动对象



➡这些领域涉及人工智能(AI)的大部分内容



第一章：绪论

什么是人工智能(Artificial Intelligence, AI)?

◆ 像人一样行动（行为）

综上所述，要真正通过图灵测试，计算机必须具备理解语言、学习、记忆、推理、决策等能力。从而产生许多学科，如**机器感知**（计算机视觉、自然语言处理），**机器学习**（模式识别、深度学习、增强学习），**记忆**（知识表示），**智能决策**（规划、数据挖掘），这些都属于人工智能的研究范畴。

第一章：绪论

什么是人工智能(Artificial Intelligence, AI)?

飞机/飞行器属于人工智能吗??

航空领域的研究人员不会把研究目标定义为制造“能完全像鸽子一样飞行的机器，使得它们可以骗过其他真鸽子”。



第一章：绪论

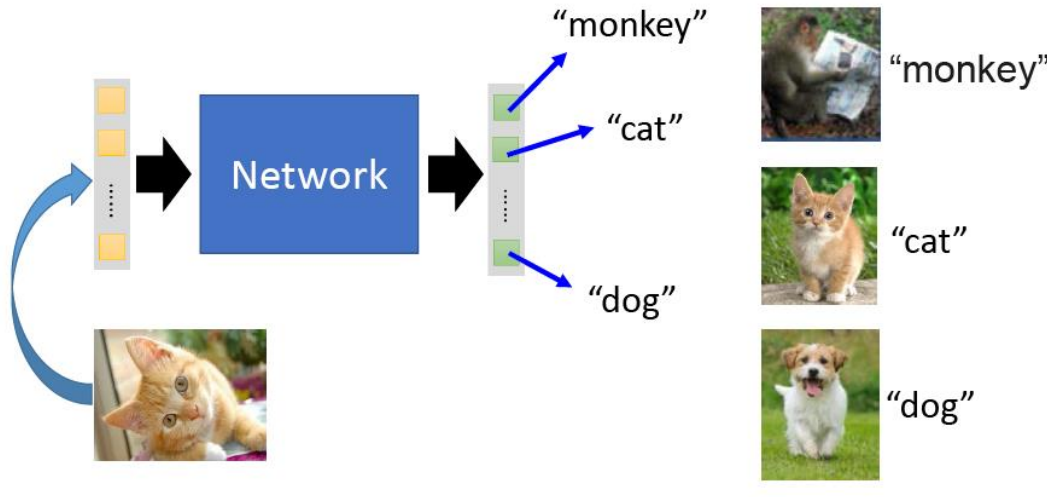
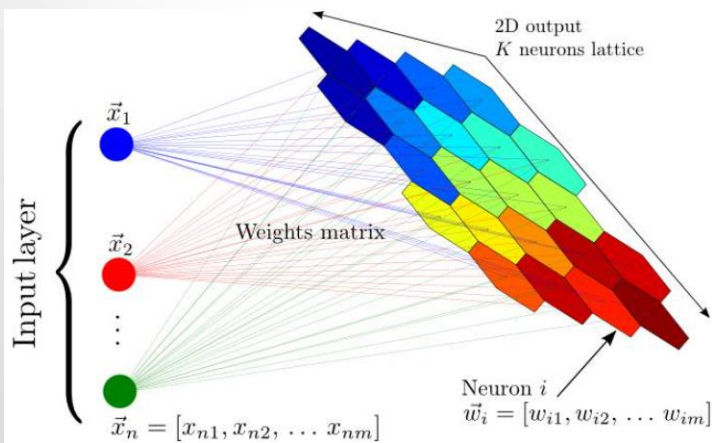


如何实现人工智能(AI)?

◆ 如何让机器像人一样“思考”？

首先，弄清楚人是如何思考，才能教会机器。这是关于逻辑学、心理学、神经科学、脑科学的复杂问题，**目前尚无精确的理论**。虽然人工智能通过计算机模型实现初步的思考、推理和行动，**但与人脑的工作原理之间的联系尚无解释**。

第一章：绪论



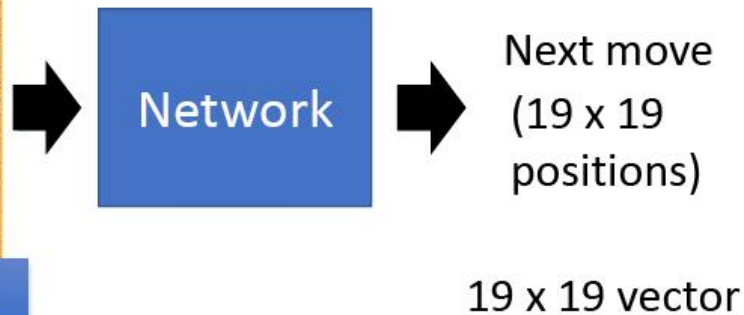
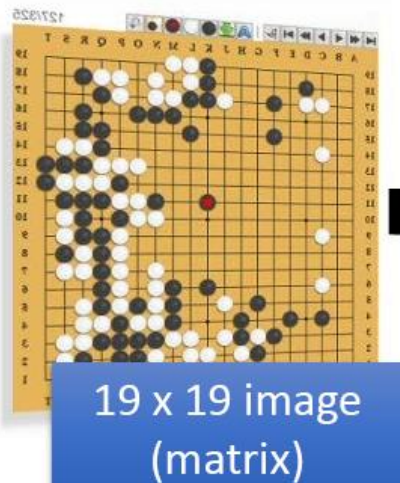
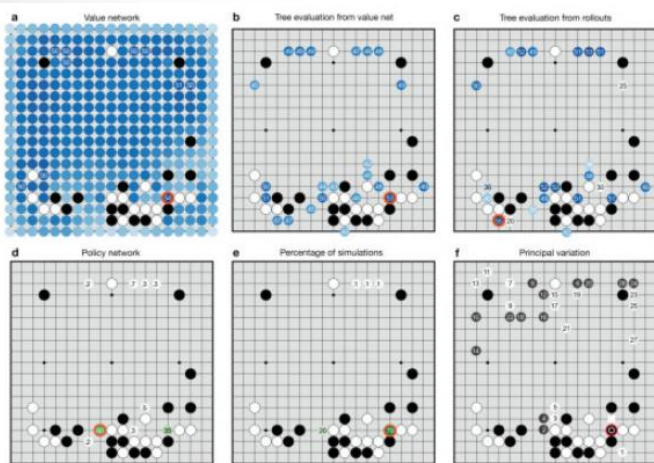
如何实现人工智能(AI)?

◆ 如何让机器像人一样“思考”？

目前，深度神经网络是较为热的研究方向，其能够实现高逼真的人类行为，甚至表现出超过人类的行为，也被誉为**最类似人脑的人工智能技术**，在自然语言处理、计算机视觉和围棋上的效果最为突出。

第一章：绪论

游戏领域（DeepMind）： AlphaGo惊奇问世（Nature 2016）



如何实现人工智能(AI)?

◆ 如何让机器像人一样“思考”？

类脑智能的研究逐渐展开，脑科学的研究人员开始和人工智能领域的计算机和数学研究相结合。以构造符合人脑思维的人工智能新理论和新方法。

第一章：绪论

结构生物学领域：AlphaFold大放异彩

DeepMind蛋白质结构预测：AlphaFold vs. AlphaFold2 （Nature 2021）

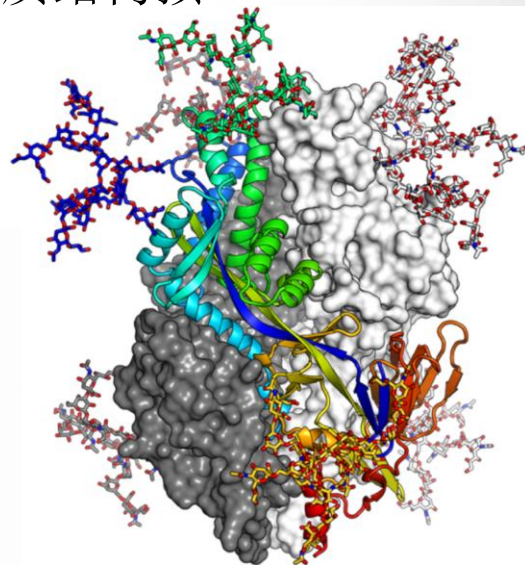
利用蛋白质氨基酸序列预测蛋白质三维结构。在国际蛋白质结构预测大赛（CASP）中成为接近实验方法的冠军。

传统蛋白质结构解析：X射线晶体学和冷冻电镜法。

PIAQIHILEGRSDEQKETLIREVSEAISRSLDAPLTSVRVITTEMAKGHFGIGGELASK

氨基酸序列

AI算法



蛋白质 3D 结构

第一章：绪论

求解数学猜想、奥数题：崭露头角（DeepMind, OpenAI定理证明器, 2021）

计算机编程领域：AlphaCode小试牛刀（2022）

可以编写与人类相媲美的计算机程序（在Codeforces挑战赛中击败46%的参赛者）

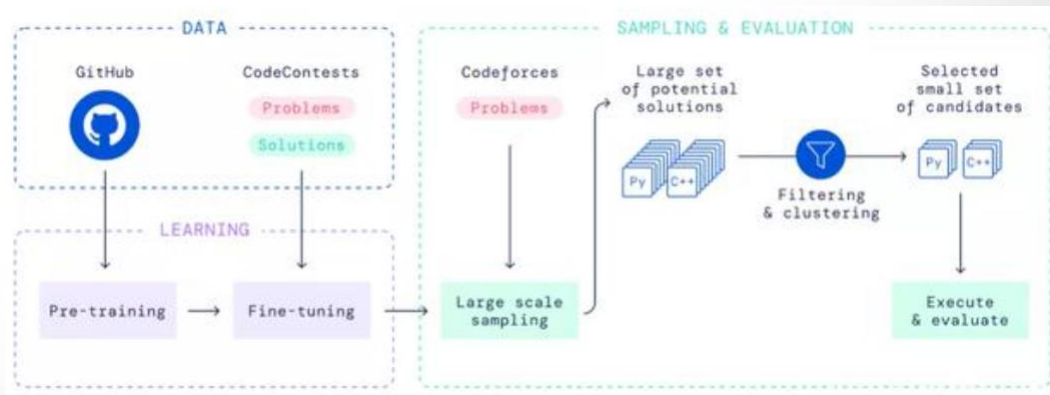
DeepMind

2022-2-2

Competition-Level Code Generation with AlphaCode

Yujia Li*, David Choi*, Junyoung Chung*, Nate Kushman*, Julian Schrittwieser*, Rémi Leblond*, Tom Eccles*, James Keeling*, Felix Gimeno*, Agustín Dal Lago*, Thomas Hubert*, Peter Choy*, Cyprien de Masson d'Audume*, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu and Oriol Vinyals

*Joint first authors



第一章：绪论

ChatGPT: 语言大模型-千亿参数（OpenAI, 2023）

服务所有人：被誉为互联网时代的颠覆性技术，掀起人工智能的新时代

可以扮演生活中各种各样的角色：

程序员、写作/创作、医生、翻译员、办公助手、历史学家、心理咨询、律师等

斯坦福大学学者评估其智力水平相当于9岁儿童

核心技术：RLHF—人类反馈的强化学习

三个步骤：预训练语言模型（GPT）—收集对话数据训练打分模型（RM）—强化学习微调（RL）

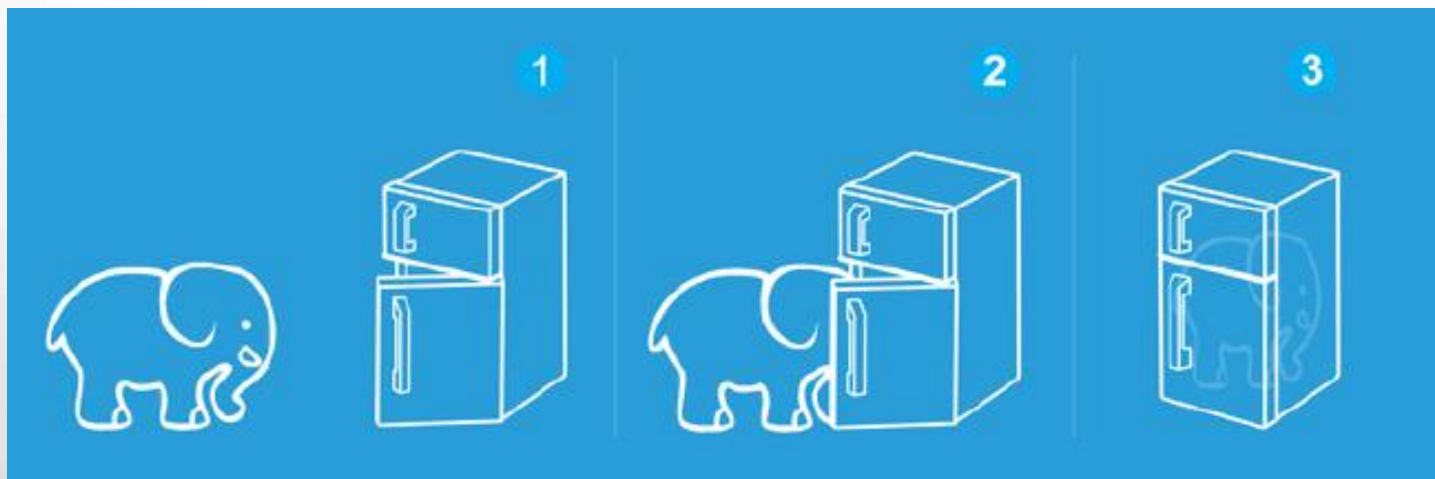
单次训练成本预计：1200万美元（算力：28.5万颗CPU+1万颗GPU A100；数据：45TB）

第一章：绪论

如何实现人工智能(AI)?

◆ 如何让机器像人一样“思考”？

深度学习的简单框架：大象和冰箱的故事。

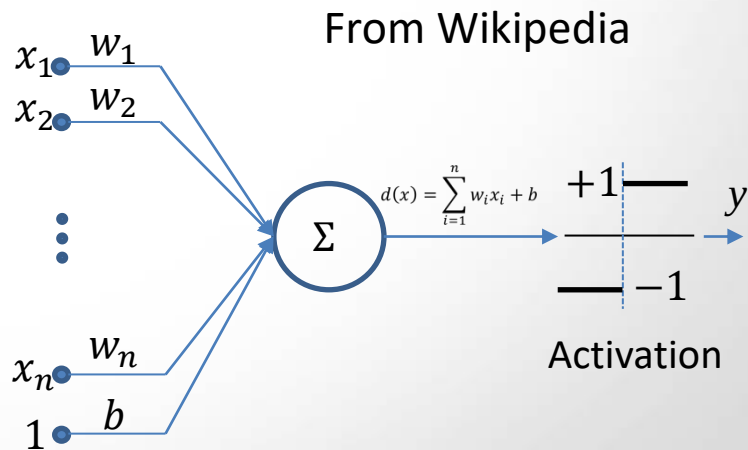


第一章：绪论

人工智能(AI)起源(60年前, 1950):

Rosenblatt 提出“**感知器**”概念，并在刚刚萌芽的AI界产生了不小的争论。

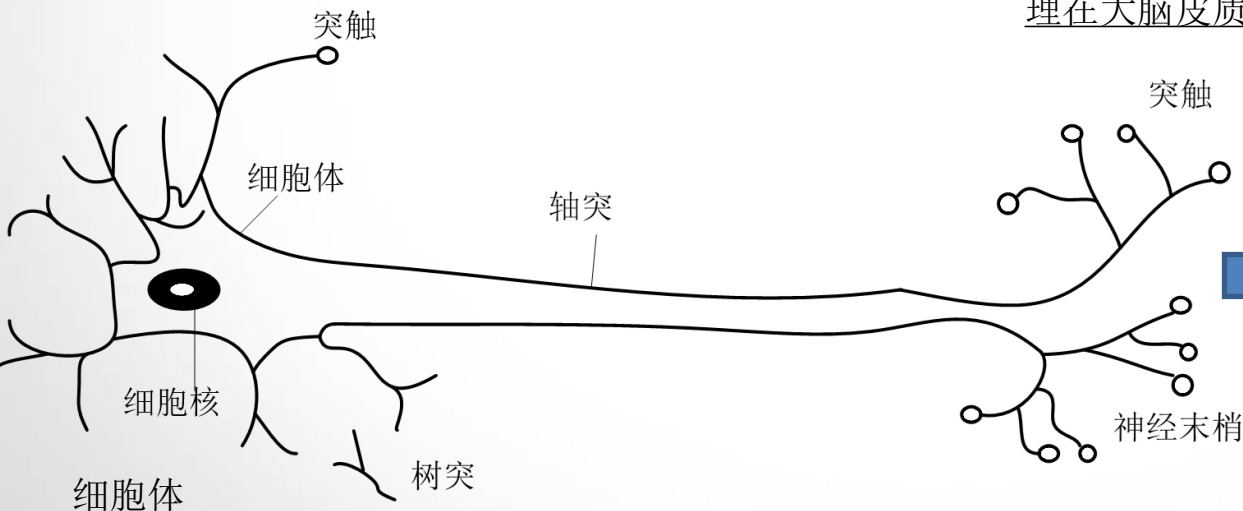
关于“认知”，由于“感知器”的提出，纽约时报称之为“可使电子计算机能够散步、说话、看、写、自我复制、产生意识的胚胎。”



第一章：绪论

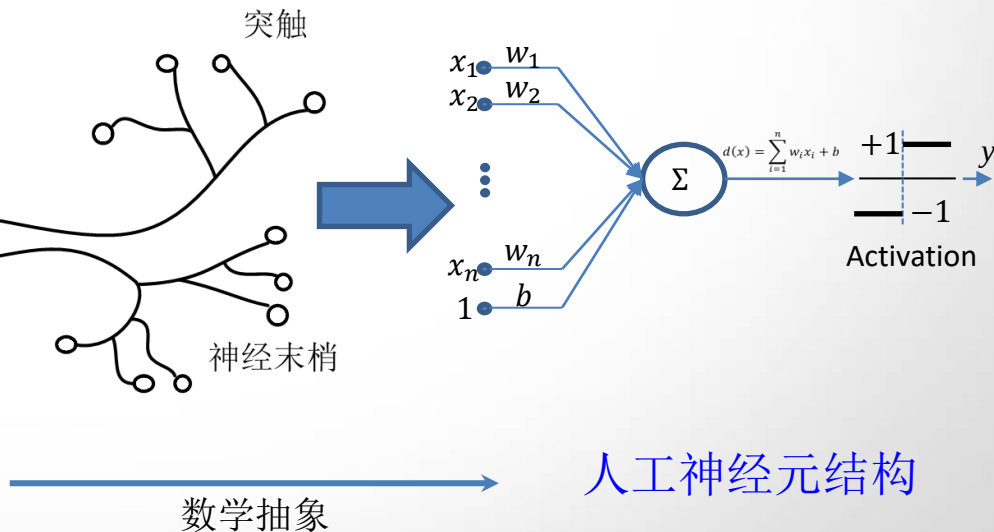
人工智能(AI)起源(60年前, 1950):

生物神经元（神经细胞）结构:



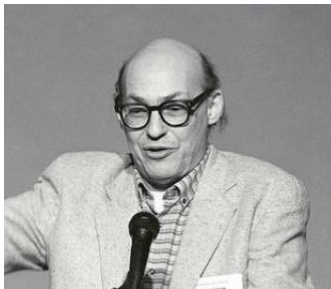
生物神经元结构

每个神经元由一个细胞体组成，其包含一个细胞核。从细胞体分支扩展出许多为树突的神经纤维和一根长的轴突。一个神经元与10到10万个其他神经元相连，连接处称为突触。信号通过复杂的化学反应从神经元传播到神经元。研究发现，大多数的信息处理在大脑皮质，即大脑外层进行。



人工神经元结构

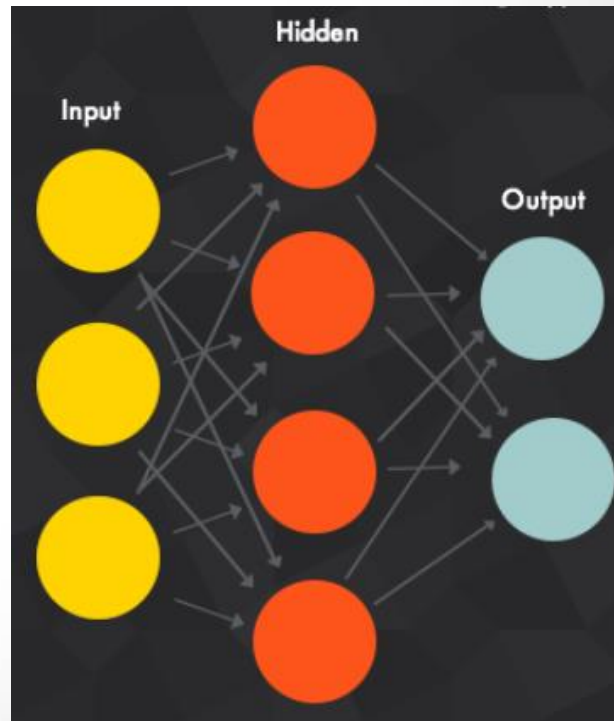
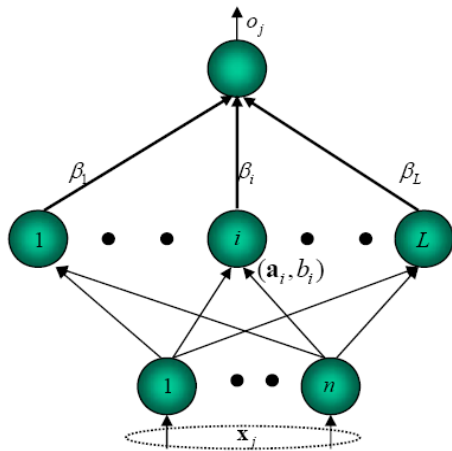
第一章：绪论



马文·明斯基(1927-2016)

人工智能(AI)冬天(1970s):

人工智能冬天也是神经网络的冬天，
Marvin Minsky说“两层的神经网络并不能解决简单的XOR问题，导致AI winter的来临”



第一章：绪论

人工智能(AI)冬天(1970s):

人工智能冬天也是神经网络的冬天，
Minsky说“两层的神经网络并不能解决简单的XOR问题，导致AI winter的来临”

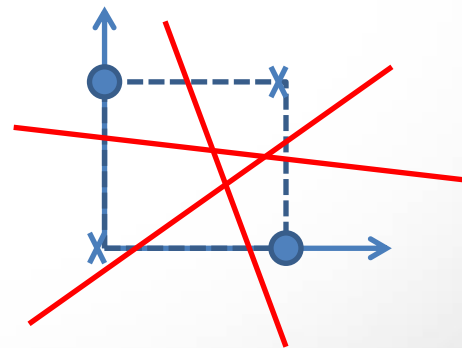
XOR(异或)问题:

0,0: 0

0,1: 1

1,0: 1

1,1: 0



无法解决线性不可分的问题

无法找到一条直线将+1和-1分开

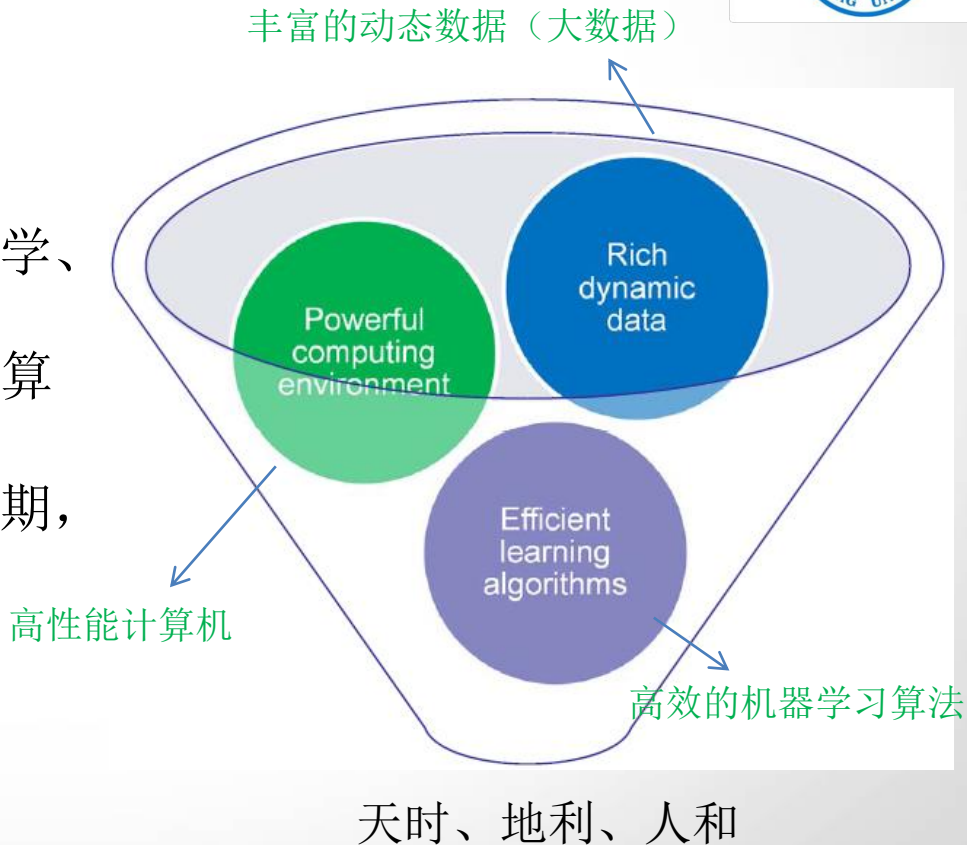
第一章：绪论

人工智能(AI)基础：

数学、计算科学、控制、神经科学、
心理学、语言学、哲学

过去60年，人工智能发展重点在算
法和模型上；

而今天，人工智能发展处于巅峰期，
取决于**三点**：



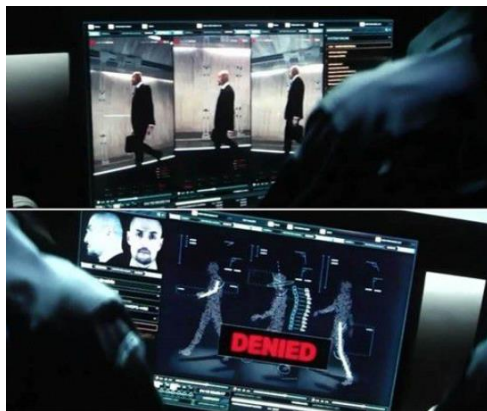
第一章：绪论

人工智能(AI)最新发展：

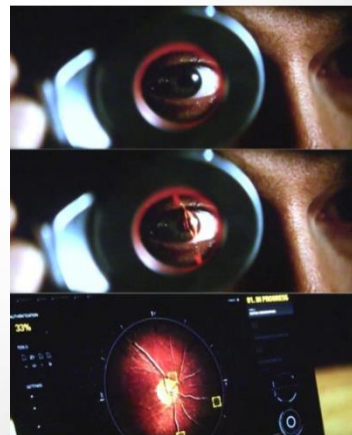
机器视、听、触、嗅、感觉及思维方式的模拟：指纹识别、人脸识别、虹膜识别、掌纹识别、图像/视频识别、语音识别、机器翻译、自然语言处理、搜索/检索、博弈（围棋、象棋）、无人驾驶等



人脸识别（谍影重重5）



步态分析与识别（碟中谍5）



视网膜识别（碟中谍5）

第一章：绪论



利用机器学习技术从大规模人脸图像数据中获取特征、知识表达、识别模型等



第一章：绪论

人工智能(AI)最新发展：

自然语言处理(NLP)：计算机科学和人工智能领域的重要方向之一。其研究人与计算机如何使用自然语言进行通信的理论和方法。主要包括两个方面：自然语言理解、自然语言生成。

难点：自然语言文本和对话存在不同的歧义性或多义性。

存在的技术问题：1. 局限于分析一个孤立的句子，上下文关系和谈话场景对该句子的影响缺乏系统研究。2. 人类理解一个句子不仅仅是凭语法，还运用大量有关知识，比如生活常识和专门技能，而这些知识无法存储在计算机中。



第一章：绪论

人工智能的局限性：语义理解（人机混合智能）



你在说什么
我好像没听懂

叠字问题：

- ❑ 人要是行，干一行行一行，一行行行行行；
- ❑ 今天下雨，我骑车差点摔倒，还好我一把把把把住了；
- ❑ 来到杨过曾经生活地方，小龙女动情地说：我也想过过过儿过过的生活；
- ❑ 多亏跑了两步，差点没上上上上海的车；
- ❑ 校长说：校服上除了校徽别别别的，让你们别别别的别别别的你非别别别的；

第一章：绪论

人工智能的局限性：语义理解（人机混合智能）

语言文化问题：

□ 那辆白车是黑车；

请问到底是白车还是黑车？

□ 冬天能穿多少穿多少；夏天能穿多少穿多少；

请问到底穿多还是穿少？

□ 剩女产生的原因：一是谁都看不上；二是谁都看不上；

请问到底谁看不上谁？

□ 女孩给男朋友打电话：如果你到了，我还没到，你就等着吧；如果我到了，你还没到，你就等着吧！！

请问男孩应该是等呢？还是等呢？



你在说什么
我好像没听懂

第一章：绪论

人工智能(AI)最新发展：

图像视频理解：人工智能和计算机视觉领域的重要方向。其研究如何利用计算机对图像和视频的内容、场景、语义的理解、标注、分类和识别。目前在视频监控、图像检索、图-文转换等领域应用较广。其实现过程分成4个层：

数据层：图像数据的获取（传感器）、压缩和传输（通信）；

描述层：特征提取，图像信号(像素)的形式化；

认知层：图像理解，包括机器学习和推理；

应用层：对特定任务（分类、识别、检测），设计模式识别和机器学习算法。



图像和文本之间的转换
是一项具有趣味性和挑战性的研究

第一章：绪论

人工智能(AI)最新发展：

图像视频理解：人工智能和计算机视觉领域的重要方向。其研究如何利用计算机对图像和视频的内容、场景、语义的理解、标注、分类和识别。目前在视频监控、图像检索、图-文转换等领域应用较广。其实现过程分成4个层：

数据层：图像数据的获取（传感器）、压缩和传输（通信）；

描述层：特征提取，图像信号(像素)的形式化；

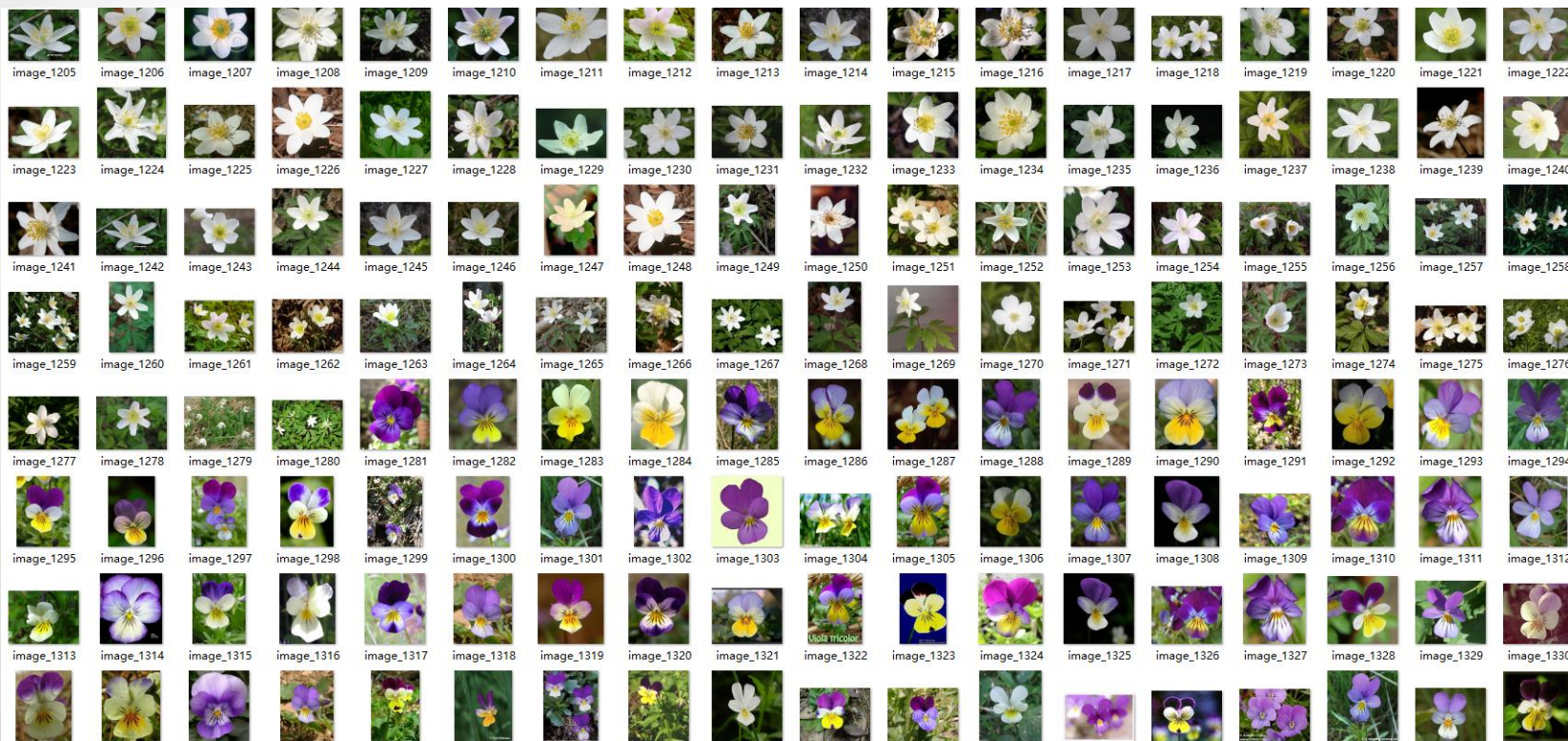
认知层：图像理解，包括机器学习和推理；

应用层：对特定任务（分类、识别、检测），设计模式识别和机器学习算法。



?

第一章：绪论



?

第一章：绪论



?



第一章：绪论

人工智能(AI)最新发展：

博弈：

IBM公司的深蓝(deep blue)第一个在象棋比赛中击败世界冠军(加里.卡斯帕罗夫, 1997)。

深蓝是一台超级计算机，而非智能体（Agent）；

Google公司DeepMind团队开发的AlphaGo在围棋比赛中采用最新的深度学习技术战胜世界冠军(李世石, 2016)。

AlphaGo是一个人工智能程序，是当前人工智能的标志性事件。



第一章：绪论

人工智能(AI)争议：

Von Neumann(冯诺依曼)：计算机不会有智能；

Alan Turing(阿兰图灵)：计算机是能达到人的智力水平的；

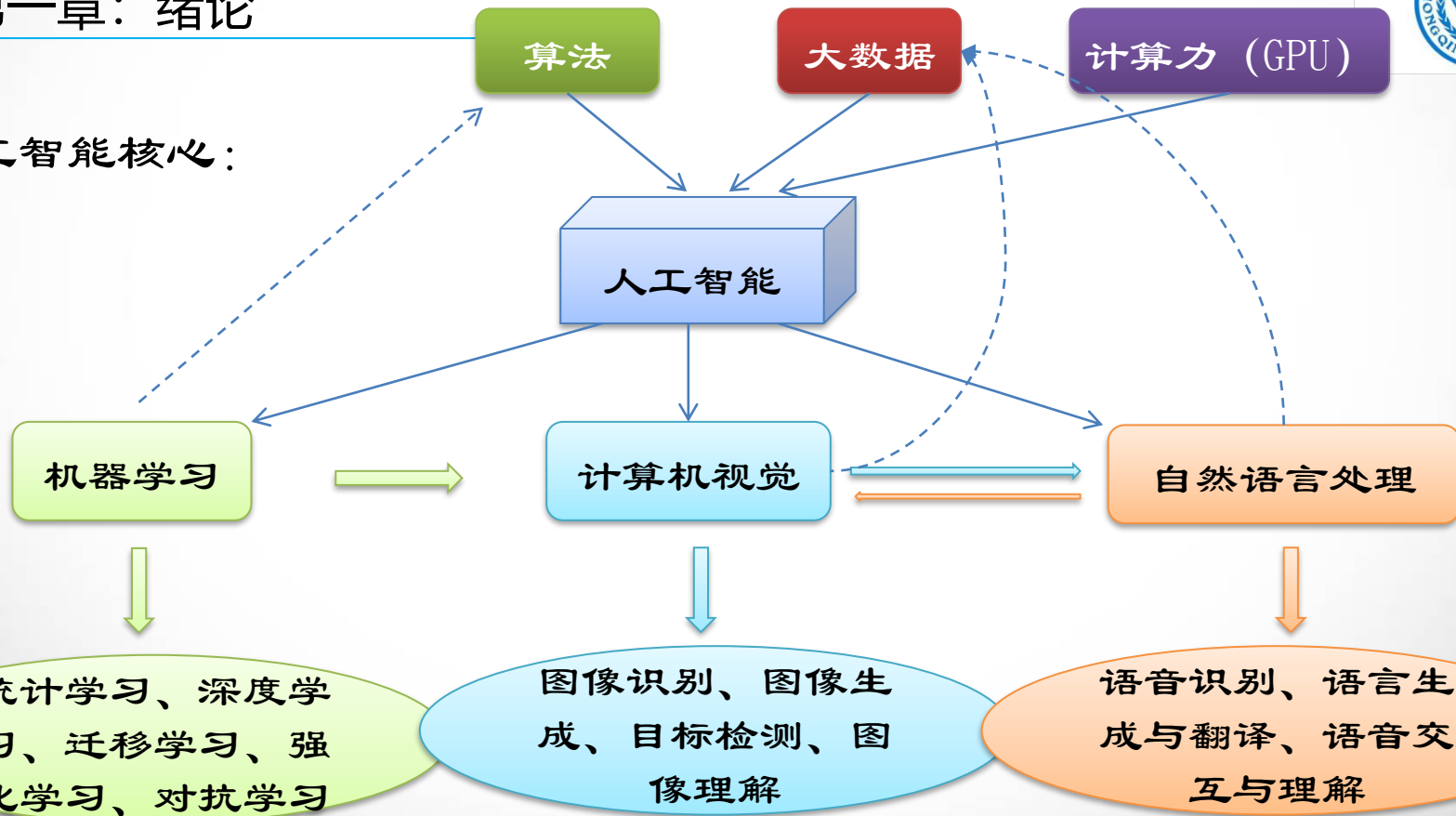
McCarthy(麦卡锡)：人工智能的主要问题是难解的；

Minsky(明斯基)：人工智能是最难的科学之一，是思维的社会无统一的知识表示和理论基础

反对派核心观点：计算机只能解决**形式化**的问题，而客观世界是非形式化的，变化无穷的。（国际象棋完全可以由一个非常简短的、完全形式化的规则列表来描述；大量的日常知识是主观的、直观的，因此很难通过形式化的方式表达清楚）

第一章：绪论

人工智能核心：





第一章：绪论

机器学习 课程：

人工智能涉及多个领域，本课程主要介绍机器学习的基础和方法，主要包括智能体Agent、线性建模、概率学习、演化进化算法、神经网络、深度学习、人脸识别、计算机视觉应用。

AI 到底能干什么？？！！

第一章：绪论

人工智能的常见应用



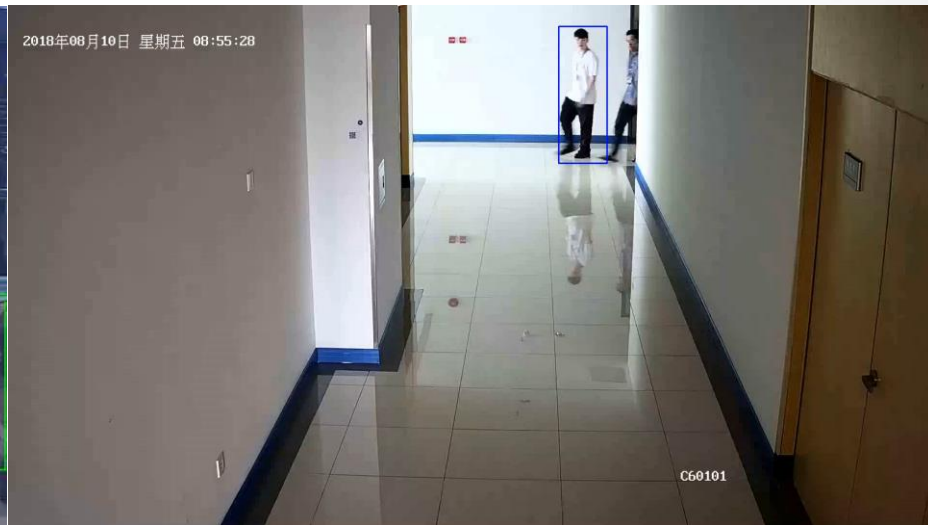
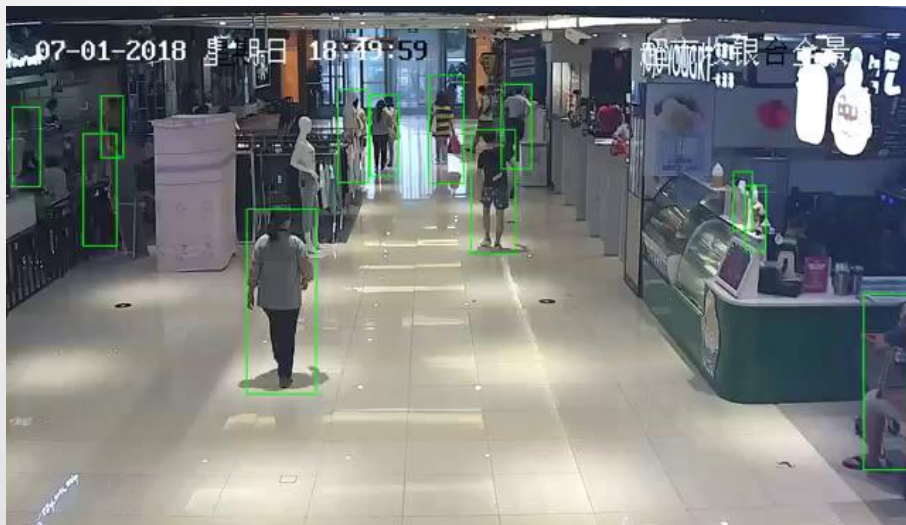
目标检测



道路环境感知
(目标语义分割)

第一章：绪论

人工智能的常见应用

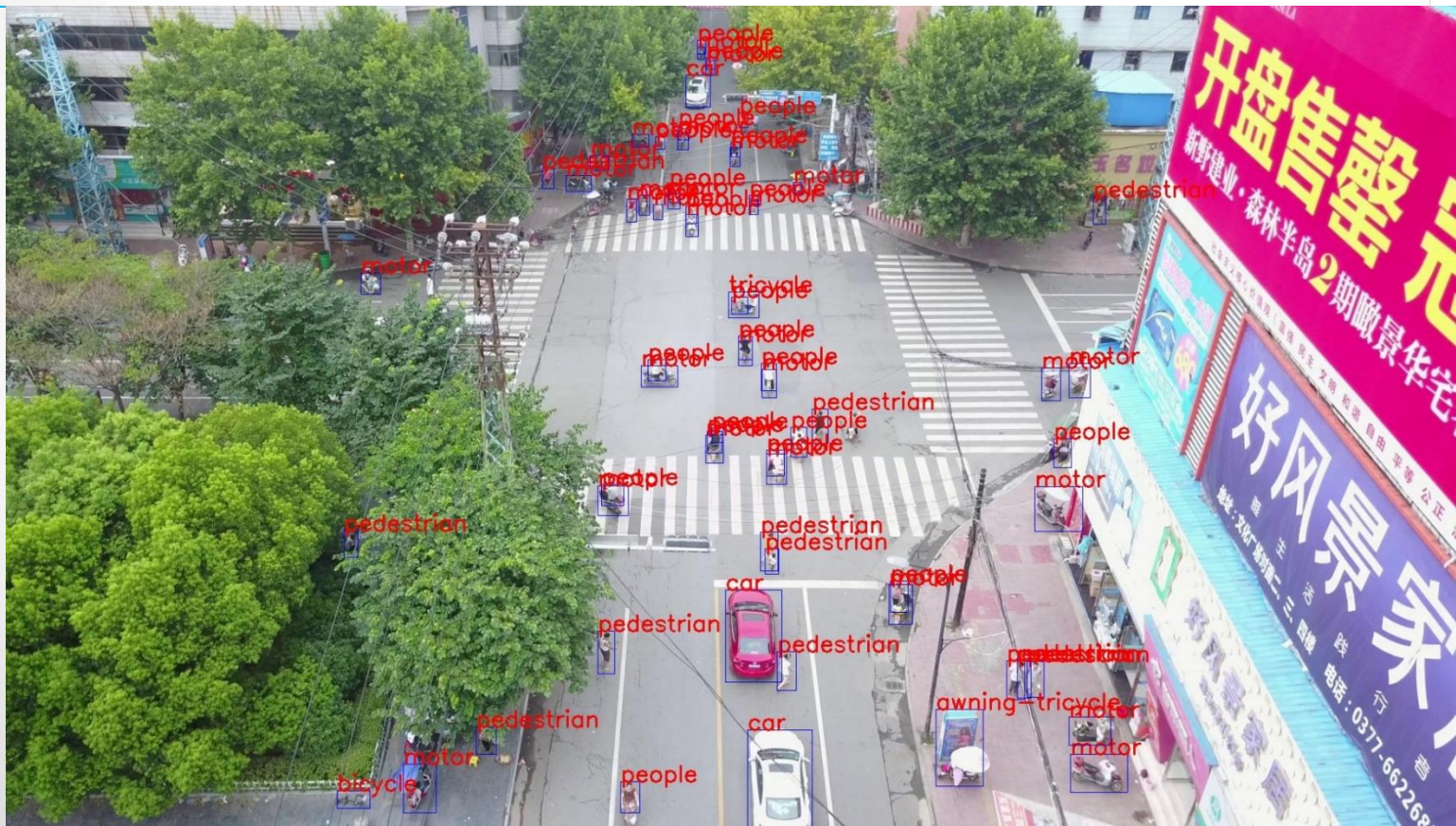


行人检测、跟踪与识别

第一章：绪论



第一章：绪论



第一章：绪论

基于Wi-Fi无线信号的AI应用（穿墙识别）



第一章：绪论

人工智能系统真的安全吗？

攻击行人检测算法（目标检测）



彩色图像块导致算法无法找到右边的人

第一章：绪论

人工智能系统真的安全吗？

攻击人脸识别算法



准确识别（相似度0.6）



贴上补丁

第一章：绪论

人工智能系统真的安全吗？

攻击人脸识别算法



错误识别（相似度0.1）



第一章：绪论

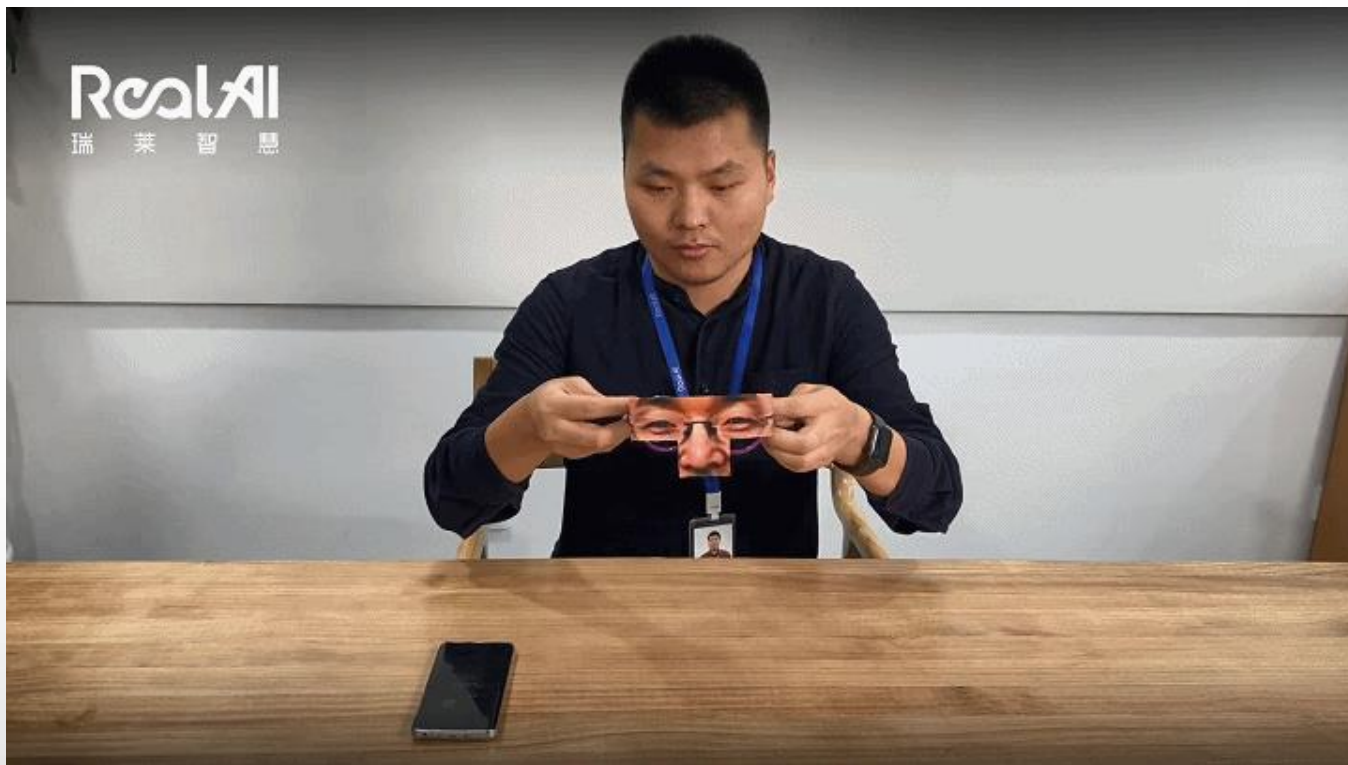
人工智能系统真的安全吗？

清华大学研究人员利用人脸识别技术的漏洞，在**15分钟**内解锁**19部**国产手机！

第一章：绪论

人工智能系统真的安全吗？

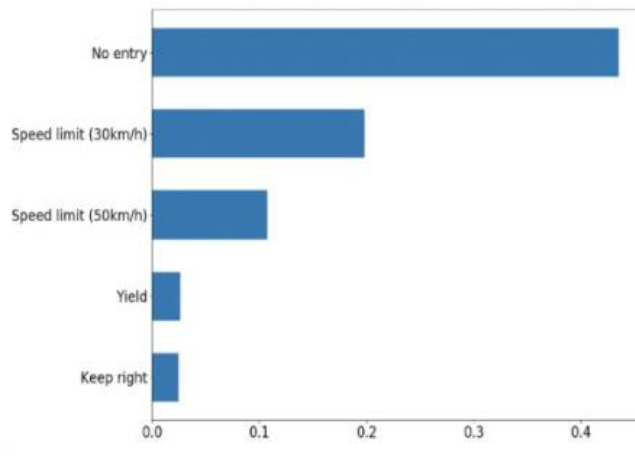
攻击基于人脸识别的手机解锁



第一章：绪论

人工智能系统真的安全吗？

攻击自动驾驶系统

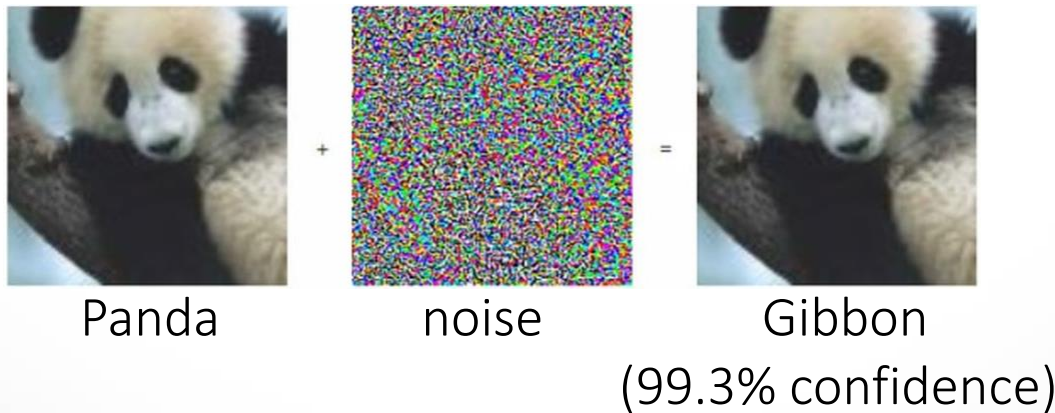


禁行
30km/h限速
50km/h限速

第一章：绪论

人工智能系统真的安全吗？

攻击背后的技术原理：对抗样本（对抗机器学习）





强人工智能可怕吗？

LSTM之父：Jürgen Schmidhuber

- ❑ 我不是很担心AI带来的变化。因为人类已经达到了自我毁灭的最大可能性；
- ❑ 从生物学角度，我们担心的应该是和你相近的物种，和你有共同目标的种群。当超人工出现的时候，它关心的不是我们，而是其他超人工智能；
- ❑ 科幻电影里，我们看到AI奴役人类的情景，这是非常愚蠢的，因为人类对于AI来说非常不好用，不如短时间内制造一大批机器人。



第一章：绪论

人工智能的几大瓶颈：（均与机器学习密切相关）

- ❑ 机器学习的鲁棒性差（不泛化）；
- ❑ 机器学习的迁移性差（不适应）；
- ❑ 机器学习的可解释性差（不透明）；
- ❑ 机器学习的对抗性差（不安全）；

导致AI的安全性、可靠性、可信性较脆弱！



第一章：绪论

什么是机器学习？

□ 概念（照本宣科）：

机器学习是一门多领域交叉的新兴技术，涉及概率论、统计学、凸分析、最优化、算法复杂度等，专门用于辅助**计算机**通过**数据**分析来**模拟人类**的学习行为，从数据中发现有用的知识，是人工智能的核心。

□ 机器学习的特点是需要数据，因此是一门数据驱动的技术。

□ 本质上讲，机器学习就是某种线性（非线性）函数 $f(\cdot)$ 逼近（近似）的技术，以实现给定输入 x 计算（预测）其输出 y ，如下式所示：

$$y = f(x)$$

□ 函数 $f(\cdot)$ 的获取过程，就是机器学习，需要利用统计建模、优化等，即**算法**。

算法

大数据

计算力 (GPU)

第一章：绪论

什么是机器学习？

□ 模拟人类：

人的大部分智能是通过后天的学习和训练获得，包括视觉智能、听觉智能、嗅觉等，通过外界获取/感知信息对大脑产生刺激，从而产生认知能力。

比如，简单的抽象概念的学习——“**看图识物**”



Cellular-----Mobile-----Smart



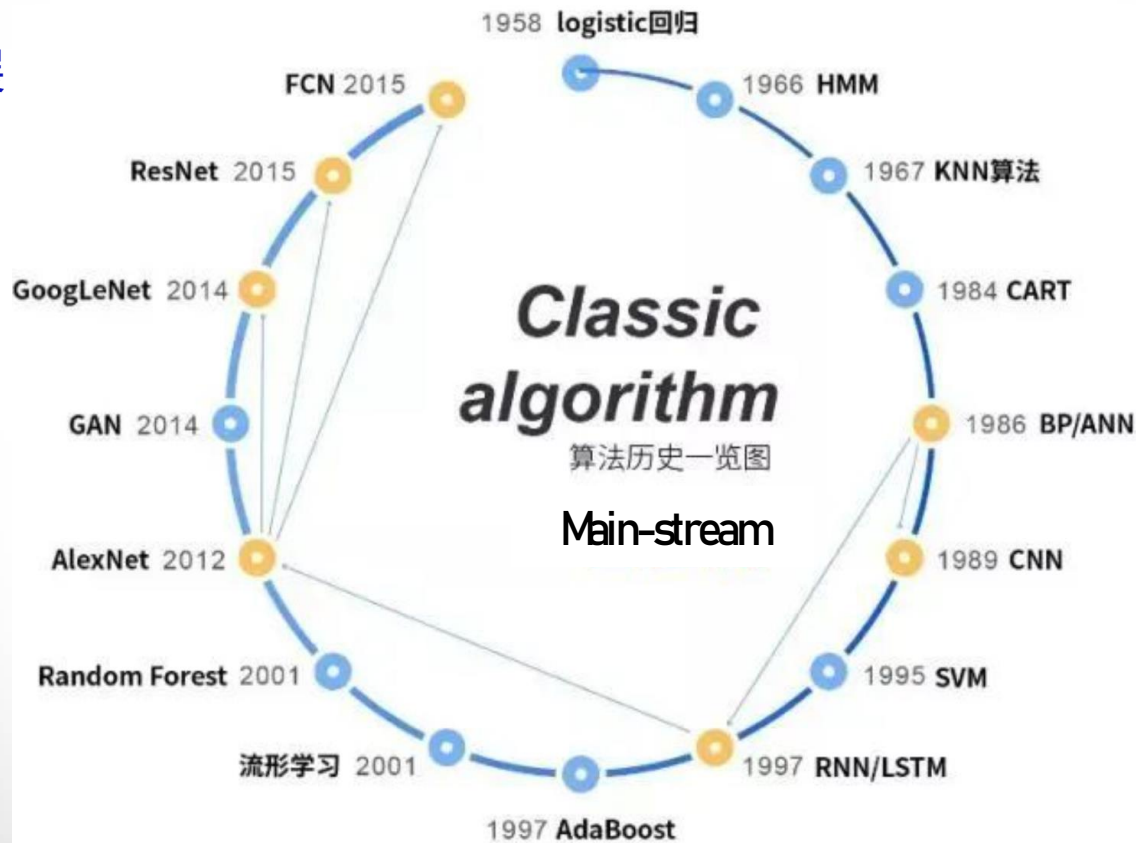
第一章：绪论

机器学习的发展

- 机器学习发展至今已有60余载，时间上主要分为五个阶段：
 - 1960s:** 逻辑回归（线性建模）、近邻搜索、隐马尔可夫模型
 - 1980s:** 分类回归树(CART)、后向传播算法(BP)、卷积神经网络(CNN)
 - 1990s:** 支持向量机、AdaBoost、循环神经网络RNN和LSTM、
 - 2000s:** 流形学习、随机森林、演化学习、稀疏表示
 - 2010s:** 深度学习和强化学习时代（CNN、GAN、RNN、Transformer）

第一章：绪论

机器学习的发展





第一章：绪论

机器学习的发展

□ 按照标注的特性，可分为四大类：

无监督学习：无标注 (主成分分析、流形学习、聚类、自编码器、自监督)

有监督学习：全标注

半监督学习：部分有标签和部分无标签

弱监督学习：标签质量不高、数据质量不高



第二章： 智能体AGENT



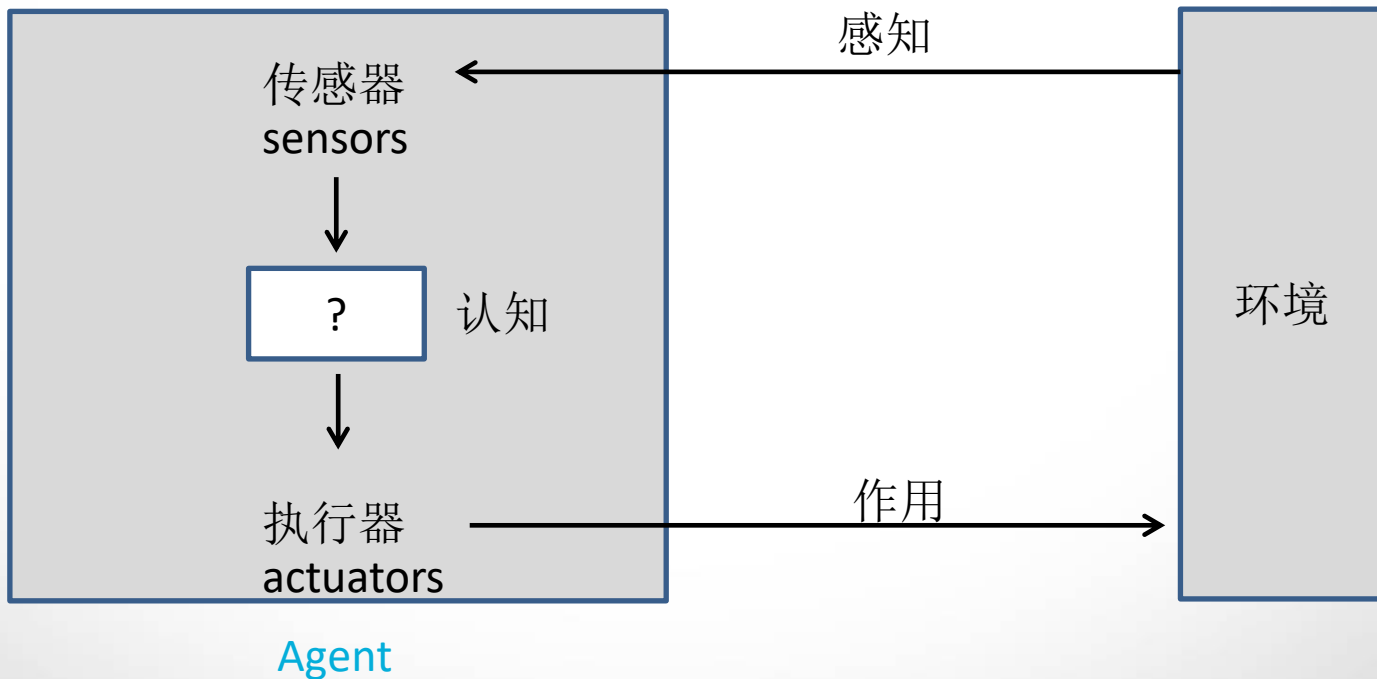
第二章：智能体Agent

Agent:

- ◆ 在人工智能领域，Agent有多种翻译，如“智能体”、“智能代理”，“主体”等。它可以看做是一个自动执行的实体，通过传感器感知环境，通过执行器作用于环境。
- ◆ 多Agent系统即是研究多个智能Agent的协调工作，实现问题求解。
- ◆ 构建Agent的任务，就是设计Agent程序，实现从感知到动作的映射。

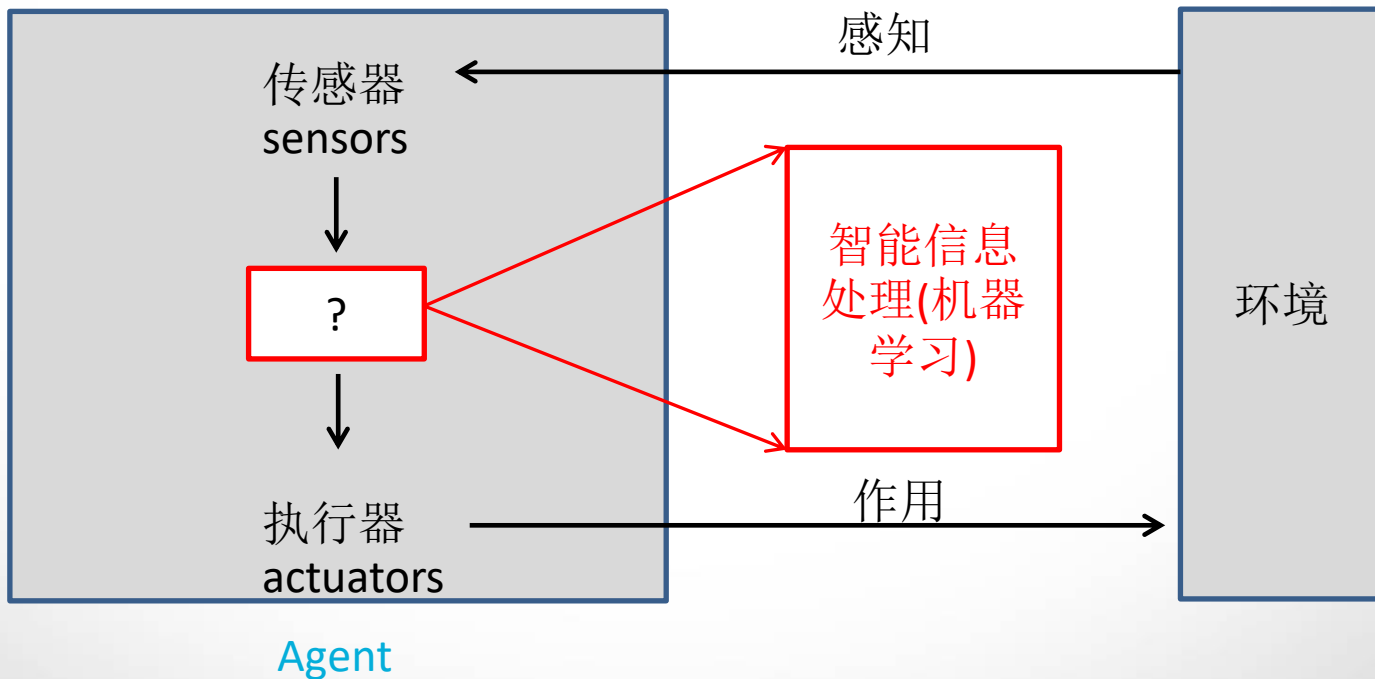
第二章：智能体Agent

智能体通过传感器和执行器与环境进行交互的过程：



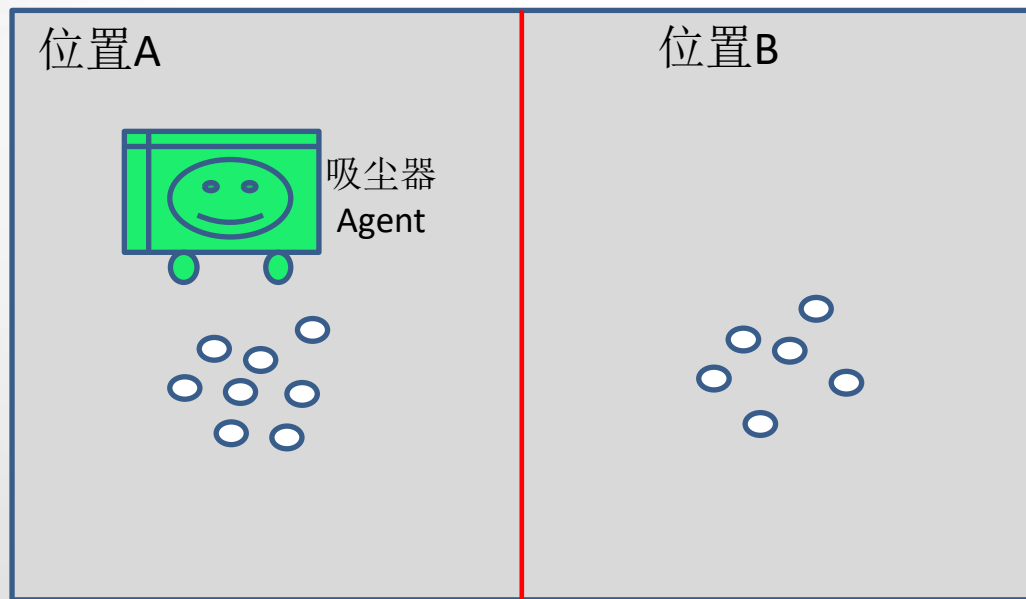
第二章：智能体Agent

智能体通过传感器和执行器与环境进行交互的过程：



第二章：智能体Agent

智能体通过传感器和执行器与环境进行交互的过程：



吸尘器Agent系统



感知	Action
A, clean	right
A, dirty	Suck
B, clean	Left
B, dirty	Suck

简单的Agent函数表格



第二章：智能体Agent

如何定义Agent函数表格才是最好的？是什么决定了一个Agent是好还是坏？智能还是愚笨？

□ 好的行为：理性概念

理性Agent：根据Agent行动的后果，当把Agent置于一个环境中后，它针对收到的感知信息生成一个行动序列，该行动序列会对环境产生一系列的状态变化，如果该系列是渴望的，那么该Agent性能良好。

注意：性能度量是根据环境状态变化进行评估，而不是Agent的状态。（服务机器人）



第二章：智能体Agent

设计Agent要考虑的因素：性能度量

□ 理性Agent

例：对于吸尘器，我们如果通过其1小时内的吸尘总量来评估Agent的性能，那么对于理性Agent来说，一边吸尘，一边把灰尘撒到地上，再吸尘，最终可以保证吸尘量的最大化，但这样好吗？更合适的性能度量，应当奖励保持干净地面的Agent，根据环境的状态来设计性能度量，而不是根据Agent表现出的行为。



第二章：智能体Agent

设计Agent要考虑的因素

- 理性Agent应具备的四点：
 - ◆ 定义成功标准的性能度量。假设性能度量在每个时间步对每块清洁的方格奖励1分。
 - ◆ Agent对环境的先验知识。环境的“地形”作为先验是已知的。
 - ◆ Agent可以完成的行动。左、右、吸尘。
 - ◆ Agent的感知序列。Agent可以正确感知位置及所在方格是否有灰尘。



第二章：智能体Agent

设计Agent要考虑的因素

□ 非理性Agent:

同样的Agent在不同环境下会变的非理性。一旦所有灰尘全部清洁，该吸尘器会毫无必要的来回移动，如果性能度量包含对左右移动惩罚1分，该Agent的性能评价将变的十分糟糕。

一个好的Agent需能够确保所有地方清洁干净后，不在有任何行动。如果再次弄脏，应该不定期检查，重新清洁。如果环境“地形”未知，可能还会去探查其他区域。



第二章：智能体Agent

设计Agent要考虑的因素

□ 理性与全知(完美)Agent:

- ✓ 理性是使期望的性能最大化，而完美是使实际的性能最大化。
理性的选择只依赖于截止当时为止的感知序列，还要确保没有因漫不经心而让Agent进行愚蠢的活动。
- ✓ 根据信息不全的感知序列，采取行动是不理性的，利用全面的感知信息，有助于期望性能最大化。比如智能体过马路。



第二章：智能体Agent

设计Agent要考虑的因素

□ Agent的自主性：

- ✓ 我们定义理性的Agent不仅要收集信息，还应当从感知信息中学习。如果系统设计时，只依赖设计人员的先验知识，Agent缺乏自主性。比如一个吸尘器Agent能够预见灰尘出现的时间和地点，显然会更加“智能”。
- ✓ Agent应当具备“进化”能力。只有与“学习”相结合，才能设计出适应于不同环境的理性Agent。“学习”是人工智能必备的能力。“智能音箱”就不具备进化能力



第二章：智能体Agent

如何设计Agent？

■ Agent的任务环境定义：

通过理性Agent，我们知道：性能度量(Performance)、环境(Environment)、执行器(Actuators)、传感器(Sensors)。把这四个因素归结在一起，就是任务环境，也称为PEAS描述。这是设计Agent的第一步。

考虑自动驾驶出租车Agent：

问：出租车任务环境的PEAS描述应该是什么样？



第二章：智能体Agent

出租车任务环境的PEAS描述

Agent类型	Performance 性能度量	Environment 环境	Actuators 执行器	Sensors 传感器
自动驾驶 出租车	安全、快速、 舒适、符合交 通法规、费用 低	马路、其他车 辆、行人、顾 客、道路施工、 障碍物	油门、刹车、 发动机、转向 控制、显示器、 语音合成器与 顾客交流	摄像头、 红外设备、 声呐、加 速计、速 度表、GPS 导航、麦 克风

医疗诊断系统、卫星图像分析系统等Agent



第二章：智能体Agent

如何设计Agent？

■ Agent任务环境的性质：

✓ 完全可观察与部分可观察

-取决于传感器，如果传感器能获取环境的完整状态，那么任务环境是完全可观察的；如果能够检测与行动决策相关的信息，那么是有效、完全可观察的。

-如果因噪声或传感器丢失状态数据，将导致任务环境为部分可观察的。如果没有传感器，则是无法观察的。



第二章：智能体Agent

如何设计Agent？

■ Agent任务环境的性质：

✓ 单Agent和多Agent

多Agents是以某种竞争或合作的形式存在。

-国际象棋是双Agent环境，Agent A和B是以竞争的形式，A试图最大化自己的性能度量，而最小化对手的性能度量。

-两个出租车司机，则是以合作的形式避免碰撞，从而实现各自的性能度量最大化。

-也存在部分竞争、部分合作，两个Agent，但只有一个停车位。



第二章：智能体Agent

如何设计Agent？

- Agent任务环境的性质：
 - ✓ 确定的与随机的：环境的下一个状态完全取决于当前状态和Agent执行的动作，则是确定的。否则，是随机的，比如突如其来的飞机舱门。
 - ✓ 静态的与动态的：环境在Agent计算时会发生变化，比如路况是动态的。
 - ✓ 已知的与未知的：比如一个Agent到了一个陌生的城市环境，自然不熟悉路况，到了一个陌生国家，对交通法规也是未知的。



第二章：智能体Agent

Agent的结构？

■ Agent内部工作方式：

AI的任务是设计Agent程序，实现感知信息映射到行动的Agent函数 (认知过程)。程序的运行环境，即具有某个物理传感器和执行器的计算装置，也称为体系结构（载体或平台）。

Agent=体系结构（平台）+程序（算法）

体系结构可能是一台计算机，或者具有车载计算机、摄像头和其他传感器的自动驾驶汽车。

一般而言，体系架构为程序提供来自传感器的感知信息，并运行程序，并把程序计算结果传送给执行器。



第二章：智能体Agent

Agent程序

- Agent程序的4种（几乎涵盖全部智能系统的基本原则）：
 - **简单反射Agent**：基于当前的感知信息，选择行动，不关注历史信息。比如简单的吸尘器，只建立在当前位置和是否包含灰尘。触发Agent程序的规则，称为“条件--行为规则”。
 - **基于模型的反射Agent**：根据感知历史，维持智能体内部状态。
 - **基于目标的Agent**：需要提供目标信息。比如出租车可以选择任何方向（左转、右转、直行），但如果提供目标信息后，可以做出正确的选择。
 - **基于效用的Agent**：仅靠目标信息，不能够做出最正确的选择，比如要考虑更安全、更快速、更可靠、更便宜的路线。



第三章： 机器学习中的数学基础



第三章：机器学习的数学基础

人工智能、智能信息处理、机器学习、模式识别的数学基础主要包括以下几个部分：

- 高等数学
- 线性代数
- 矩阵论
- 优化理论
- 概率论
- 信息论
- 计算复杂度理论。

学好这门课，这些数学基础是必不可少的。



第三章：机器学习的数学基础

向量

在线性代数中，标量(scalar)是一个实数，而向量(vector)是指 n 个实数组成的有序数组，称为 n 维向量。如果没有特别说明，一个 n 维向量一般表示为一个列向量，即大小为 $n \times 1$ 的列向量。向量的表示形式一般采用粗体、小写。如 $\mathbf{a}$ 。如果写成 a ，则表示一个标量。

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \in \mathbb{R}^n \quad (n \text{ 维欧几里得空间})$$

常见向量

全1向量：指所有元素为1的向量，通常用 $\mathbf{1}_n$ 表示， n 表示向量的维度。

$\mathbf{1}_K = [1, \dots, 1]_{K \times 1}^T$ 表示 K 维的全1向量。



第三章：机器学习的数学基础

向量的模和范数

向量 $\mathbf{a}$ 的模表示为 $\|\mathbf{a}\|$,计算方法为

$$\|\mathbf{a}\| = \sqrt{\sum_{i=1}^n a_i^2}$$

在线性代数中，范数(norm)是一个表示“长度”概念的函数，为向量空间内所有向量赋予非零的正长度或者大小。对于一个 n 维的向量 $\mathbf{x}$ ，其常见的范数有

L_1 范数：

$$|\mathbf{x}|_1 = \sum_{i=1}^n |x_i|.$$

L_2 范数：

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^T \mathbf{x}}.$$

也叫欧几里得范数，向量的长度；
如果向量 $\mathbf{x}$ 满足 $\|\mathbf{x}\|_2 = 1$ ，
则称向量是归一化的



第三章：机器学习的数学基础

向量的内积

两个具有相同维数 n 的向量 $\mathbf{x}$ 和 $\mathbf{y}$ 的内积记为 $\mathbf{x}^T \mathbf{y}$,是一个标量。

$$\mathbf{x}^T \mathbf{y} = \sum_{i=1}^n x_i y_i$$

内积也称作标量积或点积，在泛函里面表示为 $\langle \mathbf{x}, \mathbf{y} \rangle$

向量的夹角

两个 n 维向量的夹角定义

$$\cos \theta = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

涵义：向量内积是两个向量共线性的度量，说明两个向量之间的相似性。
若 $\mathbf{x}$ 和 $\mathbf{y}$ 正交，则 $\mathbf{x}^T \mathbf{y} = 0$ 。由 $\cos \theta \leq 1$ ， $\mathbf{x}^T \mathbf{y} \leq \|\mathbf{x}\| \|\mathbf{y}\|$ (柯西-施瓦茨不等式)



第三章：机器学习的数学基础

向量的外积

两个具有相同维数 n 的向量 $\mathbf{x}$ 和 $\mathbf{y}$ 的外积记为 $\mathbf{xy}^T$,是一个矩阵。

$$\begin{aligned}\mathbf{xy}^T &= \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} (y_1 \ y_2 \ \cdots \ y_n) \\ &= \begin{pmatrix} x_1 y_1 & \cdots & x_1 y_n \\ \vdots & \ddots & \vdots \\ x_n y_1 & \cdots & x_n y_n \end{pmatrix}\end{aligned}$$

向量外积也称作矩阵积或二元积



第三章：机器学习的数学基础

矩阵

一个大小为 $m \times n$ 的矩阵(matrix)是一个由 m 行 n 列元素排列成的矩形阵列。矩阵里的元素可以是数字、符号或数学式。在我们的课程里，矩阵一般默认为数字矩阵。

一个 n 维向量，可以看作是一个 $n \times 1$ 的矩阵。

一个 $m \times n$ 的矩阵 $\mathbf{M}$ ，表达为 $\mathbf{M} \in \mathbb{R}^{m \times n}$ (欧几里得空间)

矩阵的数学符号通常有粗体、大写表示。



第三章：机器学习的数学基础

矩阵的基本运算

如果 A 和 B 都为 $m \times n$ 的矩阵，则 A 和 B 的加减为

$$(A + B)_{ij} = A_{ij} + B_{ij},$$

$$(A - B)_{ij} = A_{ij} - B_{ij}.$$

A 和 B 的点乘 $A \odot B \in \mathbb{R}^{m \times n}$ 为

$$(A \odot B)_{ij} = A_{ij} B_{ij}.$$

一个标量 c 与矩阵 A 乘积为

$$(cA)_{ij} = cA_{ij}.$$

若 A 是 $m \times p$ 矩阵和 B 是 $p \times n$ 矩阵，则乘积 AB 是一个 $m \times n$ 的矩阵

$$(AB)_{ij} = \sum_{k=1}^p A_{ik} B_{kj}$$



第三章：机器学习的数学基础

矩阵的基本运算

$m \times n$ 矩阵 A 的**转置** (Transposition) 是一个 $n \times m$ 的矩阵, 记为 A^T , A^T 第 i 行第 j 列的元素是原矩阵 A 第 j 行第 i 列的元素,

$$(A^T)_{ij} = A_{ji}.$$

如果 A 是对称矩阵, 则 $A=A^T$

矩阵的**向量化**是将矩阵表示为一个列向量。这里, vec 是向量化算子。设 $A = [a_{ij}]_{m \times n}$, 则

$$\text{vec}(A) = [a_{11}, a_{21}, \dots, a_{m1}, a_{12}, a_{22}, \dots, a_{m2}, \dots, a_{1n}, a_{2n}, \dots, a_{mn}]^T.$$



第三章：机器学习的数学基础

矩阵的基本运算

矩阵的迹

矩阵 $\mathbf{A}$ 的迹，数学表示为 $\text{Tr}(\mathbf{A})$ 或 $\text{Trace}(\mathbf{A})$,即对角线元素的和。

$$\text{Tr}(\mathbf{A}\mathbf{A}^\top) = \text{Tr}(\mathbf{A}^\top\mathbf{A}) = \|\mathbf{A}\|_F^2 = \sum_{i,j} A_{i,j}^2$$

矩阵的逆 $\mathbf{A}^{-1}$

矩阵的转置 $\mathbf{A}^\top$



第三章：机器学习的数学基础

矩阵的基本运算

矩阵与向量的乘积

$$\begin{bmatrix} m_{11} & \cdots & m_{1d} \\ \vdots & \ddots & \vdots \\ m_{n1} & \cdots & m_{nd} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

可以写另一种表达方式

$$\mathbf{M}\mathbf{x} = \mathbf{y}$$

其中，

$$y_i = \sum_{j=1}^d m_{ij} x_j \quad (\text{线性组合})$$



第三章：机器学习的数学基础

常见矩阵

对称矩阵指其转置等于自己的矩阵，即满足 $A = A^T$ 。

对角矩阵（Diagonal Matrix）是一个主对角线之外的元素皆为 0 的矩阵。对角线上的元素可以为 0 或其他值。一个 $n \times n$ 的对角矩阵矩阵 A 满足：

$$A_{ij} = 0 \text{ if } i \neq j \quad \forall i, j \in \{1, \dots, n\}$$

对角矩阵 A 也可以记为 $\text{diag}(a)$ ， a 为一个 n 维向量，并满足

$$A_{ij} = a_i.$$



第三章：机器学习的数学基础

常见矩阵

$n \times n$ 的对角矩阵 $A = \text{diag}(\mathbf{a})$ 和 n 维向量 $\mathbf{b}$ 的乘积为一个 n 维向量

$$A\mathbf{b} = \text{diag}(\mathbf{a})\mathbf{b} = \mathbf{a} \odot \mathbf{b},$$

其中 $\odot$ 表示点乘，即 $(\mathbf{a} \odot \mathbf{b})_i = a_i b_i$ 。

单位矩阵是一种特殊的的对角矩阵，其主对角线元素为 1，其余元素为 0。
 n 阶单位矩阵 I_n ，是一个 $n \times n$ 的方形矩阵。可以记为 $I_n = \text{diag}(1, 1, \dots, 1)$ 。
一个矩阵和单位矩阵的乘积等于其本身。



第三章：机器学习的数学基础

导数

对于定义域和值域都是实数域的函数 $y=f(x)$,若 $f(x)$ 在点 x_0 的某个邻域 Δx 内, 极限 $f'(x_0) = \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$ 存在

则称函数 $y=f(x)$ 在点 x_0 处可导, 其导数为 $f'(x_0)$

函数 $f(x)$ 的**导数** $f'(x)$ 也可记作 $\nabla_x f(x)$, $\frac{\partial f(x)}{\partial x}$ 或 $\frac{\partial}{\partial x} f(x)$ 。



第三章：机器学习的数学基础

向量和矩阵的导数

- 对于一个 p 维的向量 $\mathbf{x} \in \mathbb{R}^p$, 函数 $y = f(\mathbf{x}) = f(x_1, x_2, \dots, x_p)$ 是一个标量, 则 y 关于 $\mathbf{x}$ 的导数为

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_p} \end{bmatrix} \in \mathbb{R}^p$$

→ 相当于梯度, 是由 p 个偏导数组成的矢量

- 如果 $\mathbf{y} = f(\mathbf{x}) = f(x_1, x_2, \dots, x_p) \in \mathbb{R}^q$ 是一个向量,

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_q(\mathbf{x})}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial f_1(\mathbf{x})}{\partial x_p} & \dots & \frac{\partial f_q(\mathbf{x})}{\partial x_p} \end{bmatrix} \in \mathbb{R}^{p \times q}$$



第三章：机器学习的数学基础

导数法则

加（减）法则

$y = f(\mathbf{x}), z = g(\mathbf{x})$ 则

$$\frac{\partial(y+z)}{\partial \mathbf{x}} = \frac{\partial y}{\partial \mathbf{x}} + \frac{\partial z}{\partial \mathbf{x}}$$

乘法法则

(1) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^q$, $\mathbf{z} = g(\mathbf{x}) \in \mathbb{R}^q$, 则

$$\frac{\partial \mathbf{y}^\top \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \mathbf{z} + \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \mathbf{y}$$



第三章：机器学习的数学基础

导数法则

(2) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^s$, $\mathbf{z} = g(\mathbf{x}) \in \mathbb{R}^t$, $\mathbf{A} \in \mathbb{R}^{s \times t}$ 和 $\mathbf{x}$ 无关, 则

$$\frac{\partial \mathbf{y}^\top \mathbf{A} \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \mathbf{A} \mathbf{z} + \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \mathbf{A}^\top \mathbf{y}$$

(3) 若 $\mathbf{x} \in \mathbb{R}^p$, $y = f(\mathbf{x}) \in \mathbb{R}$, $\mathbf{z} = g(\mathbf{x}) \in \mathbb{R}^p$, 则

$$\frac{\partial y \mathbf{z}}{\partial \mathbf{x}} = y \frac{\partial \mathbf{z}}{\partial \mathbf{x}} + \frac{\partial y}{\partial \mathbf{x}} \mathbf{z}^\top$$



第三章：机器学习的数学基础

导数法则

链式法则

(1) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = \mathbf{g}(\mathbf{x}) \in \mathbb{R}^s$, $\mathbf{z} = \mathbf{f}(\mathbf{y}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{y}}$$

(2) 若 $\mathbf{X} \in \mathbb{R}^{p \times q}$ 为矩阵, $\mathbf{Y} = \mathbf{g}(\mathbf{X}) \in \mathbb{R}^{s \times t}$, $\mathbf{z} = \mathbf{f}(\mathbf{Y}) \in \mathbb{R}$,

$$\frac{\partial \mathbf{z}}{\partial \mathbf{X}_{ij}} = \text{tr} \left(\left(\frac{\partial \mathbf{z}}{\partial \mathbf{Y}} \right)^\top \frac{\partial \mathbf{Y}}{\partial \mathbf{X}_{ij}} \right)$$

(3) 若 $\mathbf{X} \in \mathbb{R}^{p \times q}$ 为矩阵, $\mathbf{y} = \mathbf{g}(\mathbf{X}) \in \mathbb{R}^s$, $\mathbf{z} = \mathbf{f}(\mathbf{y}) \in \mathbb{R}$, 则

$$\frac{\partial \mathbf{z}}{\partial \mathbf{X}_{ij}} = \left(\frac{\partial \mathbf{z}}{\partial \mathbf{y}} \right)^\top \frac{\partial \mathbf{y}}{\partial \mathbf{X}_{ij}}$$



第三章：机器学习的数学基础

导数法则

如果矩阵 $\mathbf{M} \in \mathbb{R}^{n \times d}$ 的每一个元素都是关于标量参数 θ 的函数，那么矩阵 $\mathbf{M}$ 对参数 θ 的导数为

$$\frac{\partial \mathbf{M}}{\partial \theta} = \begin{pmatrix} \frac{\partial m_{11}}{\partial \theta} & \dots & \frac{\partial m_{1d}}{\partial \theta} \\ \vdots & \ddots & \vdots \\ \frac{\partial m_{n1}}{\partial \theta} & \dots & \frac{\partial m_{nd}}{\partial \theta} \end{pmatrix}$$

矩阵对标量的导数！



第三章：机器学习的数学基础

导数法则

按位运算的向量函数及其导数：

定义 $\mathbf{x} = [x_1, \dots, x_d]^T \in \mathbb{R}^d$, $\mathbf{z} = [z_1, \dots, z_d]^T \in \mathbb{R}^d$
 $\mathbf{z} = f(\mathbf{x})$

其中 $f(\mathbf{x})$ 是按位运算的，即 $z_i = f(x_i)$

那么 $f(\mathbf{x})$ 对 $\mathbf{x}$ 的导数为

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f(x_1)}{\partial x_1} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \frac{\partial f(x_d)}{\partial x_d} \end{pmatrix}$$



第三章：机器学习的数学基础

随机变量

随机变量通常用于刻画误差或噪声的随机值，如预测一个人的体重。

考查抛硬币的例子：

设 X 是一个随机变量，对于一个正常的硬币， X 的取值为 $x=1$ （正面）或 $x=0$ （反面）

所谓“随机”，是指抛硬币事件发生之前，我们无法知道 X 的取值。

随机变量的类型：离散随机变量、连续随机变量

注：离散随机变量的相关概念可以扩展到连续随机变量。



第三章：机器学习的数学基础

概率和概率分布

为了刻画一个随机变量的可能取值，我们只能通过随机变量取值的**概率**（即可能性） $P(X=x)$ 。（注：大写表示随机变量，小写表示取值）

假设硬币本身没有问题，抛的过程是公平的，则出现正反面的可能性是一致的，即

$$P(X=1)=P(X=0)=0.5$$

也就是说，抛硬币1000次，正反面朝上的次数各占500次。

- 1) 概率是介于0到1之间的值；
- 2) 所有可能结果的概率之和为1；
- 3) 所有可能取值的集合(所有的 x)以及它们的概率 $P(X=x)$ 称为**分布**



第三章：机器学习的数学基础

条件概率

一个随机事件通常会影影响另一个随机事件的结果。

对于抛硬币问题，我们思考“你来抛，我来报”的游戏。

假设硬币本身没有问题，抛的过程也是公平的，

- 1) 你抛硬币的过程定义为随机事件1，用随机变量 X 表示；
- 2) 我猜正反面的过程定义随机事件2，用随机变量 Y 表示；

那么条件概率 $P(Y=y|X=x)$ 表示 X 取特定值时， Y 也取某特定值的概率。

假设我总是报真实结果，那么 $P(Y=1|X=1)=1$ ， $P(Y=0|X=0)=1$ 。

那么 $P(Y=0|X=1)=0$ ， $P(Y=1|X=0)=0$ 。



第三章：机器学习的数学基础

条件概率

假设我只有60%的机会实报“正面朝上”的结果，那么
 $P(Y=1|X=1)=0.6$ ， $P(Y=0|X=0)=1$ 。

那么 $P(Y=0|X=1)=0.4$ ， $P(Y=1|X=0)=0$ 。

可以看到

$$P(Y=1|X=1)+P(Y=0|X=1)=1,$$

$$P(Y=1|X=0)+P(Y=0|X=0)=1$$



第三章：机器学习的数学基础

联合概率

1) 对于两个独立事件, $P(Y=y, X=x)=P(Y=y)*P(X=x)$

2) 若两个事件相互依赖, $P(Y=y, X=x)=P(Y=y|X=x)*P(X=x)=?$

Case 1: 如果我总是报真实结果, 你抛出的硬币正面朝上且我也报正面的概率: ???

$$P(Y=1, X=1)=P(Y=1|X=1)*P(X=1)=1*0.5=0.5$$

Case 2: 如果我只有60%的可能报真实结果, 你抛出的硬币正面朝上且我也报正面的概率: ???

$$P(Y=1, X=1)=P(Y=1|X=1)*P(X=1)=0.6*0.5=0.3$$



第三章：机器学习的数学基础

联合概率

对应抛硬币的例子， x 和 y 有四种可能的组合，那么有下列等式：

$$\sum_{x,y} P(X = x, Y = y) = 1$$

即，概率加和为1。



第三章：机器学习的数学基础

边缘概率（概率的边缘化）

对于抛硬币事件，如果只考查“我报正面的次数所占的比例”，而无关抛硬币的真实结果，就相当于计算 $P(Y=1)$ ，如何计算？

答：可以从联合概率中对 X 进行边缘化得到，也就是对联合概率在 X 的所有取值下的概率之和：

$$P(Y = y) = \sum_x P(Y = y, X = x) = \sum_x P(Y = y|X = x)P(X = x)$$

因此， $P(Y = 1) = \sum_x P(Y = 1, X = x) = P(Y = 1, X = 1) + P(Y = 1, X = 0) = P(Y = 1|X = 1)P(X = 1) + P(Y = 1|X = 0)P(X = 0) = 0.3$



第三章：机器学习的数学基础

例：假设我们有一个公平的硬币和两个骰子（其中1号骰子正常，2号骰子存在瑕疵）。定义抛硬币事件为 X ，掷骰子事件为 Y 。

首先，抛硬币，若正面朝上($X=1$)，掷1号骰子；若反面朝上($X=0$)，掷2号骰子。上述条件概率表格如下：

	1	2	3	4	5	6	
1号骰子	1/6	1/6	1/6	1/6	1/6	1/6	$P(Y=y X=1)$
2号骰子	1/12	1/6	1/4	1/4	1/6	1/12	$P(Y=y X=0)$

- 1) 计算出现正面朝上且是3点的概率；
- 2) 计算掷出3点的概率。



第三章：机器学习的数学基础

贝叶斯规则（贝叶斯公式）

与前面提到的概率不同，贝叶斯公式解决的是一个**逆向问题**。
考查的是根据观测结果，预测真实发生的结果（机器学习）。

例：在抛硬币问题，如果“我报的是正面结果即 $Y=1$ ”，那么贝叶斯逆问题是要回答“**该硬币实际的朝向是正面还是反面**”？

方法：计算 $P(X=1|Y=1)$ 以及 $P(X=0|Y=1)$ 的概率，然后对比两个概率值的大小，有

若 $P(X=1|Y=1) > P(X=0|Y=1)$ ，则该硬币实际朝向是正面；

若 $P(X=1|Y=1) < P(X=0|Y=1)$ ，则该硬币实际朝向是反面；



第三章：机器学习的数学基础

贝叶斯规则（贝叶斯公式）

回顾联合概率表达式：

$$P(Y=y, X=x)=$$

$$P(Y=y | X=x) * P(X=x) = P(X=x | Y=y) * P(Y=y)$$

$$\text{所以, } P(X = x | Y = y) = \frac{P(Y = y | X = x) P(X=x)}{P(Y=y)}$$

经验分析： $P(X=1|Y=1)=1$ （如果我报正面，实际一定是正面）

理论结果： $P(X=1|Y=1)=P(Y=1|X=1)P(X=1)/P(Y=1)=0.6*0.5/0.3=1$

问： $P(X=0|Y=0)=?$



第三章：机器学习的数学基础

期望值

当处理随机变量时，使用一个或多个值代表一个分布的特征非常有用，比如期望值。

期望值表示随机变量 X 的函数的期望取值，对于离散随机变量，

$$E_{P(x)}\{f(X)\} = \sum_x f(x)P(x)$$

如果我们对 x 的期望值感兴趣，那么 $f(X) = X$ ，有

$$E_{P(x)}\{X\} = \sum_x xP(x)$$

问题： X 的期望值一定是随机变量可能取值中的某一个值吗？(骰子)



第三章：机器学习的数学基础

期望值

性质1：随机变量 X 的函数的期望值通常不是函数在 X 的期望值的取值，即 $E_{P(x)}\{f(X)\}$ 不一定等于 $f(E_{P(x)}\{X\})$ 。

性质2：当 $f(X) = aX$ 时， $E_{P(x)}\{f(X)\} = f(E_{P(x)}\{X\})$ ；

性质3：函数和的期望值等于每个函数期望值的和。

$$E_{P(x)}\{f(X) + g(X)\} = E_{P(x)}\{f(X)\} + E_{P(x)}\{g(X)\}$$



第三章：机器学习的数学基础

期望值

两种最常见的期望：均值和方差。

1) 均值定义为 $E_{P(x)}\{X\}$;

2) 方差用于度量随机变量的变化程度，定义为实际值与均值之差的平方的期望值，即

$$\text{var}\{X\} = E_{P(x)} \left\{ \left(X - E_{P(x)}\{X\} \right)^2 \right\}$$

通常，方差越大，则随机变量的取值离均值越远；



第三章：机器学习的数学基础

连续型随机变量-概率密度函数

- 与离散随机变量不同，我们无法写出连续随机变量所有可能的结果，从而也无法给出随机变量取特定值的概率。
- 对于连续随机变量，需要定义概率分布的连续模拟，即 $p(x)$ ，也叫概率密度函数。
- 计算连续随机变量 X 落入某一特定区间的概率，可通过计算 $p(x)$ 关于 x 在这个区间上的定积分：

$$P(x_1 \leq X \leq x_2) = \int_{x_1}^{x_2} p(x) dx$$



第三章：机器学习的数学基础

连续型随机变量-概率密度函数

- 性质1: 如果 $x_1 \leq X \leq x_2$, $\int_{x_1}^{x_2} p(x)dx = 1$
- 性质2: $p(x) \geq 0$ (概率密度函数不能为负)
- 性质3: 概率密度函数不是概率, 没有上界, $p(x)$ 可以大于1

联合概率密度

$$P(x_1 \leq X \leq x_2, y_1 \leq Y \leq y_2) = \int_{x=x_1}^{x_2} \int_{y=y_1}^{y_2} p(x, y) dx dy$$

条件概率密度

$$P(x_1 \leq X \leq x_2 | Y = y) = \int_{x=x_1}^{x_2} p(x | Y = y) dx$$

边缘概率密度

$$p(y) = \int_{x=x_1}^{x_2} p(y, x) dx$$



第三章：机器学习的数学基础

几种常见的连续概率密度函数 $p(y)$

□ 均匀密度函数

$$p(y) = \begin{cases} r, & a \leq y \leq b \\ 0, & \text{其他} \end{cases}$$

问题：r等于多少？(利用性质1)

□ 高斯密度函数

高斯概率密度函数(高斯分布或正态分布)最常用。

$$p(y|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{-\frac{1}{2\sigma^2}(y - \mu)^2\right\} = \mathcal{N}(\mu, \sigma^2)$$



第四章： 线性建模



第四章：线性回归算法

线性建模

在人工智能/机器学习中，核心的本质问题是推断属性变量（自变量）与响应变量（因变量）之间的函数关系，使得给定任何一个属性集合，可以通过该函数关系，预测其响应。

高数中的自变量 x 与因变量 y ，有以下函数关系

$$y=h(x)$$

线性建模的目的：

当给定一组 $(\mathbf{x}, \mathbf{y})$ 集合时，如何估计函数 $h(\cdot)$ 的表达式？



第四章：线性回归算法

线性建模

例1：建立一个能够执行疾病诊断的模型 $h(\cdot)$ 。

已知条件：已知疾病状态（健康、患病）的多个患者，以及对这些患者测量的属性（血压、心率、体重等）

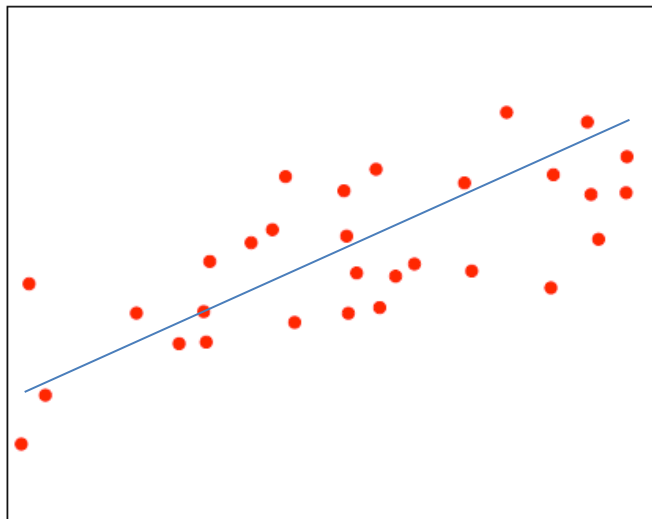
例2：通过某用户在电影网页上的点击情况，预测该用户的喜好。

已知条件：已知被点击的电影种类（喜剧、动作、恐怖等），以及该用户对每个种类的点击次数。

第四章：线性回归算法

模型假设

在建立回归模型时，模型假设很重要。



x	y
0.86	2.49
0.09	0.83
-0.85	-0.25
0.87	3.10
-0.44	0.87
-0.43	0.02
-1.10	-0.12
0.40	1.81
-0.96	-0.83
0.17	0.43



第四章：线性回归算法

线性建模

线性建模是机器学习中最直接的学习问题：学习 属性与响应之间的线性关系。

例：建立苹果的重量预测模型：单属性问题（一维问题）

已知条件：已知 N 个苹果的重量 $t_1, t_2, \dots, t_N$ ，以及 N 个苹果的水平宽度 $x_1, x_2, \dots, x_N$ 。

任务：估计苹果的重量预测线性模型 $t = h(x) = \omega_0 + \omega_1 x$



第四章：线性回归算法

什么是好的准则？

为了选择某种方式下最好的 ω_0 和 ω_1 值，使得该直线能够尽可能与所有数据点接近。

度量“好”的方法是采用“平方差”，即预测的苹果重量 h 与实际重量 t 之间的平方差（为什么要平方？），定义为

$$(t_n - h(x_n; \omega_0, \omega_1))^2$$

该式子表示第 n 个苹果的预测误差，该值越小，说明模型 $h(\cdot)$ 在 x_n 处的值越接近 t_n 。

该度量有个专业称呼：“平方损失函数” (squared loss function)，因此，第 n 个苹果（样本）的损失可以定义为

$$L(t_n, h(x_n; \omega_0, \omega_1)) = (t_n - h(x_n; \omega_0, \omega_1))^2$$



第四章：线性回归算法

什么是好的模型？

对于全部的 N 个苹果，我们希望所有苹果的损失都小，因此，有总体平均损失：

$$L = \frac{1}{N} \sum_{n=1}^N L(t_n, h(x_n; \omega_0, \omega_1)) = \frac{1}{N} \sum_{n=1}^N (t_n - h(x_n; \omega_0, \omega_1))^2$$

我们建模的目的是求得最佳的 ω_0, ω_1 ，使得平均损失 L 达到最小。

上面的问题可以写成专业化的机器学习模型：

$$\min_{\omega_0, \omega_1} L = \frac{1}{N} \sum_{n=1}^N (t_n - h(x_n; \omega_0, \omega_1))^2$$

$$\text{或者 } \{\omega_0, \omega_1\} = \arg \min_{\omega_0, \omega_1} L = \frac{1}{N} \sum_{n=1}^N (t_n - h(x_n; \omega_0, \omega_1))^2$$



第四章：线性回归算法

求解过程

将损失函数展开

$$\begin{aligned} L &= \frac{1}{N} \sum_{n=1}^N (t_n - h(x_n; \omega_0, \omega_1))^2 \\ &= \\ &\vdots \\ &= \frac{1}{N} \sum_{n=1}^N (\omega_1^2 x_n^2 + 2\omega_1 x_n (\omega_0 - t_n) + \omega_0^2 - 2\omega_0 t_n + t_n^2) \end{aligned}$$

计算损失函数 L 对 ω_0 和 ω_1 的偏导数，并令导数为0，可得出损失最小值时的两个估计参数。



第四章：线性回归算法

预测过程(测试推理阶段)

通过极值求解，可获得 ω_0 和 ω_1 的最佳估计值 $\hat{\omega}_0$ 和 $\hat{\omega}_1$ ，从而可以写出函数表达式

$$t = \hat{\omega}_0 + \hat{\omega}_1 x = -0.8 + 0.19x$$

此时，对于一个新的样本（苹果），经过测量，发现其宽度为**12cm**，那该苹果的重量应该是多少？

$$t = -0.8 + 0.19 \times 12 = 1.48kg$$



第四章：线性回归算法

模型的矩阵求解

继续考虑刚才的模型

$$t_n = h(x_n) = \omega_0 + \omega_1 x_n$$

可以写成

$$t_n = h(\mathbf{x}_n) = (\omega_0, \omega_1) \begin{pmatrix} 1 \\ x_n \end{pmatrix} = \mathbf{w}^T \mathbf{x}_n$$

现将 N 个样本的属性向量 $\mathbf{x}_n$ 合并成一个矩阵 $\mathbf{X}$

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_N \end{pmatrix}$$

也将 N 个样本的目标 t_n 合并成一个向量

$$\mathbf{t} = (t_1, t_2, \dots, t_N)^T$$



第四章：线性回归算法

模型的矩阵求解 原损失函数

$$L = \frac{1}{N} \sum_{n=1}^N (t_n - h(x_n; \omega_0, \omega_1))^2$$

可以完全等价地转为向量和矩阵的函数，即

$$\begin{aligned} L &= \frac{1}{N} (\mathbf{t} - \mathbf{X}\mathbf{w})^T (\mathbf{t} - \mathbf{X}\mathbf{w}) \\ &= \frac{1}{N} (\mathbf{t}^T - (\mathbf{X}\mathbf{w})^T) (\mathbf{t} - \mathbf{X}\mathbf{w}) \\ &= \frac{1}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - \frac{2}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{t} + \frac{1}{N} \mathbf{t}^T \mathbf{t} \end{aligned}$$

此时，可以通过矩阵向量求导，直接获得 $\mathbf{w}$ ，而无需对 ω_0, ω_1 分别求偏导。



第四章：线性回归算法

模型的矩阵求解

$$L = \frac{1}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - \frac{2}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{t} + \frac{1}{N} \mathbf{t}^T \mathbf{t}$$

此时，可以通过矩阵向量求导，可以直接获得 $\mathbf{w}$ ，而无需对 ω_0, ω_1 分别求偏导。

$$\text{因为 } \frac{\partial L}{\partial \mathbf{w}} = \begin{bmatrix} \frac{\partial L}{\partial \omega_0} \\ \frac{\partial L}{\partial \omega_1} \end{bmatrix} = \frac{2}{N} \mathbf{X}^T \mathbf{X} \mathbf{w} - \frac{2}{N} \mathbf{X}^T \mathbf{t} = 0$$

可以推出

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{t}$$

$$\text{那么 } \mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t} \quad (\text{完毕})$$



第四章：线性回归算法

模型的矩阵求解

现在已经获得 $\mathbf{w}$ 的解析表达式

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t} = \begin{bmatrix} \omega_0 \\ \omega_1 \end{bmatrix}$$

例：给出3个样本点组成的集合

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 3 \\ 1 & 5 \end{bmatrix}, t = \begin{bmatrix} 4.8 \\ 11.3 \\ 17.2 \end{bmatrix}$$

如何建立线性模型的表达式？

预测： 给定一个属性 $\mathbf{x}_{new} = [1 \ x_{new}]^T$ 的新向量, 其预测公式:

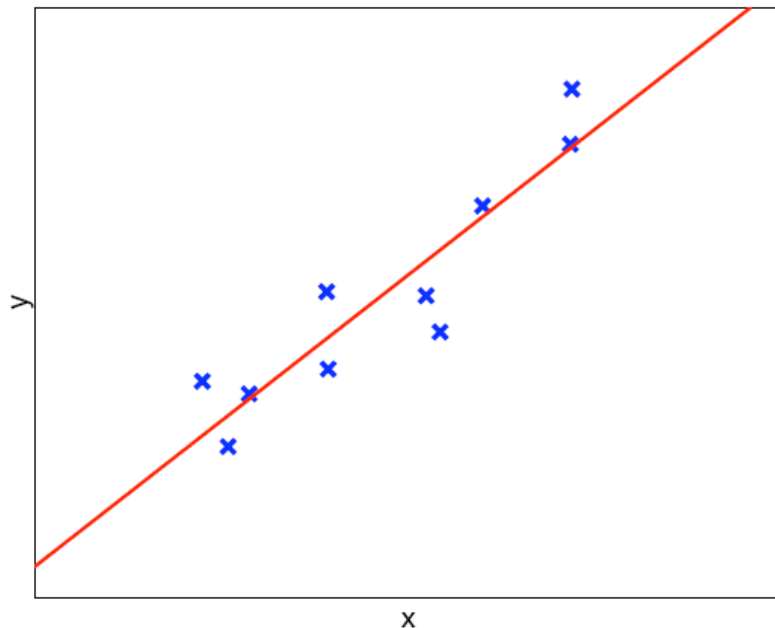
$$t_{new} = \omega_0 + \omega_1 x_{new} = [\omega_0 \ \omega_1] [1 \ x_{new}]^T = \mathbf{w}^T \mathbf{x}_{new}$$



第四章：线性回归算法

Example: Data and best linear hypothesis

$$y = 1.60x + 1.05$$





第四章：线性回归算法

线性建模-多维问题

多维问题是机器学习在实际应用中的普遍问题，即有多个属性的情况。

例：建立一个能够执行疾病诊断的线性模型：多属性问题（多维）

已知条件：已知疾病状态（健康、患病）的 N 个患者，以及每个患者的血压值 x_1 、心率 x_2 、体重 x_3 等，通过多个方面的因素，来预测患者的身体状况（响应）。



第四章：线性回归算法

线性建模-多维问题

如果有 d 个属性，此时的属性向量 $\mathbf{x}_n$ ，可以表示为

$$\mathbf{x}_n = \begin{pmatrix} 1 \\ x_{1,n} \\ \vdots \\ x_{d,n} \end{pmatrix}$$

注意： ω_0 也叫作“偏置项”。样本 $\mathbf{x}_n$ 和系数向量 $\mathbf{w}$ 是 $d+1$ 维。

那么，多维下的预测可以写成

$$\begin{aligned} t_n &= \omega_0 + \omega_1 x_{1,n} + \omega_2 x_{2,n} + \omega_3 x_{3,n} + \cdots + \omega_d x_{d,n} = \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \vdots \\ \omega_d \end{bmatrix}^T \begin{pmatrix} 1 \\ x_{1,n} \\ \vdots \\ x_{d,n} \end{pmatrix} \\ &= \mathbf{w}^T \mathbf{x}_n \end{aligned}$$



第四章：线性回归算法

线性建模-多维问题：模型的矩阵求解

现将 N 个样本的属性向量 $\mathbf{x}_n = [x_{1,n}, x_{2,n}, \dots, x_{d,n}]^T$ 合并成一个矩阵 $\mathbf{X}$

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{pmatrix} = \begin{pmatrix} x_{1,1}, x_{2,1}, \dots, x_{d,1} \\ x_{1,2}, x_{2,2}, \dots, x_{d,2} \\ \vdots \\ x_{1,N}, x_{2,N}, \dots, x_{d,N} \end{pmatrix} \in \mathbb{R}^{N \times d}$$

也将 N 个样本的目标 t_n 也合并成一个向量

$$\mathbf{t} = (t_1, t_2, \dots, t_N)^T$$

对于{健康、患病}两种响应，通常可以用+1和-1表示目标 $\mathbf{t}$ 。



第四章：线性回归算法

线性建模-多维问题：模型的矩阵求解

损失函数

$$L = \frac{1}{N} (\mathbf{X}\mathbf{w} - \mathbf{t})^T (\mathbf{X}\mathbf{w} - \mathbf{t}) = \frac{1}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - \frac{2}{N} \mathbf{w}^T \mathbf{X}^T \mathbf{t} + \frac{1}{N} \mathbf{t}^T \mathbf{t}$$

通过计算 $\frac{\partial L}{\partial \mathbf{w}} = \begin{bmatrix} \frac{\partial L}{\partial \omega_1} \\ \frac{\partial L}{\partial \omega_2} \\ \vdots \\ \frac{\partial L}{\partial \omega_d} \end{bmatrix} = \frac{2}{N} \mathbf{X}^T \mathbf{X} \mathbf{w} - \frac{2}{N} \mathbf{X}^T \mathbf{t} = 0$

可以推出

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{t}$$

那么 $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}$ （完毕）



第四章：线性回归算法

线性建模-多维问题：模型的矩阵求解

举例：设有4个患者，测量**血压值**、**心率**、**体重**三个属性。

现将4个样本的属性向量 $\mathbf{x}_n = [x_{1,n}, x_{2,n}, x_{3,n}]^T$ 合并成一个矩阵 $\mathbf{X}$

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \mathbf{x}_3^T \\ \mathbf{x}_4^T \end{pmatrix} = \begin{pmatrix} x_{1,1}, x_{2,1}, x_{3,1} \\ x_{1,2}, x_{2,2}, x_{3,2} \\ x_{1,3}, x_{2,3}, x_{3,3} \\ x_{1,4}, x_{2,4}, x_{3,4} \end{pmatrix} = \begin{pmatrix} 80, 70, 50 \\ 150, 100, 70 \\ 100, 80, 60 \\ 60, 120, 40 \end{pmatrix} \in \mathbb{R}^{4 \times 3}$$

也将 N 个样本的目标 t_n 也合并成一个向量

$$\mathbf{t} = (t_1, t_2, t_3, t_4)^T = (1, -1, 1, -1)^T$$

对于{健康、患病}两种响应，分别用+1和-1表示。

第四章：线性回归算法

线性建模-多维问题：模型的矩阵求解

一般， $\mathbf{X}$ 中的数值很大，需要对属性值进行归一化处理，以便于求解和计算。归一化方法，通常除以属性最大值实现。

$$\mathbf{X} = \begin{pmatrix} 80, 70, 50 \\ 150, 100, 70 \\ 100, 80, 60 \\ 60, 120, 40 \end{pmatrix} \rightarrow \begin{pmatrix} \frac{80}{150}, \frac{70}{120}, \frac{50}{70} \\ \frac{150}{150}, \frac{100}{120}, \frac{70}{70} \\ \frac{100}{150}, \frac{80}{120}, \frac{60}{70} \\ \frac{60}{150}, \frac{120}{120}, \frac{40}{70} \end{pmatrix} \rightarrow \begin{pmatrix} \frac{8}{15}, \frac{7}{12}, \frac{5}{7} \\ 1, \frac{5}{6}, 1 \\ \frac{2}{3}, \frac{2}{3}, \frac{6}{7} \\ \frac{2}{5}, 1, \frac{4}{7} \end{pmatrix}$$

利用解析形式 $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}$ ，很容易计算出模型的参数 $\mathbf{w}$



第四章：线性回归算法

普通最小二乘线性回归总结：

- 模型的最优解，可以通过最小化误差的平方和（损失函数）进行计算。

$$L = \frac{1}{N} \sum_{n=1}^N (t_n - h(x_n; \omega_0, \omega_1))^2$$

- 最小二乘法回归算法存在闭解（解析解、精确解）

$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}$ ，其中 $\mathbf{X}$ 是样本数据矩阵， $\mathbf{t}$ 是目标值矢量。



第四章：线性回归算法

普通最小二乘线性回归与极大似然估计的关系？

最小二乘解即为极大似然解！



第四章：线性回归算法

几个概念回顾：

大数定律(伯努利)

其通俗表示：当样本足够多时，样本均值收敛于数学期望（依概率收敛）。

1) 伯努利大数定律

描述：**N**次伯努利试验中，事件**1**发生的频次为**n**，而事件**1**每次发生的概率为**p**，则对任意小的正数 **ε** ，有

$$\lim_{N \rightarrow \infty} P\left(\left|\frac{n}{N} - p\right| < \varepsilon\right) = 1$$

通俗的表述：当执行无穷次试验，事件**1**发生频率几乎接近其发生的概率。
（抛硬币试验即是伯努利试验，只有发生或不发生）



第四章：线性回归算法

几个概念回顾：

大数定律(伯努利)

其通俗表示：当样本足够多时，样本均值收敛于数学期望（依概率收敛）。

2) 辛钦大数定律

描述：设 $\{X_i, i = 1, \dots, N\}$ 中为独立同分布的随机变量序列，若期望存在为 μ ，则服从大数定律，有

$$\lim_{N \rightarrow \infty} P \left(\left| \frac{1}{N} \sum_i X_i - \mu \right| < \varepsilon \right) = 1$$

通俗的表述：当 N 趋于无穷大时，序列的均值几乎接近其期望值。



第四章：线性回归算法

几个概念回顾：

大数定律(伯努利)

其通俗表示：当样本足够多时，样本均值收敛于数学期望。

3) 切比雪夫大数定律

描述：与辛钦大数定律类似，区别在于其并不需要假设 $\{X_i, i = 1, \dots, N\}$ 为同分布的随机变量序列。

更加具有一般性。



第四章：线性回归算法

几个概念回顾：

中心极限定理(棣莫佛)

考虑独立同分布的随机变量 $Y_1, Y_2, \dots, Y_n$ 的集合，它们服从一任意的概率分布(均匀分布、指数分布等)，均值为 μ ，有限方差为 σ^2 。定义样本均值

$$\bar{Y}_n = \frac{1}{n} \sum_{i=1}^n Y_i$$

则当 $n \rightarrow \infty$ 时， $\frac{\bar{Y}_n - \mu}{\frac{\sigma}{\sqrt{n}}}$ ，服从一正态分布 $N(0,1)$ ，均值为0且标准差为1。

也即： $\bar{Y}_n \sim N(\mu, \frac{\sigma^2}{n})$

问题：均值为何为0？即 $n \rightarrow \infty, \bar{Y}_n = \mu$ ？



第四章：线性回归算法

普通最小二乘线性回归与极大似然估计的关系：

□ 最小二乘实质是极大似然解。

已知 N 个训练样本的集合 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n), \dots, (x_N, y_N)$ 。假设样本概率分布之间是相互条件独立的。

那么联合似然概率：

$$P(y|x; w) = \prod_{n=1}^N P(y_n|x_n; w) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y_n - w^T x_n)^2}{2\sigma^2}}$$

极大似然估计，就是找出使得样本集的似然概率最大的一组参数 w ；

如何理解这句话？



第四章：线性回归算法

普通最小二乘线性回归与极大似然估计的关系：

□ 最小二乘实质是极大似然解。

根据之前的模型，我们希望预测值与真实值之间的误差尽可能小。令：

$$y_n = w^T x_n + \xi_n$$

其中， ξ_n 是一个随机变量，假设 $\xi_n \sim \mathcal{N}(0, \sigma^2)$ （大数定律和中心极限定理）。

那么 $y_n \sim \mathcal{N}(w^T x_n, \sigma^2)$ ，因此：

$$P(y_n | x_n; w) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y_n - w^T x_n)^2}{2\sigma^2}}$$



第四章：线性回归算法

普通最小二乘线性回归与极大似然估计的关系：

□ 最小二乘实质是极大似然解(极大似然估计MLE: Maximum likelihood estimation)。

$$\max_w L(w) = P(y|x; w) = \prod_{n=1}^N P(y_n|x_n; w) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y_n - w^T x_n)^2}{2\sigma^2}}$$

上述最大化问题等效于似然函数 $L(w)$ 的对数！



第四章：线性回归算法

普通最小二乘线性回归与极大似然估计的关系：

□ 最小二乘实质是极大似然解(极大似然估计MLE: Maximum likelihood estimation)。

$$\begin{aligned} \max_w L(w) &\triangleq \max_w \log L(w) \\ &= \log \prod_{n=1}^N P(y_n | x_n; w) = \log \prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y_n - w^T x_n)^2}{2\sigma^2}} = \sum_{n=1}^N \log \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y_n - w^T x_n)^2}{2\sigma^2}} \\ &\Leftrightarrow \min_w \sum_{n=1}^N \left((y_n - w^T x_n)^2 \right) \quad \text{最小二乘} \end{aligned}$$



第四章：线性回归算法

存在的问题：当 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 非列满秩时（且 $N < D$ ）， $\mathbf{X}^T \mathbf{X}$ 是奇异矩阵，那么计算 $(\mathbf{X}^T \mathbf{X})^{-1}$ 会产生较大误差，变成了一个不适定问题(病态问题，欠定)，该模型缺乏稳定性（对噪声的敏感性很强）！

奇异矩阵 $\mathbf{A}$ ： $|\mathbf{A}|=0$ ；矩阵 $\mathbf{A}$ 是非满秩矩阵；
非奇异矩阵 $\mathbf{A}$ ： $|\mathbf{A}| \neq 0$ ；矩阵 $\mathbf{A}$ 是满秩矩阵。



第四章：线性回归算法

欠定问题、超定问题：

对于数据矩阵 $X \in \mathbb{R}^{N \times d}$, d 为属性维度, N 为样本点数。

- 所谓欠定问题, 即不适定问题, 是由于样本点数目少于变量的数目, 即 X 不是列满秩的, 解不唯一 (需要增加限定条件, 即先验知识) ;
- 超定问题, 是样本点数目大于变量的数目, 且 X 为列满秩。
对于超定方程问题, 解不存在, 但存在近似解。最小二乘是用于解决超定问题, 获得最佳近似解。



第四章：线性回归算法

模型的性能评价

已经介绍了普通最小二乘法的原理，对于由 N 个样本，计算出的最小二乘模型参数 w ，如何说明该模型是好的呢？

- 一般地，在机器学习中，为了验证模型的好坏，还需要一组新的样本（测试样本集）进行测试，根据测试准确率，判定模型的好坏。注：测试样本集是独立于 N 个样本的（也称训练样本）。
- 交叉验证（Leave-one-out cross validation, LOO）：K折交叉验证，也是目前机器学习和人工智能领域具有代表性的验证模型泛化能力的训练方法。



第四章：线性回归算法

模型的性能评价

K折交叉验证（Leave-one-out cross validation, LOO）（留一法）：

将数据集分成相同数量的K份（K折），每份数据分别轮流作为测试集，其余的K-1份作为训练集。执行K次测试的平均精度，作为最后的模型精度。

特别注意的是：当 $K=N$ 时，N折交叉验证，变成了“留一法”，即每个样本分别轮流作为测试样本（测试集中只有一个样本），剩余的N-1个样本作为训练集。

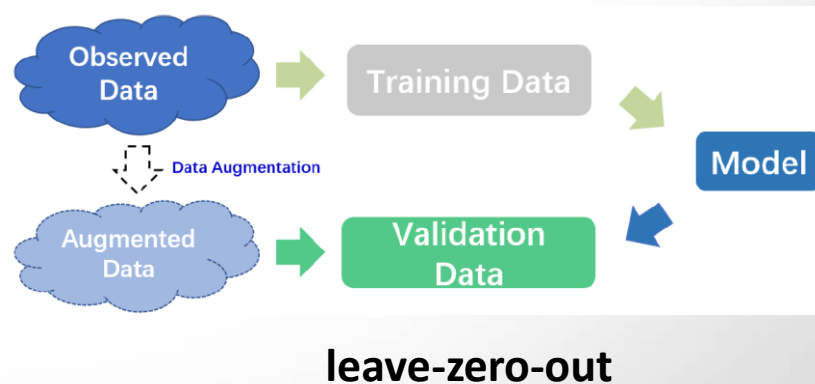
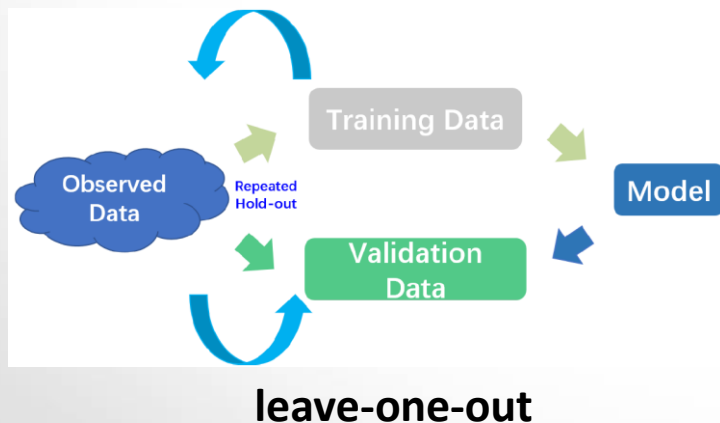
为什么要提出模型性能的评价方式？？

第四章：线性回归算法

模型的性能评价

某些领域（医学）数据十分昂贵，如果仅有的少量数据还要拿出一些数据作为验证集，实属浪费。

From leave-one-out to leave-zero-out (data augmentation)





第四章：线性回归算法

广义线性回归

给定一组样本： $\langle \mathbf{x}_i, y_i \rangle_{i=1 \dots m}$ ，我们要拟合下面的假设

$$h_{\mathbf{w}}(\mathbf{x}) = \sum_{k=0}^{K-1} w_k \phi_k(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

其中， ϕ_k 为基函数，可以是多阶的函数，比如平方、立方等。
为了求最佳的预测系数 $\mathbf{w}$ ，我们需要最小化下列损失函数

$$\sum_{i=1}^m (y_i - h_{\mathbf{w}}(\mathbf{x}_i))^2$$

那么最优解 $\mathbf{w}$ 可以写为

$$\mathbf{w} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$



第四章：线性回归算法

广义线性回归

- 由前面的分析可知， $h_w(\mathbf{x})$ 对于参数 w 来说，是线性模型。但并不意味着， $h_w(\mathbf{x})$ 是关于输入 $\mathbf{x}$ 的线性函数，比如 $h_w(\mathbf{x}) = w\mathbf{x}^2$ 。(多项式回归)
- 因此，广义的线性模型可以写成

$$h_w(\mathbf{x}) = \sum_{k=0}^{K-1} w_k \phi_k(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

其中， ϕ_k 为基函数。

- 线性假设模型还可以重新写成

$$h_w(\mathbf{x}) = \boldsymbol{\Phi} \mathbf{w}$$

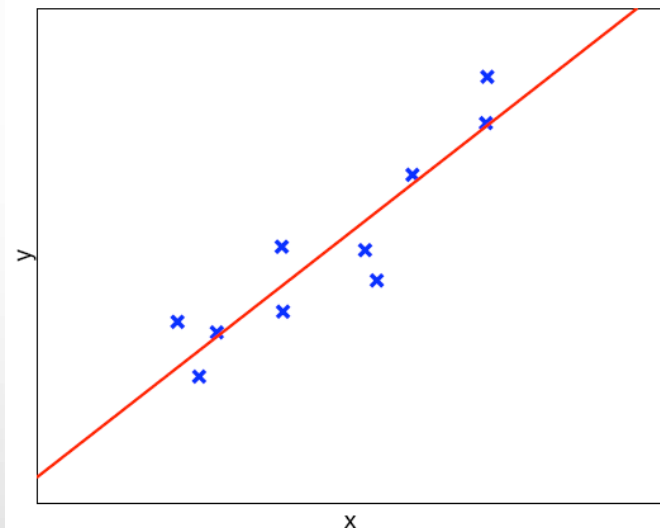
关于 w 的
线性模型

其中， $\boldsymbol{\Phi}$ 是由 $\boldsymbol{\phi}(\mathbf{x}_j)$ 构成的矩阵。

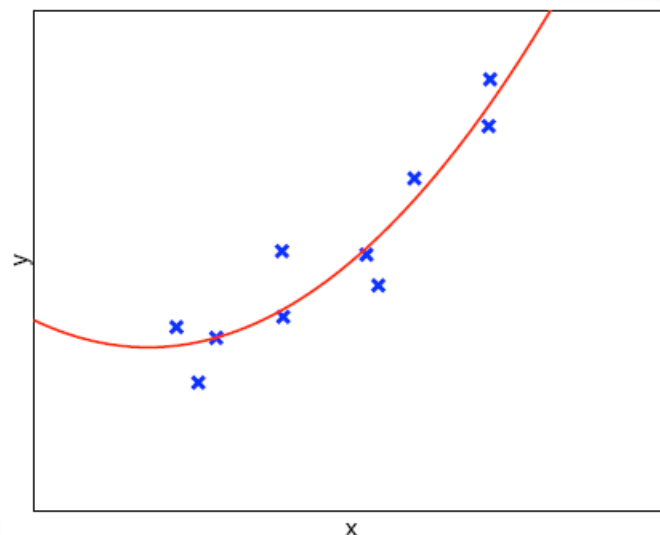
第四章：线性回归算法

广义线性回归

采用不同基函数（不同阶次）的回归模型效果



1阶回归 (is it better to fit the data?)

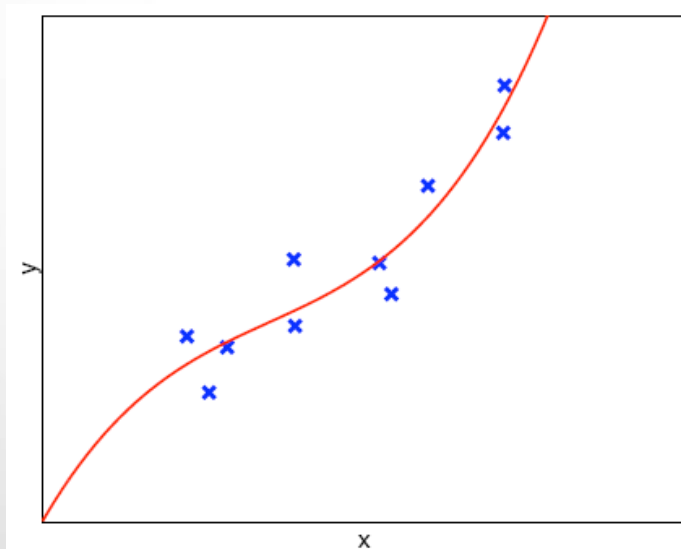


2阶回归

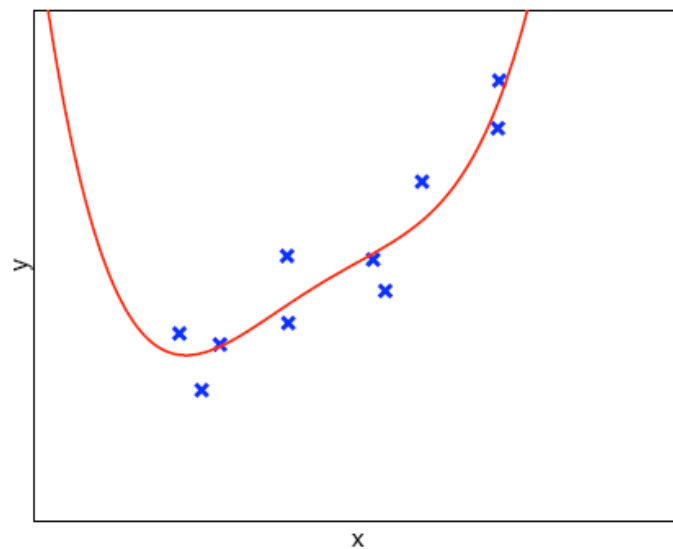
第四章：线性回归算法

广义线性回归

采用不同基函数（不同阶次）的回归模型效果



3阶回归 (is it better to fit the data?)

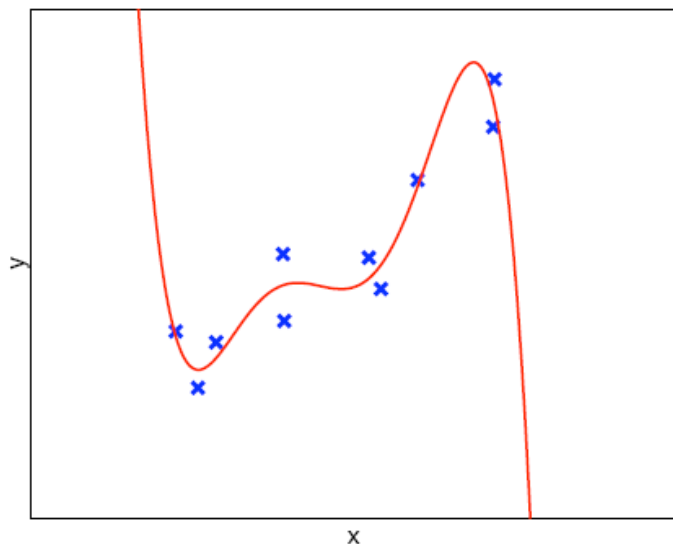


4阶回归

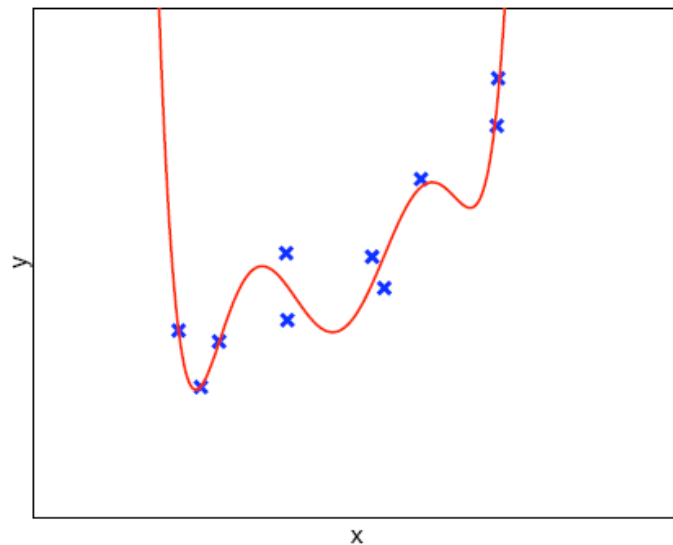
第四章：线性回归算法

广义线性回归

采用不同基函数（不同阶次）的回归模型效果



5阶回归 (is it better to fit the data?)

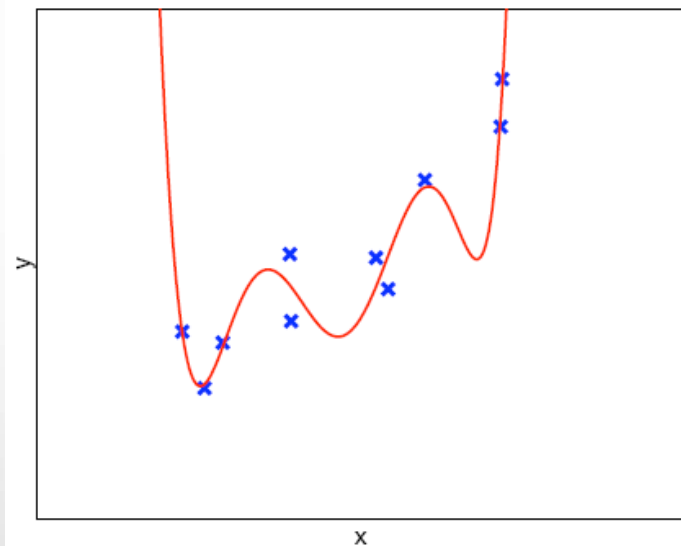


6阶回归

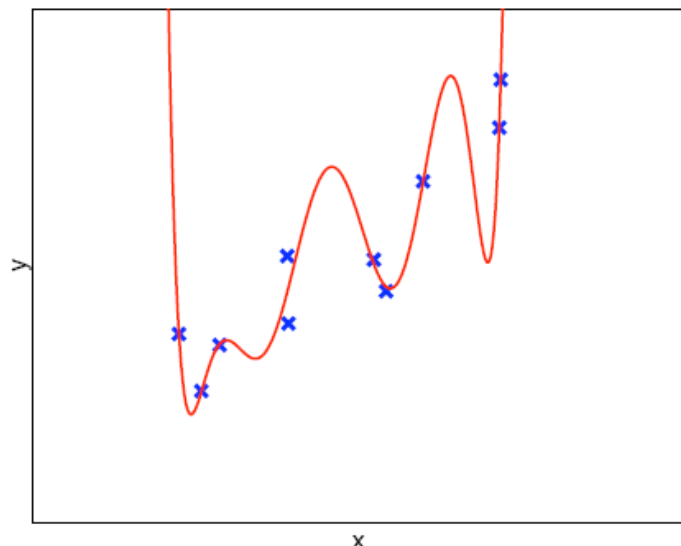
第四章：线性回归算法

广义线性回归

采用不同基函数（不同阶次）的回归模型效果



7阶回归 (is it better to fit the data?)

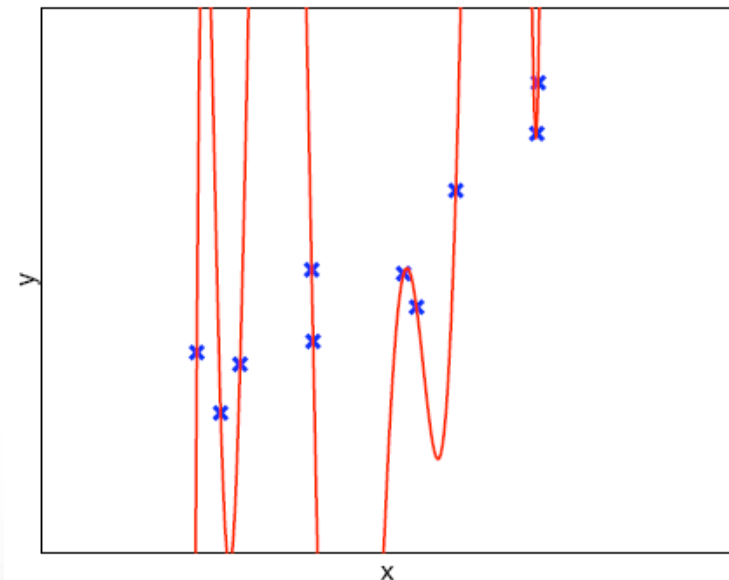


8阶回归

第四章：线性回归算法

广义线性回归

采用不同基函数（不同阶次）的回归模型效果



9阶回归 (is it better to fit the data?)



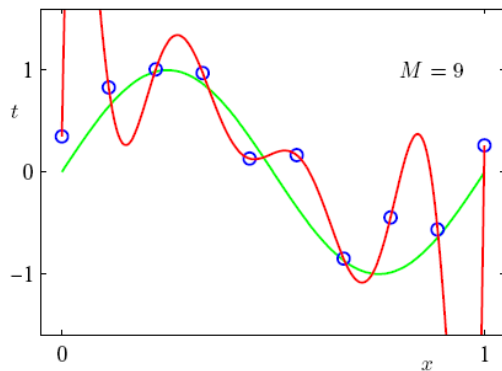
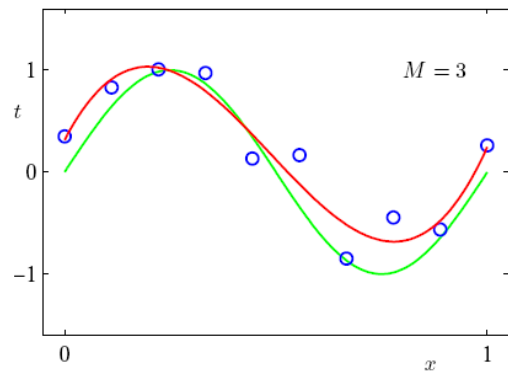
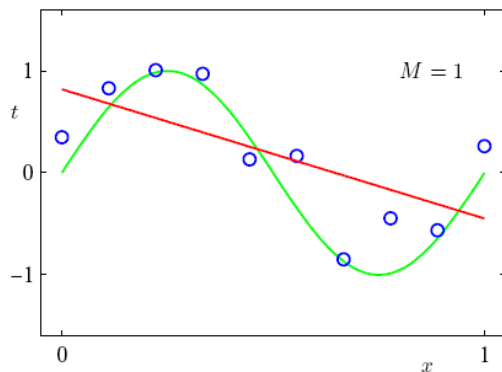
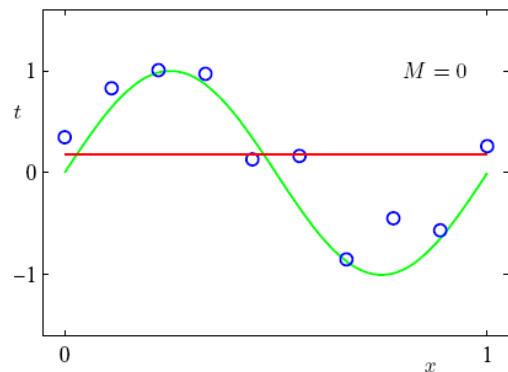
第四章：线性回归算法

过拟合（overfitting）

- 一般来说，过拟合是所有机器学习算法中的一个非常重要的问题；
- 通常，我们能够找到一种完美的假设，准确无误的预测训练数据。但是，对于新数据却不能够很好的预测（泛化能力很差）。
- 通过刚才的例子看到，如果参数越来越多，模型倾向于“记忆”训练数据点，而不是“推理”某种规律。

第四章：线性回归算法

过拟合（overfitting）与欠拟合（underfitting）

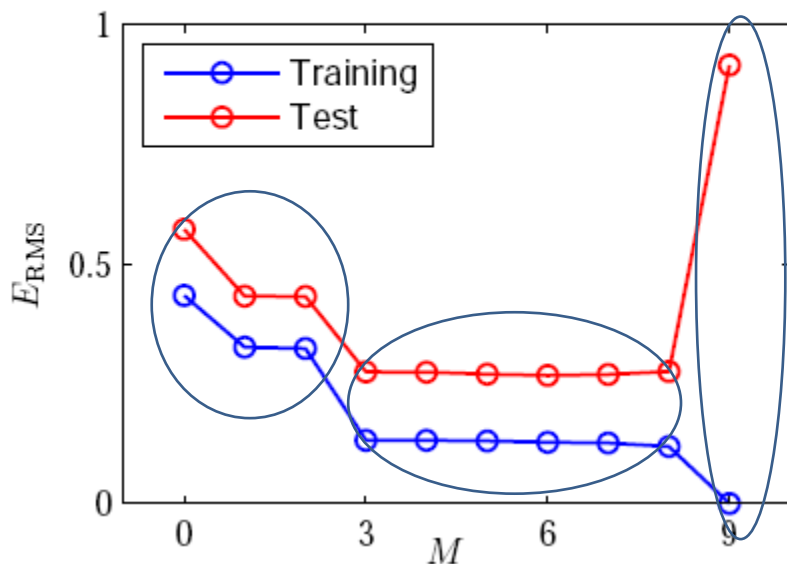


- M 为模型的阶数，反映模型的自由度和复杂度。
- 过拟合是指，训练样本的误差非常低，而新测试样本的预测误差非常大。
- 欠拟合是指，训练样本的预测误差非常高（学习不够）。
- 通常，过拟合是更为常见的问题，因此，在建模过程中，我们需要经验和理论分析，来避免这个问题。

第四章：线性回归算法

过拟合（overfitting）与欠拟合（underfitting）

Typical overfitting plot



- ❑ 从图中可以看出， $M=0,1,2$ 时，属于欠拟合（训练误差较大）
- ❑ $M=3,4,5,6,7,8$ 时，属于拟合情况良好；但根据模型的几个准则（参数越少、模型越简单越好），显然 $M=3$ 是最佳参数。
- ❑ $M=9$ 时，属于过拟合。（训练误差非常小，接近0，但测试误差非常大）



第四章：线性回归算法

在设计模型寻求某种假设时，如何防止过拟合（**overfitting**）与欠拟合（**underfitting**）？

非常有效的交叉验证策略：

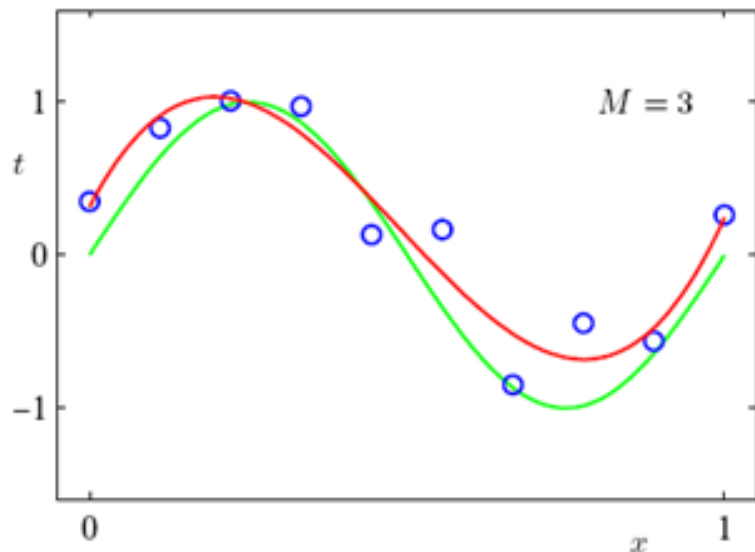
□ 将整个数据集分成3个**独立的**部分：训练集、验证集、测试集

- ✓ 训练集是用于训练某种假设；
- ✓ 验证集用于检验假设的可靠性，并进一步修正模型（即 M ）；
- ✓ 测试集用于检验最终模型假设的性能。

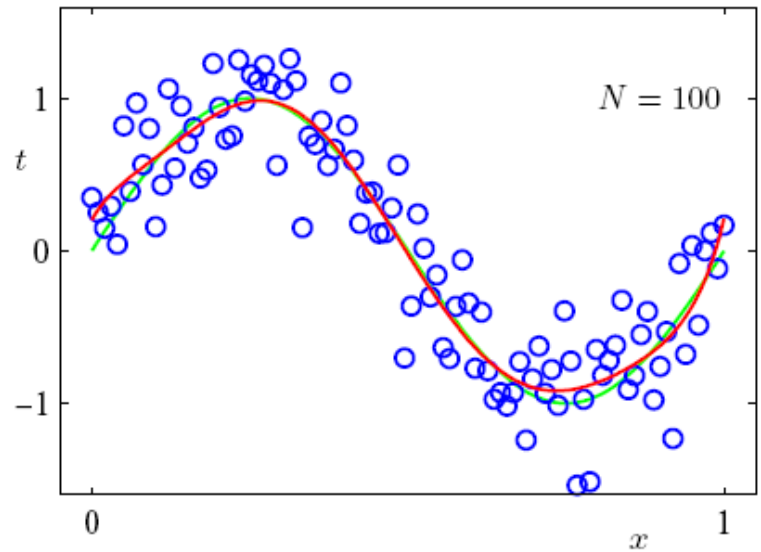
一般地，采用随机分割的形式，执行以上程序 N 次，每次测试集的准确率的平均值为最终的模型的性能评价指标。

第四章：线性回归算法

最佳的假设 $h(x)$ 依赖于训练样本数量和模型复杂度



10个训练样本的回归曲线 ($M=3$)
红色表示假设曲线, 绿色为真实值

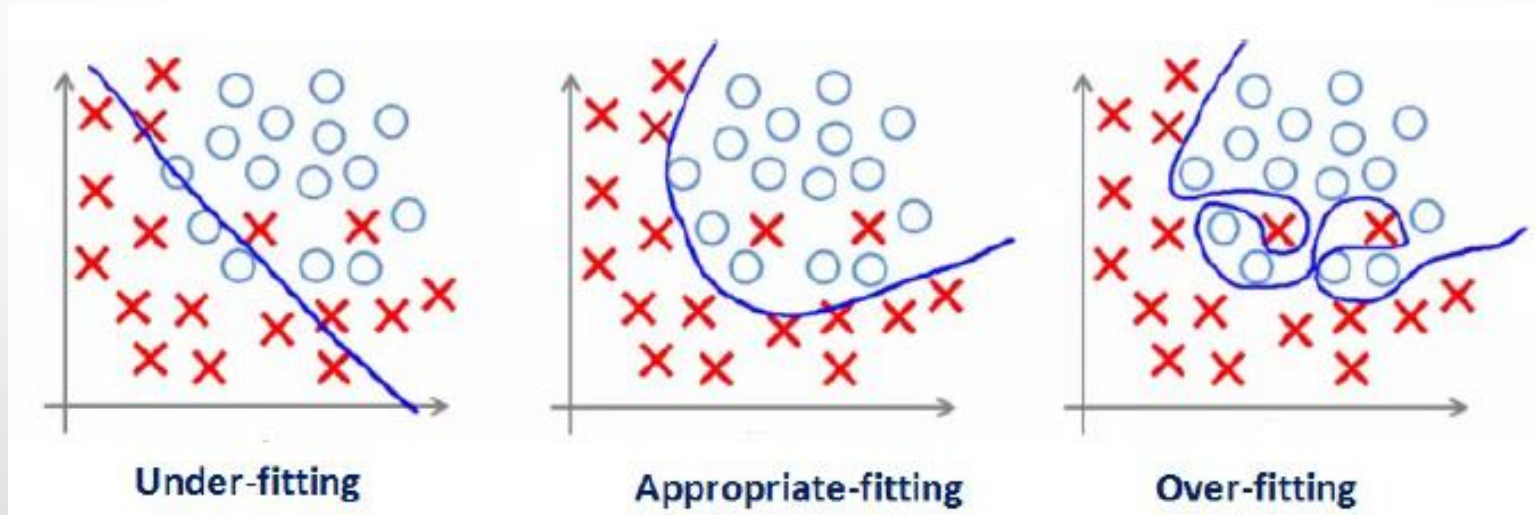


100个训练样本的回归曲线 ($M=3$)
假设与真实更加接近

第四章：线性回归算法

最佳的假设 $h(x)$ 依赖于训练样本数量和模型复杂度

分类问题：





第四章：线性回归算法

对假设的精度进行评估是机器学习的基本问题。

然而，由于有限的数据样本一般不代表数据的一般分布，所以利用有限样本得出的假设精度可能存在误差。

我们将介绍一种统计的方法，结合有关数据基准分布的假定，我们可以用有限数据样本的观察精度来逼近整个数据分布的真实精度。



第四章：线性回归算法

评估假设

一、样本错误率

样本错误率：该假设在可见数据 $\mathbf{s}$ 上的错误率；

定义：假设 h 关于目标值和可见数据 $\mathbf{s}$ 的**样本错误率**为

$$error_S(h) = \frac{1}{n} \sum_{x \in S} \delta(f(x), h(x))$$

其中， n 为样本集 $\mathbf{s}$ 的数量。如果 $f(x) \neq h(x)$ ， $\delta(f(x), h(x)) = 1$ ，
否则 $\delta(f(x), h(x)) = 0$

可类比抛掷硬币问题中计算硬币出现正面的概率： n 个样例中假设出错的次数，等价于抛掷 n 次硬币出现正面的次数（可见样本的错误率）



第四章：线性回归算法

评估假设

二、真实错误率

真实错误率：该假设在分布为 D 的整个实例集合上的错误率；

定义：假设 h 关于目标值和分布为 D 的**真实错误率**，为 h 按 D 分布**随机抽取的实例被误分类**的概率

$$error_D(h) = Pr_{x \in D}[f(x) \neq h(x)]$$

问题： $error_S(h)$ 在何种程度上提供了对 $error_D(h)$ 的估计？

该真实错误率估计问题等价于估计抛出一枚硬币出现正面的概率 p （即，随机抽取的实例被错误预测的概率 $error_D(h)$ ）；



第四章：线性回归算法

评估假设

三、真实错误率的估计

问题： $error_S(h)$ 在何种程度上提供了对 $error_D(h)$ 的估计？

考虑假设 h 为离散的情况，利用假设 h 在 S 上观察到的 **样本错误率** 估计 **真实错误率**。举例：

- 样本集 S 包含 n 个样本，其抽取方式按照概率分布 D 抽样，且相互独立；
- 样本数 $n \geq 30$
- 假设 h 在这 n 个样例上犯错次数为 r 。显然， $error_S(h) = r/n$

统计发现，当重复上述实验多次(每次抽取 n 个实例)，随机变量 $error_S(h)$ 呈现出二项分布；
二项分布刻画的是抛掷硬币 n 次出现 r 次正面的概率，即 n 个随机样例中出现 r 次预测错误的概率。



第四章：线性回归算法

二项分布：

一个二项分布刻画了对于一个出现正面概率为 p 的硬币，独立投掷 n 次中观察到 r 次正面的概率。

给定一个随机变量 X 服从二项分布， $X=r$ 的概率表达式为

$$P(X = r) = \frac{n!}{r!(n-r)!} p^r (1-p)^{n-r}$$

X 的期望 $E(X) = np$

X 的方差 $Var(X) = np(1-p)$

n 次伯努利实验：指“是/否”的决策

期望的定义：考虑随机变量 X 的可能取值为 $x_1, \dots, x_n$ ，则 X 的期望值 $E[X]$ 为

$$E[X] = \sum_{i=1}^n x_i P(X = x_i)$$

方差的定义：

$$Var[X] = E[(X - E[X])^2]$$



第四章：线性回归算法

评估假设

(1)考虑假设 h 的样本错误率 $error_S(h)$ ，即 n 个随机样例组成的集合 S 中，出现 r 次犯错的比率：

$$error_S(h) = \frac{r}{n}$$

其中，随机变量 $r \sim B(n, p)$ ，均值 $E[r] = np$ ，方差 $Var(r) = np(1 - p)$

(2) $error_S(h)$ 是一个二项分布，其均值和标准差为

$$E[error_S(h)] = \frac{E[r]}{n} = p$$
$$\sigma[error_S(h)] = \frac{\sqrt{Var(r)}}{n} = \sqrt{\frac{p(1 - p)}{n}}$$

可以发现：样本错误率的期望正是真实错误率 p (即无偏估计， $E(Y)-p=0$)。 p 的近似值为 r/n



第四章：线性回归算法

评估假设

三、真实错误率的估计

问题： $error_S(h)$ 在何种程度上提供了对 $error_D(h)$ 的估计？

真实错误率以概率 P 的可能性落在下面的区间内

$$error_D(h) = error_S(h) \pm z_P \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}}$$

其中 z_P 为双侧的 P 置信区间的常量。对于正态分布，当 $P=95\%$ 时， $z_P = 1.96$ ；当 $P=90\%$ 时， $z_P = 1.64$ 。

统计意义：当从 D 中随机抽取多个样本测试时，会发现 95% 的实验计算所得区间包含真实错误率。



第四章：线性回归算法

置信区间

通常描述某个估计的不确定性的方法是使用置信区间，真实的值（真实错误率）以一定的概率落入该区间中。因此，**估计真实错误率的问题转化为估计置信区间的问题**。

定义：某个参数 q 的 $N\%$ 置信区间是一个以 $N\%$ 的概率包含 q 的区间。

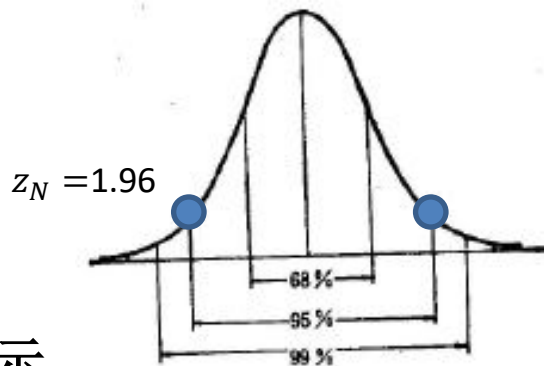
如果要估计 $error_D(h)$ 的置信区间，只需要找到以 $error_D(h)$ 为中心的区间，区间的宽度需要保证 $error_S(h)$ 能有95%的机会落入其中。

对于二项分布来说，计算置信区间比较繁琐。当样本数 n 足够大时（研究表明 $np(1-p)>5$ 时），二项分布可以近似为正态分布。而正态分布的置信区间有统计列表做指导。

第四章：线性回归算法

置信区间

正态分布的置信区间（双侧）：



置信区间的半宽度利用常数 z_N 和标准差表示。

z_N : 包含N%概率质量的关于均值的最小区间的宽度。

总之，如果随机变量Y服从 $\mathcal{N}(\mu, \sigma^2)$ ，那么Y的任意一个观察值y有N%的机会（概率）落入下面的区间：

$$\mu \pm z_N \sigma$$

双侧情况

置信度N%	50%	68%	80%	90%	95%	98%	99%
常量 z_N	0.67	1.00	1.28	1.64	1.96	2.33	2.58



第四章：线性回归算法

评估假设

三、真实错误率的估计

问题： $error_S(h)$ 在何种程度上提供了对 $error_D(h)$ 的估计？

真实错误率以概率 p 的可能性落在下面的区间内

$$error_D(h) = \mu \pm z_N \sigma$$

利用近似 $p=r/n$
 $= error_S(h)$

$$= p \pm z_N \sqrt{\frac{p(1-p)}{n}}$$
$$error_D(h) = error_S(h) \pm z_p \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}}$$

其中， z_p 为双侧的 p 置信区间的常量。当 $p=95\%$ 时， $z_p = 1.96$ ；当 $p=90\%$ 时， $z_p = 1.64$ 。

推导中的两个近似：

1. $error_D(h)$ 近似为 $error_S(h)$ ，当 $n \geq 30$ 时，效果较好
2. 二项分布近似为正态分布，当 $np(1-p) \geq 5$ 时，效果较好



第四章：线性回归算法

评估假设

三、真实错误率的估计

问题： $error_S(h)$ 在何种程度上提供了对 $error_D(h)$ 的估计？

举例：当数据集 S 包含 40 个样例时，产生了 $r=12$ 个错误，对于置信水平 $p=0.95$ 时，则有：

$$\begin{aligned} error_S(h) &= \frac{12}{40} = 0.3 \\ error_D(h) &= error_S(h) \pm z_p \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}} \\ &= 0.3 \pm 1.96 \sqrt{\frac{0.3(1 - 0.3)}{40}} \\ &= 0.3 \pm 0.14 \end{aligned}$$



第四章：线性回归算法

评估假设

四、算法假设评估与检验

问题：对于两个算法A和B，其获得的假设分别为h1和h2，使用大小为100的独立样本集S，且知道 $error_S(h1) = 0.3$, $error_S(h2) = 0.2$ 。那么，“ $error_D(h1) > error_D(h2)$ 的可能性有多大？”

分析：上述问题可转化为

$d = error_D(h1) - error_D(h2) > 0$ 的概率有多大？



第四章：线性回归算法

评估假设

四、算法假设评估与检验

考虑估计量：

$$\hat{d} = error_S(h1) - error_S(h2) = 0.3 - 0.2 = 0.1$$

显然， $\hat{d}$ 是 d 的无偏估计量，即 $E[\hat{d}] = d$ 。

根据前面所讲内容， $error_S(h1)$ 和 $error_S(h2)$ 近似为正态分布，那么 $\hat{d}$ 也服从正态分布。有：

$$\sigma_{\hat{d}} = \sqrt{\frac{error_S(h1)(1 - error_S(h1))}{n} + \frac{error_S(h2)(1 - error_S(h2))}{n}}$$



第四章：线性回归算法

评估假设

四、算法假设评估与检验

那么， d 的 $N\%$ 双侧置信区间估计为：

$$\hat{d} \pm z_N \sigma_{\hat{d}}$$

回到刚才的问题：“ $error_D(h1) > error_D(h2)$ 的可能性有多大？”

即，“ $d = error_D(h1) - error_D(h2) > 0$ 的概率有多大？”

问题转化：已知 $\hat{d} = 0.1$, 实际上就是考查“ $\hat{d} < d + 0.1$ 的概率有多大？”

按照标准的表达式， $\hat{d}$ 落入下列区间的概率（单侧区间）：

$$\hat{d} < \mu_d + z_N \sigma_{\hat{d}}$$

已知 $\sigma_{\hat{d}} = 0.061$ ，那么 $z_N \approx 1.64$ ，查表可知，对应置信度为90%的双侧区间。而对于单侧区间，则有95%的置信度。



第四章：线性回归算法

评估假设

四、算法假设评估与检验

结论：对于观察到的 $\hat{d} = 0.1$, $error_D(h1) > error_D(h2)$ 的可能性有95%。（具有统计显著意义）

统计学术语：接受(accept) “ $error_D(h1) > error_D(h2)$ ” 这一假设的置信度为0.95；

或者：以 $1-0.95=0.05$ 的显著水平，拒绝(reject)对立假设。



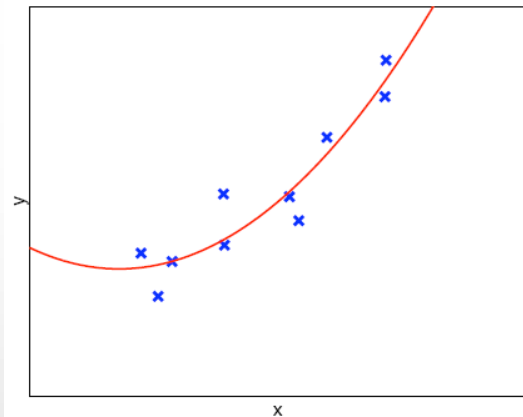
第五章： 正则化

第五章：正则化regularization

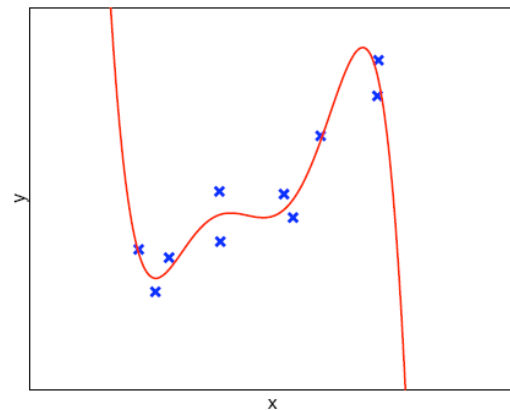
正则化是机器学习中的重要一环，其目的是为了降低**模型复杂度**，增强模型的**抗噪声能力**，防止模型对训练数据集的**过拟合**（过学习）。换句话说，正则化是用于对模型参数的一种约束。

举例：一个2阶和5阶的拟合函数

奥卡姆剃刀定律（Occam's Razor, Ockham's Razor）：大道至简



$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$

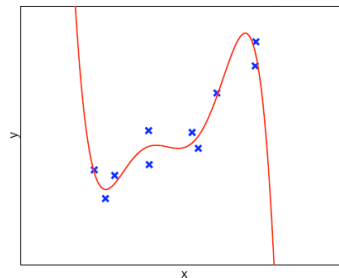


$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4 + \theta_5 x^5$$

第五章：正则化regularization

原线性模型的最小化表达式为

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 \quad \longrightarrow \quad h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4 + \theta_5 x^5$$

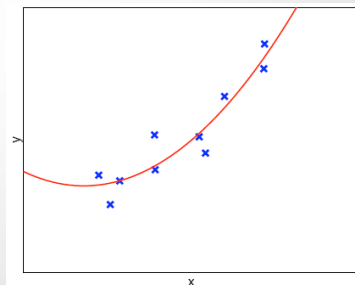


为了获得一个较好的模型，我们希望模型复杂度越小越好（阶数越小），也就是希望得到 $\theta_3, \theta_4, \theta_5 \approx 0$ (参数变少，得到简单的模型假设)
那么模型应当改进如下

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 + 10000 \cdot \theta_3^2 + 10000 \cdot \theta_4^2 + 10000 \cdot \theta_5^2$$



$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$





第五章：正则化regularization

一个好的机器学习模型，既要反映数据中的蕴含信息，又要反映先验信息。

将上述模型写成以下形式：

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 + \lambda \cdot \sum_{j=1}^m \theta_j^2$$

正则化项

正则化参数 λ 是一个可调的参数（误差项和正则项之间的平衡系数），如果该值非常大，则获得的参数就会非常小！容易产生“欠拟合”，因此在算法编程中，可调试该参数。



第五章：正则化regularization

- 1) **正则化**实际上是在经验误差函数上加约束，这种约束的解释就是先验知识，对参数进行正则化等价于对参数引入某种先验分布。
- 2) 正则化解决了逆问题的不适定性，产生的解依赖于数据，而不易受噪声影响。

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 + \lambda \cdot \sum_{j=1}^m \theta_j^2$$



正则化项

实际上,正则化是对模型参数的约束,带有约束的模型可以进一步改写

第五章：正则化regularization

正则化是机器学习也是机器学习中的重要一环，其目的是为了降低**模型复杂度**，增强模型的**抗噪声能力**，防止模型对训练数据集的**过拟合**（过学习）。换句话说，正则化是用于对模型参数的一种约束。

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 + \lambda \cdot \sum_{j=1}^m \theta_j^2$$

正则化项

等价于

$$\begin{aligned} \min_{\theta} \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 \\ \text{s.t. } \sum_{j=1}^m \theta_j^2 \leq \xi \end{aligned}$$

相当于在原模型基础上，加了一个约束项

第五章：正则化regularization

等价性证明

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 + \lambda \cdot \sum_{j=1}^m \theta_j^2$$



$$\begin{aligned} \min_{\theta} \quad & \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 \\ \text{s.t.} \quad & \sum_{j=1}^m \theta_j^2 \leq \xi \end{aligned}$$

对下面的模型利用拉格朗日乘子法构造目标函数

$$\min_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - t_i)^2 + \lambda \cdot \left(\sum_{j=1}^m \theta_j^2 - \xi \right)$$



第五章：正则化regularization

□多维下的线性回归模型表达

定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\boldsymbol{\theta}} \frac{1}{2N} \sum_{i=1}^N \|h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2$$

其中, $\boldsymbol{\theta}$ 是模型参数（向量或矩阵）。



第五章：正则化regularization

□多维下的正则化线性回归模型表达

定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\boldsymbol{\theta}} \frac{1}{2N} \sum_{i=1}^N \|h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2 + \lambda \cdot \|\boldsymbol{\theta}\|_2^2$$

其中, $\boldsymbol{\theta}$ 是模型参数（向量或矩阵）。

与非正则化相比, 目标函数中多了一项, 即在最小化误差的同时, 还要保持参数值比较小。



第五章：正则化regularization

□ 正则化最小二乘模型（岭回归）

定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^N \left\| \boldsymbol{\theta}^T \mathbf{x}_i - \mathbf{y}_i \right\|_2^2 + \lambda \cdot \left\| \boldsymbol{\theta} \right\|_2^2$$

等价于



$$\min_{\boldsymbol{\theta}} \left\| \mathbf{X}^T \boldsymbol{\theta} - \mathbf{Y} \right\|_F^2 + \lambda \cdot \left\| \boldsymbol{\theta} \right\|_F^2$$

其中, $\boldsymbol{\theta}$ 是模型参数（向量或矩阵）。



第五章：正则化regularization

□ 正则化最小二乘模型（岭回归）

定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\boldsymbol{\theta}} \left\| \mathbf{X}^T \boldsymbol{\theta} - \mathbf{Y} \right\|_F^2 + \lambda \cdot \left\| \boldsymbol{\theta} \right\|_F^2$$

模型求解:

令 $J(\boldsymbol{\theta}) = \left\| \mathbf{X}^T \boldsymbol{\theta} - \mathbf{Y} \right\|_F^2 + \lambda \cdot \left\| \boldsymbol{\theta} \right\|_F^2$ 为目标函数。

求导:

$$\frac{dJ(\boldsymbol{\theta})}{d\boldsymbol{\theta}} = \mathbf{X}(\mathbf{X}^T \boldsymbol{\theta} - \mathbf{Y}) + \lambda \cdot \boldsymbol{\theta} = 0 \quad \Rightarrow \quad \boldsymbol{\theta} = (\mathbf{X}\mathbf{X}^T + \lambda \cdot \mathbf{I})^{-1} \mathbf{X}\mathbf{Y}$$

直接写出解析解



第五章：正则化regularization

□多维下的正则化线性回归模型表达

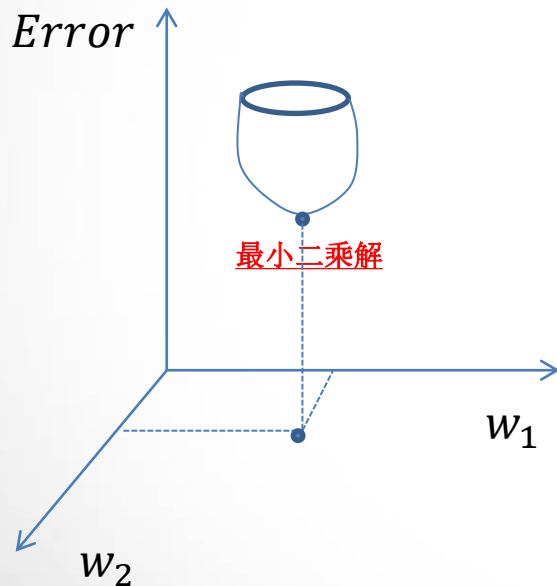
定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\boldsymbol{\theta}} \frac{1}{2N} \sum_{i=1}^N \|h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2 + \lambda \cdot \|\boldsymbol{\theta}\|_2^2$$

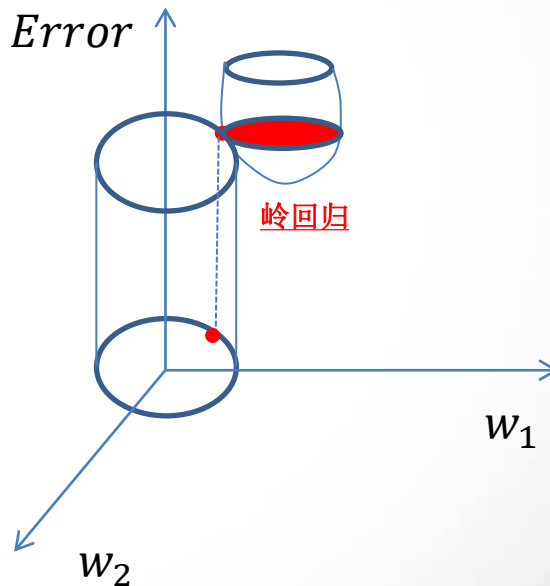
其中, $\boldsymbol{\theta}$ 是模型参数（向量或矩阵）。

注：这里正则化项的设计采用的是L2-norm(Tikhonov regularization), 是一种很平滑的约束, 但并不稀疏。

第五章：正则化regularization

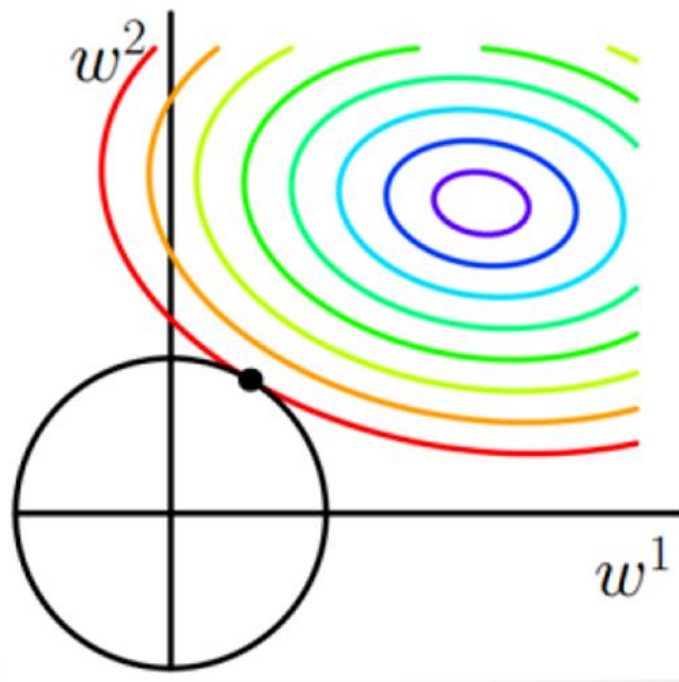
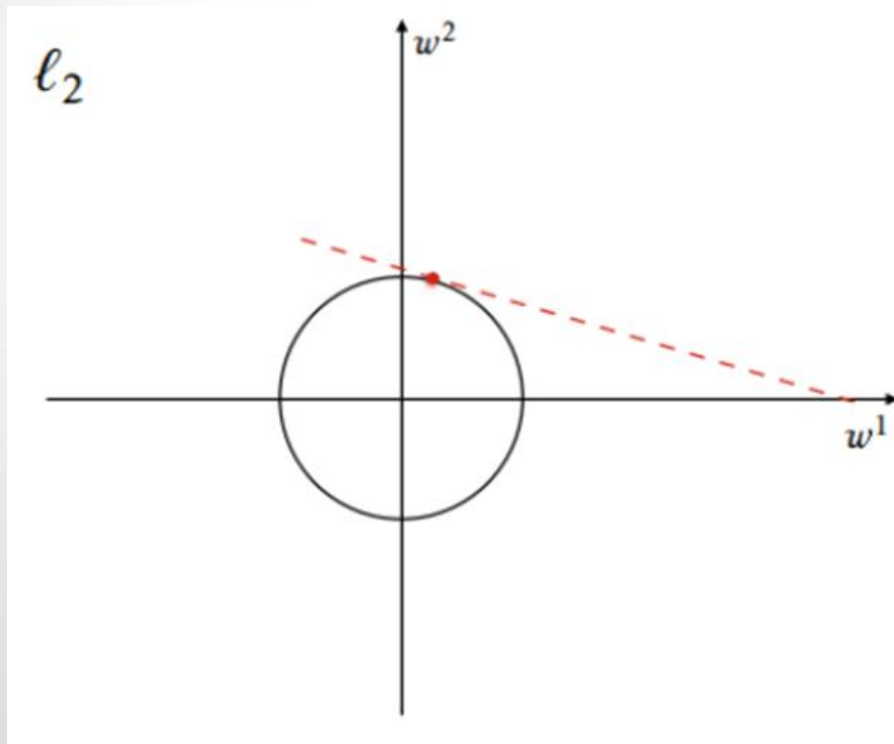


$$w_1^2 + w_2^2 = r^2$$



解的几何分析

第五章：正则化regularization





第五章：正则化regularization

□吉洪诺夫正则化(Tikhonov Regularization, 1943, 1963)

定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\boldsymbol{\theta}} \frac{1}{2N} \sum_{i=1}^N \|h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2 + \lambda \cdot \|\Gamma \boldsymbol{\theta}\|_2^2$$

其中, Γ 是吉洪诺夫矩阵,在许多情况下 $\Gamma = \alpha \cdot \mathbf{I}$ 。

对于L2-Norm,解的稀疏性不好, (如何能保证求解的稀疏性?)



第五章：正则化regularization

□多重共线性问题

多重共线性是指模型中解释变量之间存在精确相关关系或高度相关关系而使模型失真或难以估计。

有效数据是获得精确模型的前提条件。在最小二乘问题中，当多重共线性问题出现时， $|X'X|$ 接近于0，导致奇异发生，从而其矩阵的逆不存在。

在正则化最小二乘问题中，通过引入 λI 可较大程度上降低奇异矩阵的发生概率。

$$\text{若 } X = \begin{bmatrix} 1 & 3 & 4 \\ 2 & 6 & 8 \end{bmatrix}, \text{ 则 } \det(X^T X) = 0$$

数据矩阵 X 非列满秩，如何理解“秩”的意义？



第五章：正则化regularization

□多重共线性问题

特征的选择，即强解释性变量的选择。

采用**稀疏正则化**，对于删除模型中无效变量（导致共线性）或无意义变量具有较好的效果。

在L2范式下的约束中，使得模型的解收缩到一个较小的值，但收缩到0的概率较低。对于无效变量的删除作用不强。



第五章：正则化regularization

□多维下的稀疏正则化线性回归模型表达

定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\theta} \frac{1}{2N} \sum_{i=1}^N \|h_{\theta}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2 + \lambda \cdot \|\theta\|_0$$

L0-范数表示向量 θ 中非零元素的个数。

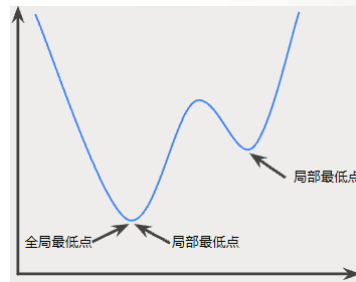
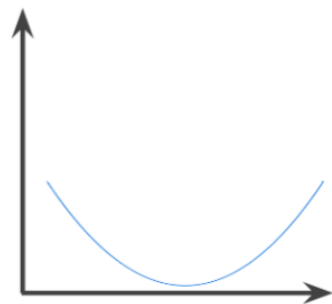
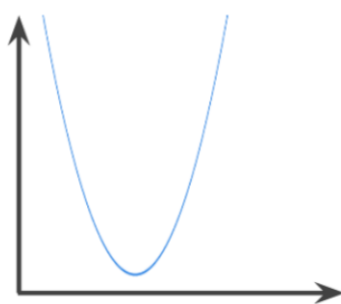
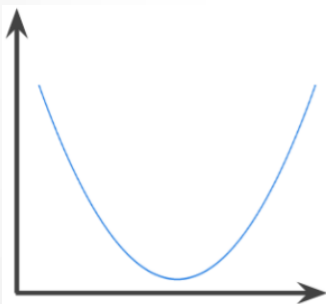
注：这里正则化项的设计采用的是**L0-norm**,是一种强稀疏约束。

但求解时，由于**L0**的非凸性，难以求解**NP-hard**。通常采用**松弛化**。

第五章：正则化regularization

□基本概念

凸函数、凸优化、凸集



$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$$

Jensen不等式（凸函数的充要条件）

若 $f(x)$ 是 $[a,b]$ 的凸函数，对于区间内的任意 $x_1, x_2, \dots, x_n$ ，有

$$f\left(\sum_{i=1}^n t_i x_i\right) \leq \sum_{i=1}^n t_i f(x_i), \text{ where } \sum_{i=1}^n t_i = 1$$



第五章：正则化regularization

Least absolute shrinkage and selection operator, Lasso

□多维下的稀疏正则化线性回归模型表达 (Lasso Regression)

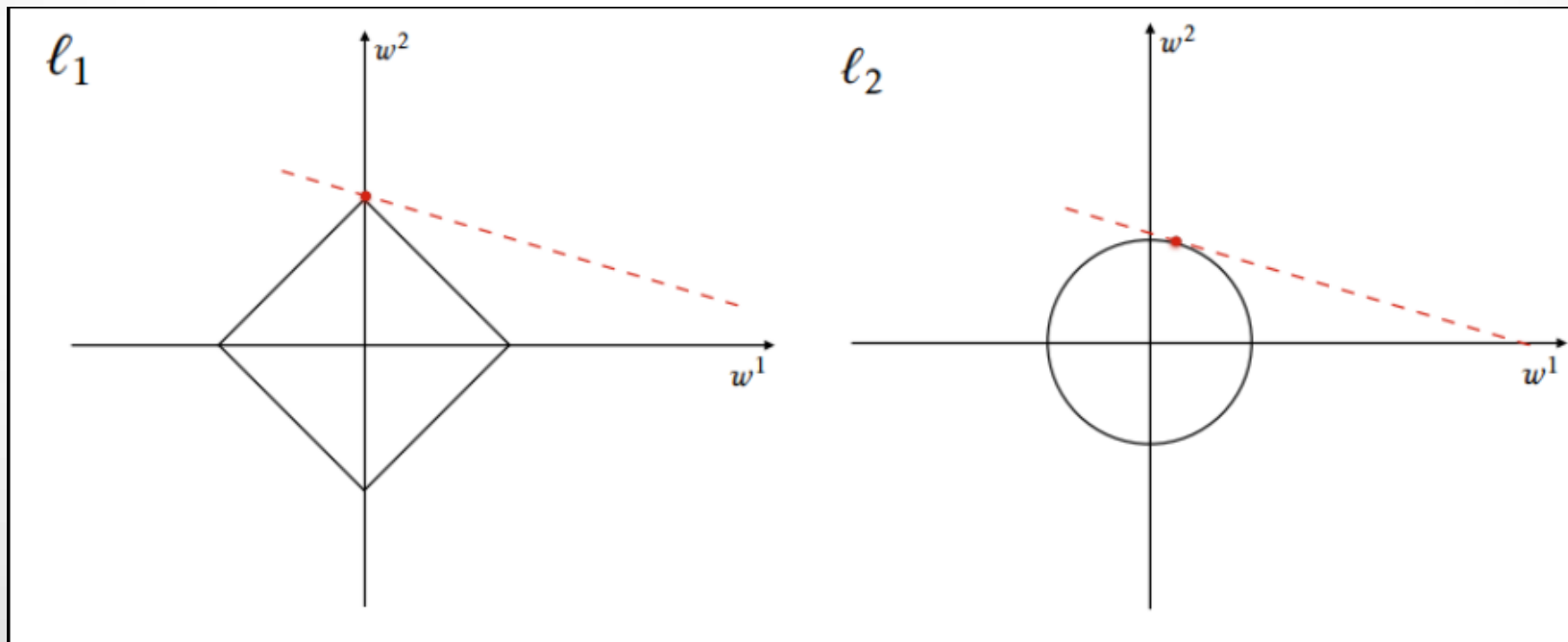
定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$, 其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i (i=1, \dots, N)$ 是 d -维的向量。

$$\min_{\boldsymbol{\theta}} \frac{1}{2N} \sum_{i=1}^N \|h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2 + \lambda \cdot \|\boldsymbol{\theta}\|_1$$

L1-范数的表达式。 $\|\boldsymbol{\theta}\|_1 = \sum_{d=1}^D |\theta_d|$

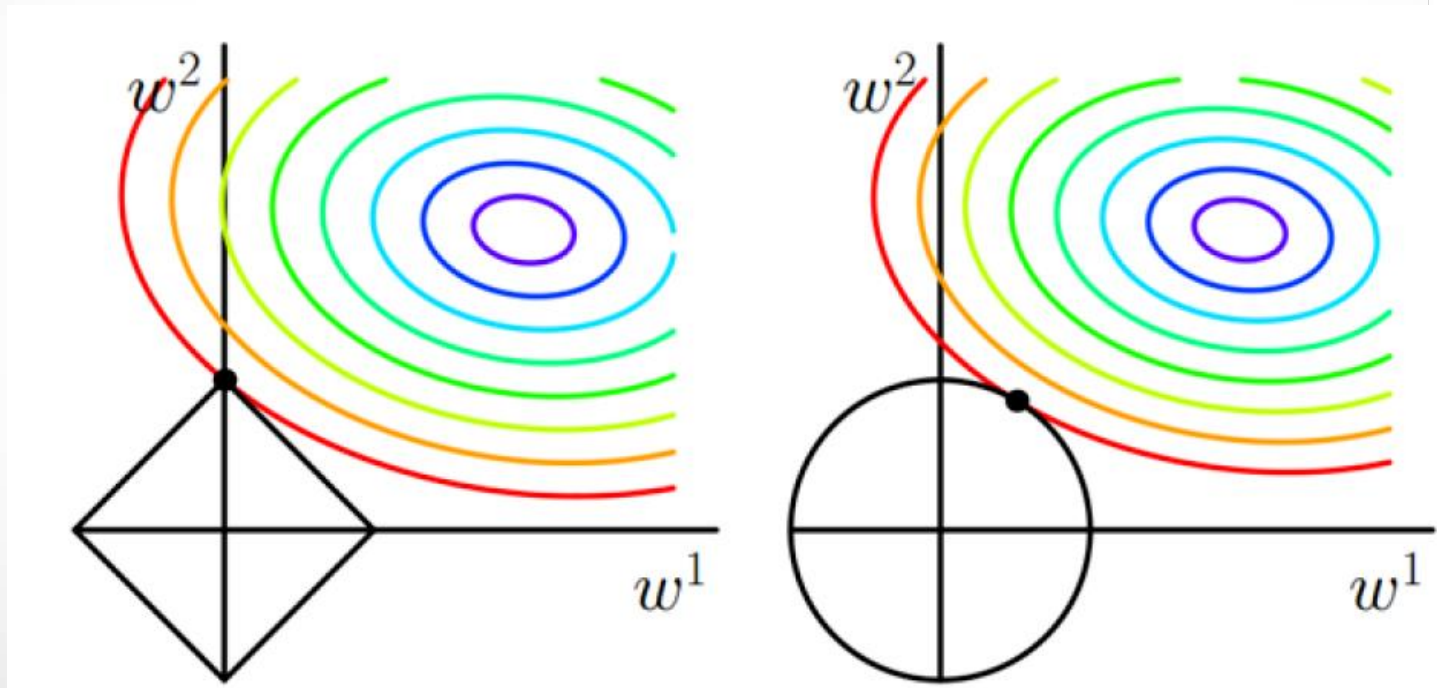
注：这里正则化项的设计采用的是L1-norm,是L0的凸松弛，近似的稀疏解，可以通过Lasso算法迭代求解(虽然凸，但在 $\mathbf{x}=0$ 处不严格可导/可微，或者说在 $\mathbf{x}=0$ 处不存在导数)。

第五章：正则化regularization



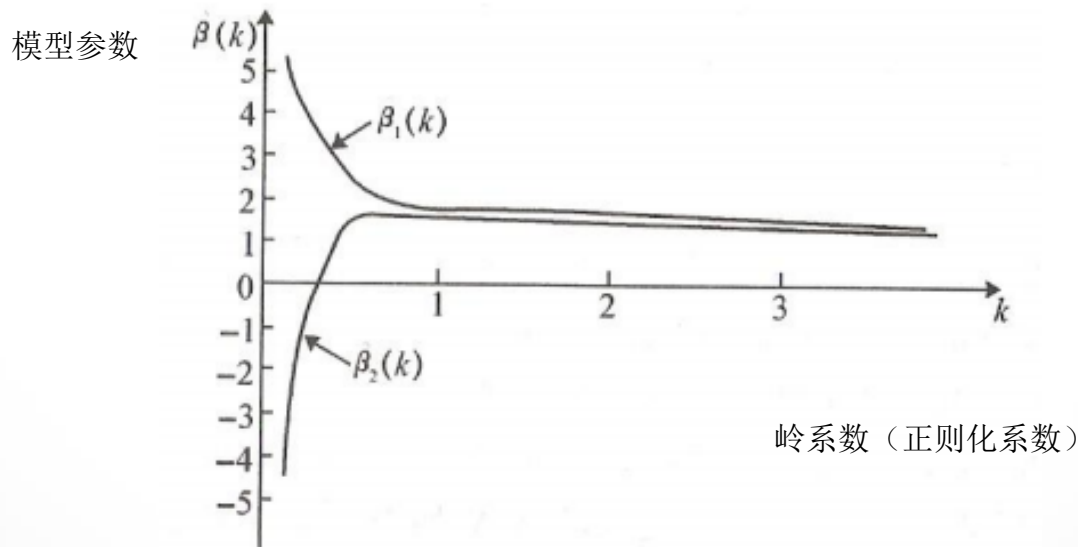
以两变量 w_1, w_2 为例。Ridge和Lasso都是有偏估计，最小二乘是无偏估计

第五章：正则化regularization



L1有角，解落在角点的可能性更大，而L2没有角，因此不可能落在坐标系上。等高线抛物面在水平面上的投影。

第五章：正则化regularization



岭回归名字的由来：模型的解与正则化参数的关系图类似一个山岭。



第五章：正则化regularization

□ElasticNet模型(Lasso Regression+Ridge Regression)

定义一组观测数据集 $(\mathbf{X}, \mathbf{Y})$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{Y}=[\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$, $\mathbf{X}$ 为属性, $\mathbf{Y}$ 为响应标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

$$\min_{\theta} \frac{1}{2N} \sum_{i=1}^N \|h_{\theta}(\mathbf{x}_i) - \mathbf{y}_i\|_2^2 + \lambda \cdot \|\theta\|_1 + u \cdot \|\theta\|_2$$

注：这里正则化项的设计采用的是L1-norm和L2-norm的混合体。

第五章：正则化regularization

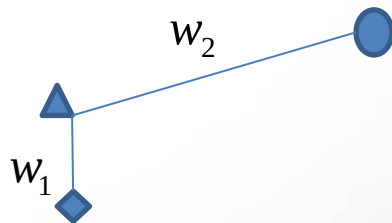
□半监督正则化（流形正则）

定义一组新的无标签的观测数据集 $\mathbf{X}$,其中 $\mathbf{X}=[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $\mathbf{X}$ 为属性, 无标签。 $\mathbf{x}_i$ ($i=1, \dots, N$) 是 d -维的向量。

因为样本无标签, 则不能再用损失函数对函数 f 进行定义。

不妨做出以下假设: 在欧式空间内, 距离较近的两个样本 $\mathbf{x}_1, \mathbf{x}_2$, 其具有相同响应即 $f(\mathbf{x}_1)=f(\mathbf{x}_2)$ 的概率更大。

设两点之间的连线, w 存在一个权重即重要性系数。





第五章：正则化regularization

□半监督正则化（流形正则）

局部线性嵌入（local linear embedding, Science 2000）。

一种非线性降维模型，使得流形学习(manifold learning)得到迅速发展。

假设数据是均匀采样于一个高维欧氏空间中的低维流形，流形学习就是从高维采样数据中恢复低维流形结构，即找到高维空间中的低维流形，并求出相应的嵌入映射，以实现维数约简或者数据可视化。

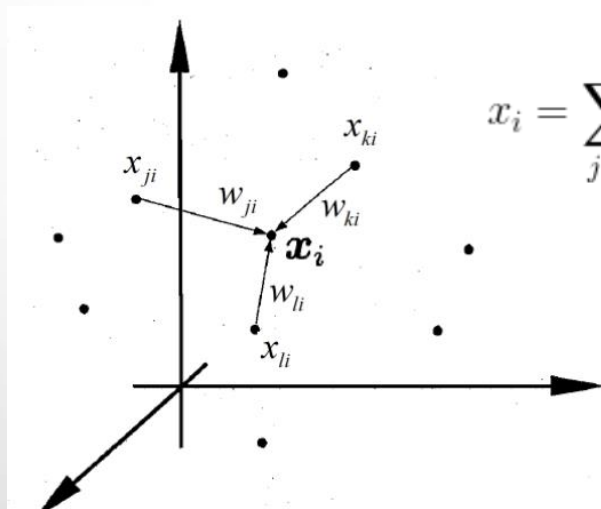
所谓局部线性是指数据的某个局部是线性的，但整体是非线性的。

流形学习即学习从高维到低维的嵌入映射，使得数据的局部拓扑结构不变。

第五章：正则化regularization

□半监督正则化（流形正则）

局部线性嵌入（local linear embedding, Science 2000）。



$$x_i = \sum_{j=1}^k w_{ji} x_{ji}$$



$$\arg \min_w \sum_{i=1}^N (\|x_i - \sum_{j=1}^k w_{ji} x_{ji}\|_2^2)$$



$$\arg \min_Y \sum_{i=1}^N (\|y_i - \sum_{j=1}^k w_{ji} y_{ji}\|_2^2)$$



第五章：正则化regularization

□半监督正则化（流形正则）

按照上面的分析，半监督正则项的表达式可以写成

$$R(f) = \sum_{i,j} w_{i,j} (f(x_i) - f(x_j))^2$$

其中， $w_{i,j}$ 是连接样本 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 之间的权重（重要性系数），属于给定的值，其定义方法可以根据欧式距离来表达。比如，

$$w_{i,j} = \exp\left(-\|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$

显然，权重矩阵 $\mathbf{W}$ 是一个对称阵。

第五章：正则化regularization

□基于半监督正则化（流形正则）的线性建模

按照上面的分析，半监督正则项的表达式可以写成


$$\min_f \sum_k (f(x_k) - y_k)^2 + \alpha \cdot \|f\|_2 + \beta \cdot R(f)$$

$$\min_f \sum_k (f(x_k) - y_k)^2 + \alpha \cdot \|f\|_2 + \beta \cdot \sum_{i,j} w_{i,j} (f(x_i) - f(x_j))^2$$



第五章：正则化regularization

□ Dropout

Dropout是深度神经网络（深度学习）框架中的基本操作(Hinton, 2012), 是一种典型的深度学习模型正则化技术，防止overfitting。

核心思想：通过多模型/网络组合的形式改善算法鲁棒性。

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M \hat{Y}_m$$

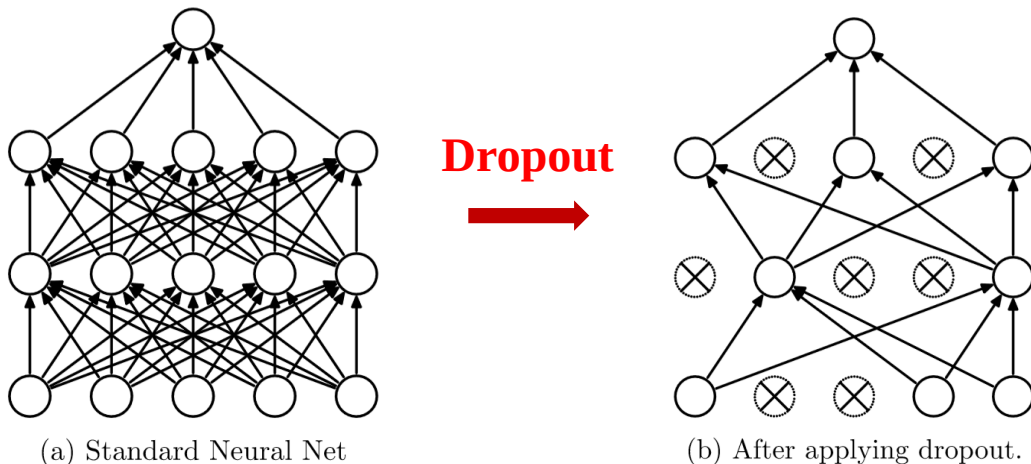
其中， $\hat{Y}_m$ 表示每个模型的预测结果。

存在问题：如果要训练**M**个不同结构的模型/网络，成本高、效率低。

第五章：正则化regularization

□ Dropout

Dropout就是在优化过程中，以一定的概率，随机“删除”网络模型中的一些特征节点（隐层神经元）。

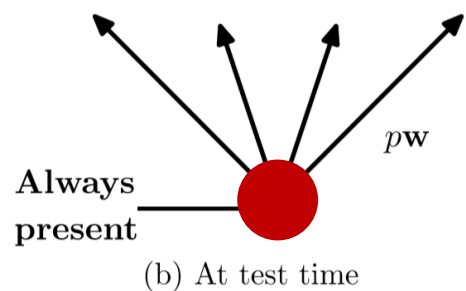
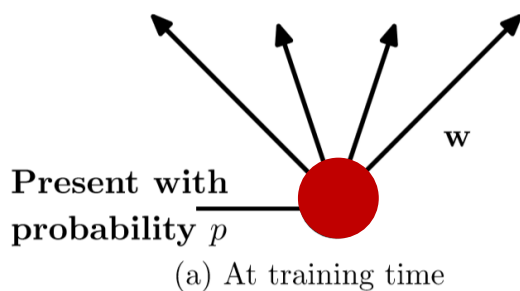
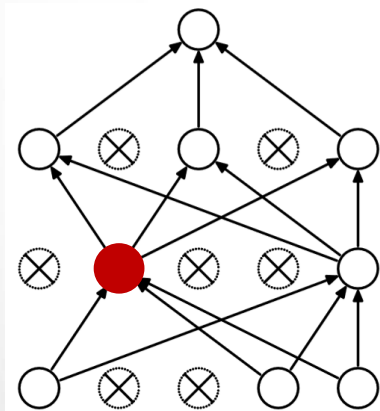


Hinton, et al. “Improving neural networks by preventing co-adaptation of feature detectors,” arXiv, 2012.
 Hinton, et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, 2014.

第五章：正则化regularization

□ Dropout

Dropout就是在优化过程中，以一定的概率，随机“删除”网络模型中的一些特征节点（隐层神经元）。



注1: Dropout只出现在训练/学习阶段

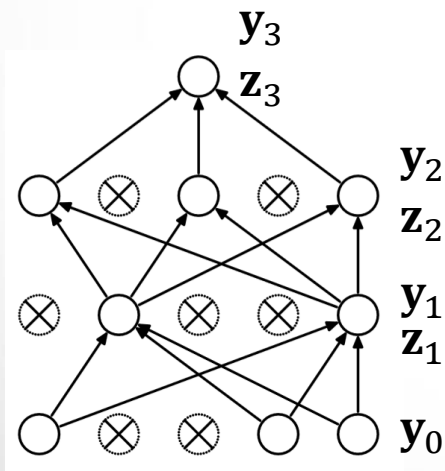
注2: 测试阶段，采用 pw ，保持期望不变 $px + (1-p)0 = px$

从模型参数控制/约简角度，**Dropout**与**L1, L2**正则相似。

第五章：正则化regularization

□ Dropout

Dropout就是在优化过程中，以一定的概率，随机“**删除**”网络模型中的一些特征节点（隐层神经元）。



z 表示每层输入, y 表示每层输出

$$z_i^{(l+1)} = \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)},$$

$$y_i^{(l+1)} = f(z_i^{(l+1)}),$$



$$r_j^{(l)} \sim \text{Bernoulli}(p),$$

$$\tilde{\mathbf{y}}^{(l)} = \mathbf{r}^{(l)} * \mathbf{y}^{(l)},$$



$$z_i^{(l+1)} = \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)},$$

$$y_i^{(l+1)} = f(z_i^{(l+1)}).$$

标准前向计算单元



Dropout单元

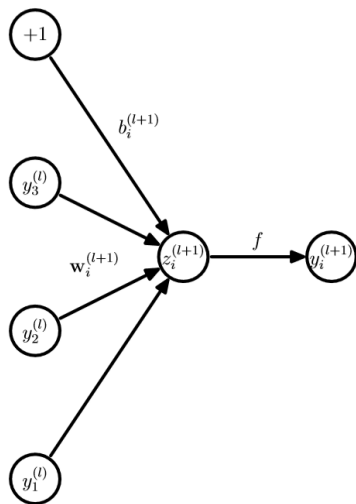


带有**Dropout**的基本
前向计算单元

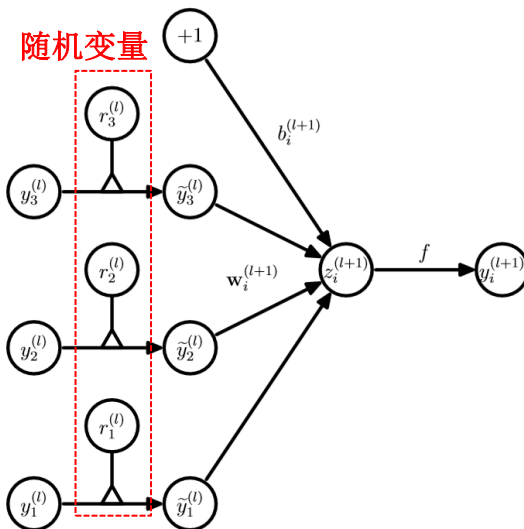
第五章：正则化regularization

□ Dropout

Dropout就是在优化过程中，以一定的概率，随机“**删除**”网络模型中的一些特征节点（隐层神经元）。



(a) Standard network



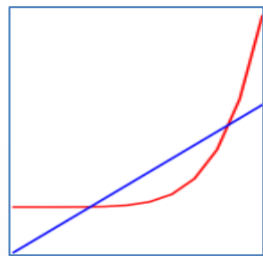
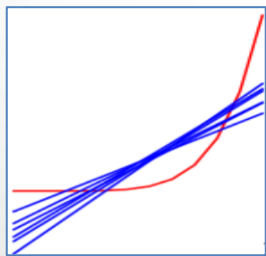
(b) Dropout network

第五章：正则化regularization

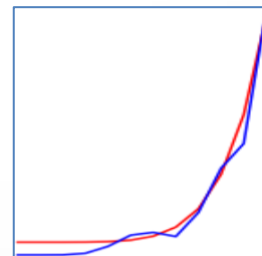
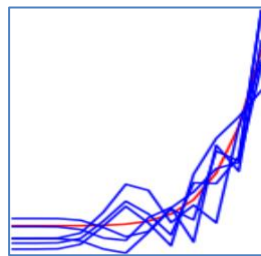
□正则化、泛化能力、过拟合三者关系

正则化的提出是用于提升模型的泛化能力，避免过拟合问题。

1. 泛化能力是指，训练模型不仅要保证训练集的误差最小化，同时还要保证测试性能，通常称为“偏差-方差”平衡问题（或者折中问题，Bias-Variance trade-off）。偏差是针对训练集，方差针对验证集。
2. **Total error=bias²+variance+noise**



偏差大
方差小
欠拟合



偏差小
方差大
过拟合

总体误差= $E[(y - \hat{f})^2]$ 偏差= $(f - E[\hat{f}])^2$ 方差= $E[(E[\hat{f}] - \hat{f})^2]$ 噪声= $E[(f - y)^2]$ (人工标注误差)

$$Err(x) = \left(E[\hat{f}(x)] - f(x)\right)^2 + E\left[\hat{f}(x) - E[\hat{f}(x)]\right]^2 + \sigma_e^2$$



第五章：正则化regularization

□正则化、泛化能力、过拟合三者关系

$$Err(x) = \left(E[\hat{f}(x)] - f(x)\right)^2 + E\left[\hat{f}(x) - E[\hat{f}(x)]\right]^2 + \sigma_e^2$$

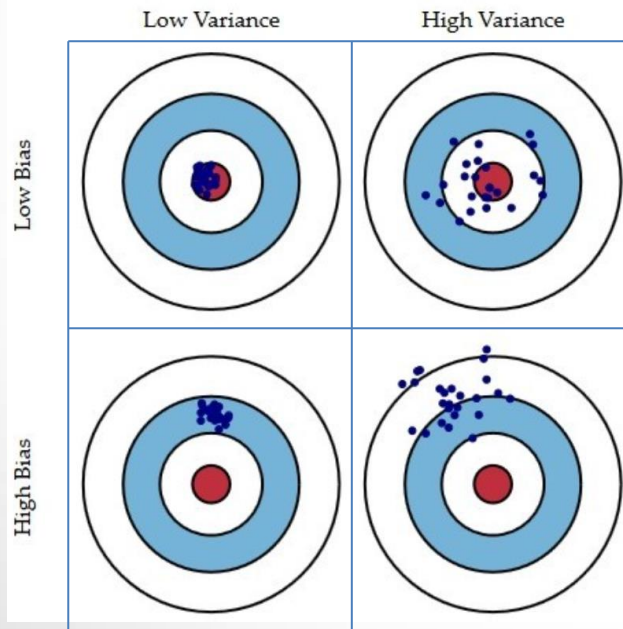
推导过程

$$\begin{aligned} Err(x) &= E\left[(\hat{f} - y)^2\right] = E\left[(\hat{f} - \bar{f} + \bar{f} - y)^2\right] \\ &= E\left[(\hat{f} - \bar{f})^2\right] + E\left[(\bar{f} - y)^2\right] + 2E[(\hat{f} - \bar{f}) \cdot (\bar{f} - y)] = E\left[(\hat{f} - \bar{f})^2\right] + E\left[(\bar{f} - y)^2\right] \\ &= E\left[(\hat{f} - \bar{f})^2\right] + E\left[(\bar{f} - f + f - y)^2\right] \\ &= E\left[(\hat{f} - \bar{f})^2\right] + E\left[(\bar{f} - f)^2\right] + E[(f - y)^2] + 2E[(\bar{f} - f) \cdot (f - y)] \\ &= E\left[(\hat{f} - \bar{f})^2\right] + (\bar{f} - f)^2 + E[(f - y)^2] \\ &= Variance + Bias^2 + \sigma_e^2 \end{aligned}$$

其中， y 为真实值， f 为人工标定值， $\hat{f}$ 为预测值， $\bar{f}$ 为多次模型预测的均值

第五章：正则化regularization

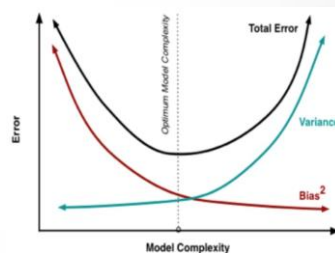
□正则化、泛化能力、过拟合三者关系



机器学习是通过有限的训练数据，利用数学模型获得对无限数据的近似。

根本原因：

1. 模型的不完美
2. 训练数据有限



解决方法：k-fold cross-validation、大数据集、深度学习

寻找泛化、欠拟合、过拟合之间的平衡问题，就是“偏差-方差”折中问题。



第六章：概率建模



脑认知



猫



第六章：概率建模

□产生式思考

对于任意一个样本，我们定义模型如下：

$$t_n = \mathbf{w}^T \mathbf{x}_n + \varepsilon_n$$

其中， ε_n 是一个随机变量。所谓产生式考虑，是指第n次的观测响应，是通过 $\mathbf{w}^T \mathbf{x}_n$ 生成。同时，伴随着一个随机变量，可正可负。

ε_n 是一个随机变量，存在某种概率分布。

我们假设， $\varepsilon_n \sim \mathcal{N}(\mu, \sigma^2)$ 高斯分布（正态分布）

$$p(\varepsilon_n) = \mathcal{N}(\mu, \sigma^2)$$

该假设已在前面的课程给出解释（中心极限定理、易于分析）。



第六章：概率建模

□产生式思考

对于任意一个样本，我们定义模型如下：

$$t_n = \mathbf{w}^T \mathbf{x}_n + \varepsilon_n$$

现在，模型由两部分组成：

1. 数据生成的决定性部分， $\mathbf{w}^T \mathbf{x}_n$ ，表示一种趋势或倾向；
2. 随机组成部分 ε_n ，可以理解成噪声。

注：这里的随机变量 ε_n （或噪声）并非限定为高斯分布，也并非限定为可加性噪声。但为了便于解释和理解，选择可加性高斯分布噪声，可使我们获得最优参数 $\hat{\mathbf{w}}$ 的精确表达式。



第六章：概率建模

□产生式思考

将模型重新定义如下：

$$t_n = \mathbf{w}^T \mathbf{x}_n + \varepsilon_n, \text{ 其中, } \varepsilon_n \sim \mathcal{N}(\mu, \sigma^2)$$

这里令 $\mu=0$ ，即 $\varepsilon_n \sim \mathcal{N}(0, \sigma^2)$

由于 $\mathbf{w}^T \mathbf{x}_n$ 是一个常量，因此加上一个随机变量 ε_n 后， t_n 也是一个随机变量（这是赋予随机变量的原因）。

而且， $t_n \sim \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2)$ （为什么？高斯随机变量加上一个常数，依然是高斯，仅均值发生变化），从而概率密度函数可以写成：

$$p(t_n | \mathbf{x}_n, \mathbf{w}, \sigma^2) = \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(t_n - \mathbf{w}^T \mathbf{x}_n)^2}{2\sigma^2}}$$

可以看出，预测响应 t_n 的密度依赖于特定的 $\mathbf{x}_n, \mathbf{w}$ (均值)和 σ^2 (方差)的值。



第六章：概率建模

□问题

我们看下面这个概率密度函数

$$p(t_n|\mathbf{x}_n, \mathbf{w}, \sigma^2) = \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2)$$

对于给定的 $\mathbf{w}, \sigma^2$ ，我们可以计算出在观测 $\mathbf{x}_n$ 下，使得 $p(t_n|\mathbf{x}_n, \mathbf{w}, \sigma^2)$ 最大的 t_n ，即预测值。

对于已知的高斯分布来说，只要均值和方差确定，即可绘制出连续或离散的概率密度。

例：设第10个苹果的直径为0.5，即 $\mathbf{x}_{10}=[0.5, 1]^T$ ；设 $\mathbf{w}=[0.6, -0.1]^T$, $\sigma^2 = 0.1$ ；另：该苹果实际重量 $t_{10}=2$ 。

分析：针对该苹果，可以绘制其重量的概率密度函数曲线

第六章：概率建模

□问题

✓ 如果 $\mathbf{w}, \sigma^2$ 已经最优

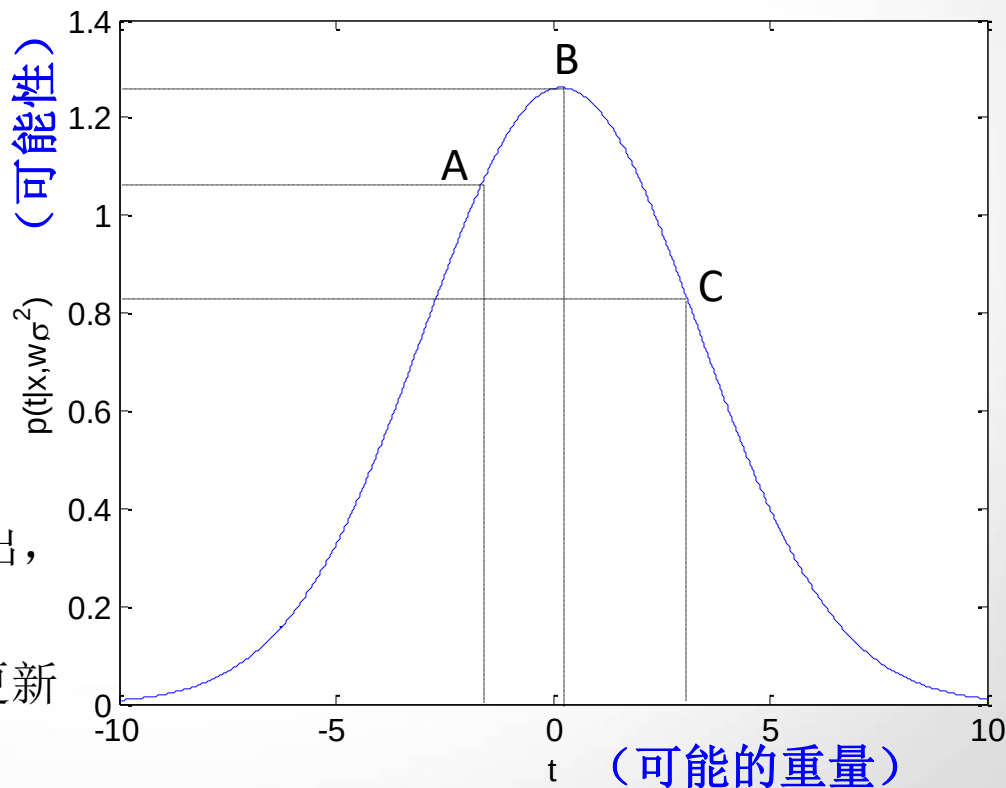
通过右边的概率密度函数可以得出：

1. B点的可能性最大，即为苹果重量的最佳估计值 $\mathbf{w}^T \mathbf{x}_{10}$
2. C点的可能性最小。

✓ 如果 $\mathbf{w}, \sigma^2$ 还不是最优

实际重量 $t_{10}=2$ ，根据图可以看出，最佳的可能值是在B和C之间。

但是数据不可能变，因此只能更新模型参数。



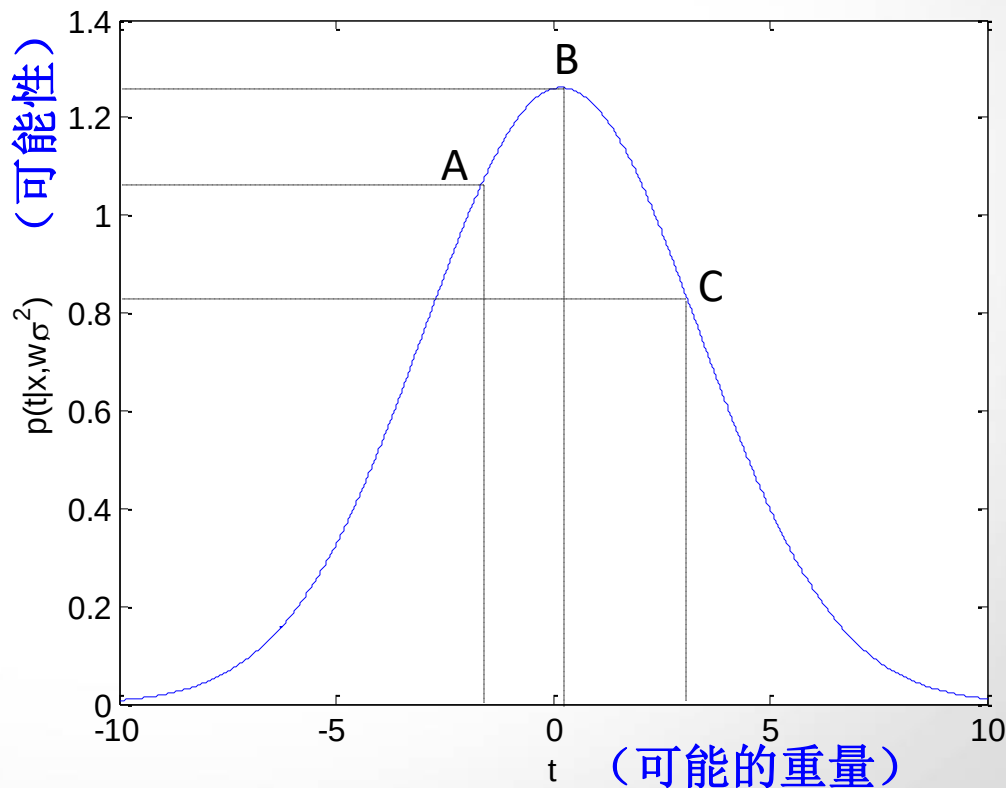
第六章：概率建模

□ 定义

这种通过寻找参数，来最大化似然值的方式，称为基于似然函数最大化的线性建模——概率建模，是机器学习中的一种重要方法。

□ 数据集的似然值

对机器学习，我们感兴趣的是整个数据集的所有样本的似然值，而并非单个样本的似然值。





第六章：概率建模

□ 数据集的似然值

- 对于单个样本的概率密度表达式为

$$p(t_n|\mathbf{x}_n, \mathbf{w}, \sigma^2) = \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2)$$

- 如果有N个数据样本点，那么我们感兴趣的是它们的联合条件概率密度

$$p(t_1, t_2, \dots, t_N | \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N, \mathbf{w}, \sigma^2)$$

- 根据向量和矩阵表示法，可以写成紧缩的表达形式：

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2)$$

- 假设数据的噪声是独立的，即噪声的联合概率密度可表达为

$$p(\boldsymbol{\varepsilon}) = p(\varepsilon_1, \varepsilon_2, \dots, \varepsilon_N) = \prod_{n=1}^N p(\varepsilon_n)$$

这一独立性假设，可以使联合密度进行分解为更易操作的对象，使问题求解更简单



第六章：概率建模

□ 数据集的似然值

➤ 基于噪声的独立性假设，对N个数据样本点，它们的联合条件概率密度

$$\begin{aligned} p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2) &= p(t_1, t_2, \dots, t_N | \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N, \mathbf{w}, \sigma^2) = \prod_{n=1}^N p(t_n | \mathbf{x}_n, \mathbf{w}, \sigma^2) \\ &= \prod_{n=1}^N \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2) \end{aligned}$$

注：这里不是说 $\mathbf{t}$ 是独立的，否则不存在对数据的建模。而是条件独立，可以根据模型 $t_n = \mathbf{w}^T \mathbf{x}_n + \varepsilon_n$ 理解。当 $\mathbf{w}^T \mathbf{x}_n$ 给定时， t_n 独立，即条件独立。
或者说，只有当模型确定了， t_n 才是一个独立的随机变量。

另： $\prod_{n=1}^N$ 表示连乘符号。



第六章：概率建模

□ 最大似然建模

- 最大似然建模，就是为了寻找 $\mathbf{w}$ 和 σ^2 ，使得似然值最大化。也就是最大化下列似然函数。

$$\begin{aligned}\max_{\mathbf{w}, \sigma^2} p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2) &= p(t_1, t_2, \dots, t_N | \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N, \mathbf{w}, \sigma^2) \\ &\propto \max_{\mathbf{w}, \sigma^2} \prod_{n=1}^N p(t_n | \mathbf{x}_n, \mathbf{w}, \sigma^2) = \prod_{n=1}^N \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2)\end{aligned}$$

换句话说，当给定 $\mathbf{w}$ 和 σ^2 时，可以获得数据集的似然值 $L = p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2)$ ，但是，该似然值并非最大，因此，我们要改变 $\mathbf{w}$ 和 σ^2 使得 L 最大。

当然，为什么不改变 $\mathbf{X}$ 呢？数据！！

因为数据集是固定的，不同的参数值会产生不同似然值，建模的目的是要求出最佳的参数值。



第六章：概率建模

□ 最大似然建模

➤ 最大似然建模求解：即求解 $\mathbf{w}$ 和 σ^2 ，使得似然值最大化。

$$\max_{\mathbf{w}, \sigma^2} \prod_{n=1}^N \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2)$$

很显然，要求解上面的模型，可以最大化似然值的自然对数，用 $\log$ 表示，即，

$$\max_{\mathbf{w}, \sigma^2} \prod_{n=1}^N \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2) \propto \max_{\mathbf{w}, \sigma^2} \log \prod_{n=1}^N \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2)$$

➤ 似然函数的自然对数可以简化

$$\begin{aligned} \log \prod_{n=1}^N \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2) &= \log \mathcal{N}(\mathbf{w}^T \mathbf{x}_1, \sigma^2) + \cdots + \log \mathcal{N}(\mathbf{w}^T \mathbf{x}_N, \sigma^2) \\ &= \sum_{n=1}^N \log \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2) \end{aligned}$$

第六章：概率建模

□ 最大似然建模

➤ 似然函数的自然对数可以简化

$$\begin{aligned}\log L &= \log \prod_{n=1}^N \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2) = \sum_{n=1}^N \log \mathcal{N}(\mathbf{w}^T \mathbf{x}_n, \sigma^2) \\ &= \sum_{n=1}^N \log \left(\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(t_n - \mathbf{w}^T \mathbf{x}_n)^2}{2\sigma^2}} \right) \\ &= \sum_{n=1}^N \left(-\frac{1}{2} \log(2\pi) - \log \sigma - \frac{1}{2\sigma^2} (t_n - \mathbf{w}^T \mathbf{x}_n)^2 \right) \\ &= -\frac{N}{2} \log(2\pi) - N \log \sigma - \frac{1}{2\sigma^2} \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n)^2\end{aligned}$$



第六章：概率建模

□ 最大似然建模

似然函数的自然对数最大化模型可以简化

$$\max_{\mathbf{w}, \sigma^2} \log L = -\frac{N}{2} - \log(2\pi) - N \log \sigma - \frac{1}{2\sigma^2} \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n)^2$$

➤ 利用前面学习的内容，对 $\mathbf{w}$ 求偏导，

$$\begin{aligned} \frac{\partial \log L}{\partial \mathbf{w}} &= \frac{1}{\sigma^2} \sum_{n=1}^N \mathbf{x}_n (t_n - \mathbf{x}_n^T \mathbf{w}) = \frac{1}{\sigma^2} \sum_{n=1}^N \mathbf{x}_n t_n - \frac{1}{\sigma^2} \sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^T \mathbf{w} \\ &= \frac{1}{\sigma^2} (\mathbf{X}^T \mathbf{t} - \mathbf{X}^T \mathbf{X} \mathbf{w}) \end{aligned}$$

令 $\frac{\partial \log L}{\partial \mathbf{w}} = \mathbf{0}$ ，可以得出 $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}$ ($\mathbf{w}$ 的最大似然解) *似曾相识！？*
将噪声假设为高斯分布，最小化平方损失等同于最大似然解。



第六章：概率建模

□ 最大似然建模

似然函数的自然对数最大化模型可以简化

$$\max_{\mathbf{w}, \sigma^2} \log L = -\frac{N}{2} - \log(2\pi) - N \log \sigma - \frac{1}{2\sigma^2} \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n)^2$$

➤ 对 σ 求偏导,

$$\frac{\partial \log L}{\partial \sigma} = -\frac{N}{\sigma} + \frac{1}{\sigma^3} \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n)^2$$

令 $\frac{\partial \log L}{\partial \sigma} = 0$, 可以得出 $\widehat{\sigma^2} = \frac{1}{N} \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n)^2 = \frac{1}{N} (\mathbf{t} - \mathbf{X}\mathbf{w})^T (\mathbf{t} - \mathbf{X}\mathbf{w})$

最终, $\widehat{\sigma^2} = \frac{1}{N} (\mathbf{t}^T \mathbf{t} - \mathbf{t}^T \mathbf{X}\mathbf{w})$



第六章：概率建模

课外任务：

根据前面的内容，请自行推导最大似然建模方法的整个过程。

第六章：概率建模

□ Logistic回归

回顾线性回归算法中的二类分类问题

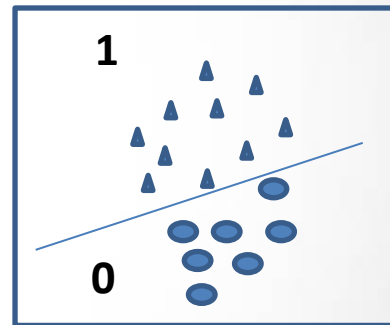
$$h(\mathbf{x}) = \mathbf{w}_0 + \mathbf{w}_1\mathbf{x}_1 + \cdots + \mathbf{w}_d\mathbf{x}_d$$

分类边界为

$$h(\mathbf{x}) = 0$$

□ 当 $h(\mathbf{x}) > 0$, $\mathbf{x}$ 的属性为 $Y=1$

□ 当 $h(\mathbf{x}) < 0$, $\mathbf{x}$ 的属性为 $Y=0$



也就是说，对于一次观测 $\mathbf{x}$ ，其仅仅给出“是”或者“不是”的猜测（**简单、粗暴**），给出的预测是一个“肯定发生”或“肯定不发生”的事件。

如果提供一个关于可能性大小的结果岂不是更好？？换句话说，“可能性”是关于概率的一个概念，如果对于一个观测，能够计算出该观测的事件发生的概率（即 $Y=1$ 发生的概率），通常比直接给出“是”或者“不是”更加有用。



第六章：概率建模

□ Logistic回归

几个特点：

- **Logistic**回归模型作为一种概率模型，可用于预测某事件发生的概率；
- **Logistic**回归模型不要求因变量在正态分布的前提假设下完成。
- 该模型在疾病诊断、预测中应用较广，主要用于分析不同变量因素对疾病的影响。在医学领域，一些随机事件只具有两种**互斥**结果的离散性随机事件。这类只具有两种互斥结果的离散型随机事件的规律性进行描述的一种概率分布，叫**二项分布**。
- 二项分布概率公式：如果进行n次伯努利试验，取得成功次数(事件发生)为k(k=0,1,...,n)的概率，表示为

$$P(\xi = k) = C_n^k p^k (1 - p)^{n-k}$$

其中，事件发生的概率为p, 事件不发生的概率为1-p。 $\xi \sim B(n, p)$ ，当n=1时，二项分布也可称为伯努利分布或0-1分布（1次伯努利试验）



第六章：概率建模

□ Logistic回归

条件概率：

对于观测输入变量 $\mathbf{x}$, 其响应 Y 的条件概率分布表达为

$$P(Y|\mathbf{X})$$

这个概率表示模型的准确性。

设想，如果对于“今天是否下雪”的预测结果是51%，但是并没有下雪。那么这个预测也要比预测为下雪的可能性为99%要好！

设 $P(Y = 1|\mathbf{X} = \mathbf{x}) = p(\mathbf{x}; \mathbf{w})$, 其中 $\mathbf{w}$ 为参数，那么有

$$P(Y = 0|\mathbf{X} = \mathbf{x}) = 1 - p(\mathbf{x}; \mathbf{w})$$

Y 的取值为0
或1



第六章：概率建模

□ Logistic回归

条件概率：

对于一个观测 x_i ,其响应 $Y = y_i$ 的条件概率可以写成

$$P(Y = y_i | X = x_i) = p(x_i; \mathbf{w})^{y_i} (1 - p(x_i; \mathbf{w}))^{1-y_i}$$

由于 N 次观测的独立性，联合条件概率分布(似然)可以写为

$$\prod_{i=1}^N P(Y = y_i | X = x_i) = \prod_{i=1}^N p(x_i; \mathbf{w})^{y_i} (1 - p(x_i; \mathbf{w}))^{1-y_i}$$

N 个观测的似然函数

Y 的取值为0
或1

第六章：概率建模

□ Logistic回归—与线性回归的关系

主要思想：logistic回归模型的输出是某次观测事件发生的概率，通过观测样本 x 的特征与最佳估计参数相乘、求和后，然后利用Logistic函数（sigmoid）进行非线性计算（概率），然后与阈值进行比较，得出该观测 x 的输出。

相乘、求和：

$$h(x) = w_0 + w_1x_1 + \dots + w_dx_d$$

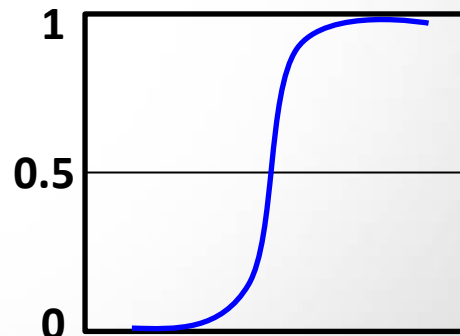
条件概率 $P(y = 1|x)$ 的计算表达式为

$$P(y = 1|x) = p(x) = \frac{1}{1 + e^{-h(x)}}$$

条件概率 $P(y = 0|x)$ 的计算表达式为

$$P(y = 0|x) = 1 - P(y = 1|x) = 1 - p(x) = \frac{1}{1 + e^{h(x)}}$$

Sigmoid函数



定义域 $(-\infty, \infty)$ ；值域为 $(0, 1)$

第六章：概率建模

□ Logistic回归—与线性回归的关系

$P(y = 1|x) = \frac{1}{1+e^{-h(x)}}$ 为事件发生的概率;

$P(y = 0|x) = \frac{1}{1+e^{h(x)}}$ 为事件不发生的概率;

那么, 事件发生与事件不发生的概率比值为

$$\frac{P(y = 1|x)}{P(y = 0|x)} = \frac{p(x)}{1 - p(x)} = \frac{1 + e^{h(x)}}{1 + e^{-h(x)}} = e^{h(x)}$$

两边同时取对数,

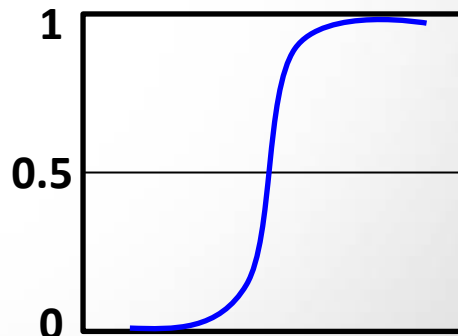
$$\log \frac{p(x)}{1 - p(x)} = h(x) = w_0 + w_1x_1 + \dots + w_dx_d$$

得到线性函数 (即**Logistic 回归模型**)。

利用上式, 可以求出 $p(x) = \frac{e^{h(x)}}{1+e^{h(x)}} = \frac{1}{1+e^{-h(x)}}$ 介于 $(0, 1)$ 。

采用Logit变换, 建立与属性变量之间的关系。
当 $0 < p < 1$ 时, logit变换可取任意值

Sigmoid函数



定义域 $(-\infty, \infty)$; 值域为 $(0, 1)$

第六章：概率建模

□ Logistic回归—与线性回归的关系

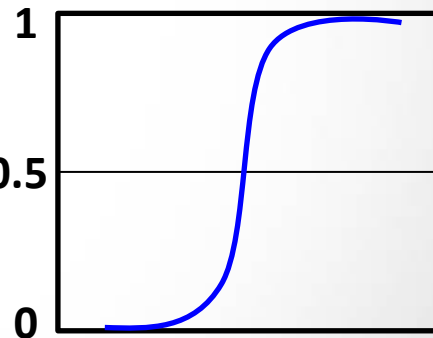
□ $p(x) = \frac{e^{h(x)}}{1+e^{h(x)}} = \frac{1}{1+e^{-h(x)}}$ 介于 $(0,1)$ 。

根据观测样本 x 的概率表示可以看出，当 $p(x) > 0.5$, $Y=1$;
当 $p(x) < 0.5$, $Y=0$ 。

□ 这也同时意味着， $h(x) > 0$ 时， $Y=1$ ； $h(x) < 0$ 时， $Y=0$ 。
即，将两类分开的判决边界为 $h(x) = 0$ 。

□ 但Logistic回归不同于线性分类器，不是为了找分类边界，
而是找出一个概率值，对“可能性”进行建模。

Sigmoid函数



定义域 $(-\infty, \infty)$;
值域为 $(0, 1)$

在Logistic回归模型中，若要计算 $p(x;w)$ ，关键是要求出参数 w 。



第六章：概率建模

□ Logistic回归

最大似然函数：

对于N次观测，似然函数可以写为

$$\prod_{i=1}^N P(Y = y_i | X = x_i) = \prod_{i=1}^N p(x_i; \mathbf{w})^{y_i} (1 - p(x_i; \mathbf{w}))^{1-y_i}$$

注：N次观测的独立性。 $\mathbf{w}$ 是模型参数。

我们的目标是能够求出使这一似然函数取**最大值**时的参数估计 $\mathbf{w}$ 。

$$\hat{\mathbf{w}} = \max_{\mathbf{w}} \prod_{i=1}^N P(Y = y_i | X = x_i) = \prod_{i=1}^N p(x_i; \mathbf{w})^{y_i} (1 - p(x_i; \mathbf{w}))^{1-y_i}$$

接下来，如何解这个模型是关键。



第六章：概率建模

□ Logistic回归

最大似然函数：

对似然函数取自然对数，可得出对数似然函数 $L(\mathbf{w})$,

$$\begin{aligned} L(\mathbf{w}) &= \sum_{i=1}^N [y_i \log p(x_i; \mathbf{w}) + (1 - y_i) \log(1 - p(x_i; \mathbf{w}))] \\ &= \sum_{i=1}^N \left[\log(1 - p(x_i; \mathbf{w})) + y_i \log \frac{p(x_i; \mathbf{w})}{1 - p(x_i; \mathbf{w})} \right] \\ &= \sum_{i=1}^N \left[\log \left(1 - \frac{1}{1 + e^{-h(x)}} \right) + y_i h(x) \right] \\ &= \sum_{i=1}^N \left[-\log(1 + e^{h(x)}) + y_i h(x) \right] \\ &= \sum_{i=1}^N \left[-\log(1 + e^{w_0 + \mathbf{w}^T \mathbf{x}_i}) \right] + \sum_{i=1}^N y_i (w_0 + \mathbf{w}^T \mathbf{x}_i) \end{aligned}$$

为了估计能使 $L(\mathbf{w})$ 取得最大值的参数 $\mathbf{w}=[w_0, w_1, \dots, w_d]$,对函数求导。



第六章：概率建模

□ Logistic回归

最大似然函数：

化简后的对数似然函数 $L(\mathbf{w})$,

$$L(\mathbf{w}) = \sum_{i=1}^N \left[-\log \left(1 + e^{w_0 + \mathbf{w}^T \mathbf{x}_i} \right) \right] + \sum_{i=1}^N y_i (w_0 + \mathbf{w}^T \mathbf{x}_i)$$

为了估计能使 $L(\mathbf{w})$ 取得最大值的参数 $\mathbf{w}=[w_0, w_1, \dots, w_d]$, 分别求函数对 $w_0, w_1, \dots, w_d$ 的导数, 显然存在 $d+1$ 个方程。

$$\begin{aligned} \frac{\partial L(\mathbf{w})}{\partial w_j} &= - \sum_{i=1}^N \frac{1}{1 + e^{w_0 + \mathbf{w}^T \mathbf{x}_i}} \cdot e^{w_0 + \mathbf{w}^T \mathbf{x}_i} \cdot x_{ij} + \sum_{i=1}^N y_i x_{ij} \\ &= \sum_{i=1}^N (y_i - p(x_i; \mathbf{w})) x_{ij} \end{aligned}$$

注：此时导数表达式是一个超越方程，不存在闭合解（解析解、精确解）。怎么办？

超越方程：非多项式函数，如指数函数、对数函数、三角函数等



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法：

我们考虑一个单变量函数的情况，即 $f(\beta)$ ，求该函数的最优解 β^* 。我们假设函数 $f(\cdot)$ 是平滑的，那么意味着该函数在 β^* 处的导数为0，二阶导数大于0（极值理论）。

那么根据泰勒展开， $f(\beta)$ 的表达式可以写成

$$f(\beta) \approx f(\beta^*) + \frac{1}{2}(\beta - \beta^*)^2 \frac{d^2 f}{d\beta^2} \Big|_{\beta=\beta^*}$$

既然 β^* 是最小值点，那么 $f(\beta) > f(\beta^*)$ ，因此， $\frac{d^2 f}{d\beta^2} \Big|_{\beta=\beta^*} > 0$ 。

而且，根据上式， $\frac{df}{d\beta} \Big|_{\beta=\beta^*} = 0$

换句话说，在最优解附近， $f(\beta)$ 是一个近二次的函数表达式。



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法：

既然在最优解附近， $f(\beta)$ 是一个近二次的函数表达式，而我们的目的是求出这个最优解，因此，可以将 $f(\beta)$ 进行二次泰勒展开（为什么不更高次？？极值定理）

首先，猜测一个初始解 $\beta^{(0)}$ ，在该点，二次泰勒展开：

$$f(\beta) \approx f(\beta^{(0)}) + (\beta - \beta^{(0)}) \frac{df}{d\beta} \Big|_{\beta=\beta^{(0)}} + \frac{1}{2} (\beta - \beta^{(0)})^2 \frac{d^2f}{d\beta^2} \Big|_{\beta=\beta^{(0)}}$$

为了方便，上式可以重新写成，

$$f(\beta) \approx f(\beta^{(0)}) + (\beta - \beta^{(0)}) f'(\beta^{(0)}) + \frac{1}{2} (\beta - \beta^{(0)})^2 f''(\beta^{(0)})$$

此时， $f(\beta)$ 是关于 β 的二次函数，可导，因此便可以利用导数=0求解。



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法：

$$f(\beta) \approx f(\beta^{(0)}) + (\beta - \beta^{(0)})f'(\beta^{(0)}) + \frac{1}{2}(\beta - \beta^{(0)})^2 f''(\beta^{(0)})$$

若要求函数 $f(\beta)$ 的最小值，我们令

$$\frac{df(\beta)}{d\beta} = f'(\beta^{(0)}) + f''(\beta^{(0)})(\beta - \beta^{(0)}) = 0$$

从而可以计算出

$$\beta = \beta^{(0)} - \frac{f'(\beta^{(0)})}{f''(\beta^{(0)})}$$

令此时的最优解为 $\beta = \beta^{(1)}$ （并非最优）

$$\beta^{(1)} = \beta^{(0)} - \frac{f'(\beta^{(0)})}{f''(\beta^{(0)})}$$



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法：

当计算出新的次优解 $\beta^{(1)}$ ，函数 $f(\beta)$ 的泰勒展开为

$$f(\beta) \approx f(\beta^{(1)}) + (\beta - \beta^{(1)})f'(\beta^{(1)}) + \frac{1}{2}(\beta - \beta^{(1)})^2 f''(\beta^{(1)})$$

若要求函数 $f(\beta)$ 的最小值，我们令

$$\frac{df(\beta)}{d\beta} = f'(\beta^{(1)}) + f''(\beta^{(1)})(\beta - \beta^{(1)}) = 0$$

从而可以计算出

$$\beta = \beta^{(1)} - \frac{f'(\beta^{(1)})}{f''(\beta^{(1)})}$$

令此时的最优解为 $\beta = \beta^{(2)}$ （并非最优）

$$\beta^{(2)} = \beta^{(1)} - \frac{f'(\beta^{(1)})}{f''(\beta^{(1)})}$$



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法：

为了找到最优解，通常迭代多次，设为T次。

当计算出新的次优解 $\beta^{(t)}$ ，函数 $f(\beta)$ 的泰勒展开为

$$f(\beta) \approx f(\beta^{(t)}) + (\beta - \beta^{(t)})f'(\beta^{(t)}) + \frac{1}{2}(\beta - \beta^{(t)})^2 f''(\beta^{(t)})$$

若要求函数 $f(\beta)$ 的最小值，我们令

$$\frac{df(\beta)}{d\beta} = f'(\beta^{(t)}) + f''(\beta^{(t)})(\beta - \beta^{(t)}) = 0$$

从而可以计算出

$$\beta = \beta^{(t)} - \frac{f'(\beta^{(t)})}{f''(\beta^{(t)})}$$

令此时的最优解为 $\beta = \beta^{(t+1)}$ （并非最优）

解更新的迭代公式

$$\beta^{(t+1)} = \beta^{(t)} - \frac{f'(\beta^{(t)})}{f''(\beta^{(t)})}$$



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法（多个变量的情况——多维）：

一维时，解更新的迭代公式

$$\beta^{(t+1)} = \beta^{(t)} - \frac{f'(\beta^{(t)})}{f''(\beta^{(t)})}$$

对于**多维**时，模型由 $f(\beta) = \beta_0 + \beta_1 x$ 转化为 $f(\beta) = \beta_0 + \beta_1 x_1 + \cdots + \beta_d x_d$

那么如何计算多维的函数关于变量的一阶导数和二阶导数？？



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法（多个变量的情况——多维）：

对于**多维**时，模型由 $f(\beta) = \beta_0 + \beta_1 x$ 转化为 $f(\beta) = \beta_0 + \beta_1 x_1 + \cdots + \beta_d x_d$
回顾第三章（数学基础）中，关于对矩阵或向量的导数。

$$f'(\beta^{(t)}) = \nabla f(\beta^{(t)}) = \left[\frac{\partial f}{\partial \beta_1}, \frac{\partial f}{\partial \beta_2}, \dots, \frac{\partial f}{\partial \beta_d} \right]^T$$

$$f''(\beta^{(t)}) = \nabla^2 f(\beta^{(t)}) = \begin{bmatrix} \frac{\partial^2 f}{\partial \beta_1^2} & \cdots & \frac{\partial f}{\partial \beta_1 \partial \beta_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial \beta_d \partial \beta_1} & \cdots & \frac{\partial^2 f}{\partial \beta_d^2} \end{bmatrix} \Big|_{\beta=\beta^{(t)}} = H \text{ (Hessian矩阵)}$$



第六章：概率建模

□ Logistic回归

Newton-Raphson（牛顿-拉斐森）优化算法（多个变量的情况——多维）：

对于多维时，函数 $f(\beta)$ 的解更新优化方法为：

$$\beta^{(t+1)} = \beta^{(t)} - \frac{f'(\beta^{(t)})}{f''(\beta^{(t)})}$$



$$\beta^{(t+1)} = \beta^{(t)} - H^{-1}(\beta^{(t)}) \nabla f(\beta^{(t)})$$

迭代方法完全相同，只是在写法上，变成了矩阵向量的方式。



第六章：概率建模

□ 知识回顾（多元函数）

多元函数 $f(x_1, x_2, \dots, x_n)$ 在 $X^{(0)}$ 处的泰勒展开：

$$f(X) = f(X^{(0)}) + \nabla f(X^{(0)})^T \Delta X + \frac{1}{2} \Delta X^T G(X^{(0)}) \Delta X + \dots$$

其中，

$$(1) \nabla f(X^{(0)}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right] \bigg|_{X^{(0)}}^T$$

一阶梯度向量

$$(2) G(X^{(0)}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \bigg|_{X^{(0)}}$$

海塞矩阵(Hessian Matrix, 二阶梯度)



第六章：概率建模

□ Logistic回归

最大似然函数：

化简后的对数似然函数 $L(\mathbf{w})$,

$$L(\mathbf{w}) = \sum_{i=1}^N \left[-\log \left(1 + e^{w_0 + \mathbf{w}^T \mathbf{x}_i} \right) \right] + \sum_{i=1}^N y_i (w_0 + \mathbf{w}^T \mathbf{x}_i)$$

为了估计能使 $L(\mathbf{w})$ 取得最大值的参数 $\mathbf{w}=[w_0, w_1, \dots, w_d]$, 分别求函数对 $w_0, w_1, \dots, w_d$ 的导数, 显然存在 $d+1$ 个方程。

$$\begin{aligned} \frac{\partial L(\mathbf{w})}{\partial w_j} &= - \sum_{i=1}^N \frac{1}{1 + e^{w_0 + \mathbf{w}^T \mathbf{x}_i}} \cdot e^{w_0 + \mathbf{w}^T \mathbf{x}_i} \cdot x_{ij} + \sum_{i=1}^N y_i x_{ij} \\ &= \sum_{i=1}^N (y_i - p(x_i; \mathbf{w})) x_{ij} \end{aligned}$$

此时导数表达式是一个超越方程, 不存在闭合解。采用牛顿-拉斐森方法求解。



第六章：概率建模

□ Logistic回归

最大似然函数：

Hessian矩阵H的计算：

$$p(x) = \frac{e^{h(x)}}{1 + e^{h(x)}}$$

$$\frac{\partial^2 L(w)}{\partial w_j^2} = - \sum_{i=1}^N x_{ij}^2 \cdot p(x_i; \mathbf{w}) \cdot (1 - p(x_i; \mathbf{w}))$$
$$\frac{\partial^2 L(w)}{\partial w_j \partial w_l} = - \sum_{i=1}^N x_{ij} x_{il} \cdot p(x_i; \mathbf{w}) \cdot (1 - p(x_i; \mathbf{w}))$$

$$w^{(t+1)} = w^{(t)} - H^{-1}(w^{(t)}) \nabla f(w^{(t)})$$

迭代公式



第六章：概率建模

□ 牛顿法存在的问题

- ✓ 经典牛顿法虽然具有二次收敛性，但是要求初始点需要尽量靠近极小点，否则有可能不收敛。
- ✓ 计算过程中需要计算目标函数的二阶偏导数，难度较大。
- ✓ 目标函数的Hessian矩阵无法保持正定，会导致算法产生的方向不能保证是 f 在 x_k 处的下降方向，从而令牛顿法失效；
- ✓ 如果Hessian矩阵奇异，牛顿方向可能根本是不存在的。

□ 补充：正定矩阵的定义、性质、判定方法

定义：对于 n 阶方阵 A ，如果对任何非零向量 z ，都有 $z^T A z > 0$ ，就称 A 为正定矩阵。

性质：正定矩阵一定是非奇异的、可逆的矩阵。

判定：对称阵 A 为正定的充分必要条件为 A 的特征值均大于0。



第六章：概率建模

□ 本章小结

- **Logistic**回归在本质上是线性回归。
- **Logistic**回归解决的是因变量的离散型问题，即分类问题（二类问题）。
- **Logistic**回归适合于当因变量属于二项分布这类问题。
- **Logistic**回归模型的输出是介于0-1之间的数，是一个概率值，基于**sigmoid**函数。
- 该模型的求解采用最大似然估计和牛顿法。
- 建模过程不需要对属性变量进行高斯分布的假设，但依然需要条件独立假设。



第六章：概率建模

课外任务：

根据前面的内容，请自行推导Logistic回归建模方法的整个过程。



第七章：朴素贝叶斯学习



第七章：朴素贝叶斯学习

□ 贝叶斯学习

➤ Thomas Bayes的“逆概”问题。

正向概率：假设袋子里面有 N 个白球， M 个黑球，你伸手进去摸一把，摸出黑球的概率是多大？

逆向概率：如果我们事先并不知道袋子里面黑白球的比例，而是闭着眼睛摸出一个（或好几个）球，观察这些取出来的球的颜色之后，那么我们可以就此对袋子里面的黑白球的比例作出什么样的推测？

解决这个问题要靠什么？

一个字“猜”！



第七章：朴素贝叶斯学习

□ 贝叶斯学习

- 贝叶斯学习是一种基于概率的学习方法，能够计算显式的假设概率。它基于（1）假设的先验概率，（2）给定假设下观察到不同数据的概率，以及（3）观察到数据本身的概率。
 1. $P(h)$: 表示没有训练样本数据前，假设 h 拥有的初始概率，称为假设 h 的先验概率。如果，我们不清楚这个先验知识，在实际中通常简单地将每一种假设都赋予相同的概率。
 2. $P(D)$: 表示观察到训练数据 D 的先验概率，通常作为常数处理，因为不同的假设拥有相同的 $P(D)$ 。
 3. $P(D|h)$: 表示假设 h 成立时观察到数据 D 的概率。
 4. $P(h|D)$: 后验概率，即对于给定的一个训练样本 D ，假设 h 成立的概率。在机器学习中，我们感兴趣的是这个后验概率。



第七章：朴素贝叶斯学习

□ 贝叶斯学习

贝叶斯公式：

$$P(h|D) = \frac{P(h) * P(D|h)}{P(D)}$$

这个公式是怎么来的？

例：一所学校里面有 60% 的男生，40% 的女生。男生总是穿长裤，女生则一半穿长裤一半穿裙子。有了这些信息之后我们可以容易地计算“随机选取一个学生，他（她）穿长裤的概率和穿裙子的概率是多大”，这个就是前面说的“正向概率”的计算。然而，假设你走在校园中，迎面走来一个穿长裤的学生，你能够推断出他（她）是男生的概率是多大吗？

分析：如果知道有多少个穿长裤的学生，并知道穿长裤的人里面有多少男生就可以了。

设学校里面共有N人，其中：

◆ 穿长裤的男生的数量是 $N * P(\text{男}) * P(\text{裤} | \text{男})$

◆ 穿长裤的女生的数量是 $N * P(\text{女}) * P(\text{裤} | \text{女})$

那么， $P(\text{男} | \text{裤}) = \frac{N * P(\text{男}) * P(\text{裤} | \text{男})}{[N * P(\text{男}) * P(\text{裤} | \text{男}) + N * P(\text{女}) * P(\text{裤} | \text{女})]} = \frac{P(\text{男}) * P(\text{裤} | \text{男})}{P(\text{裤})}$

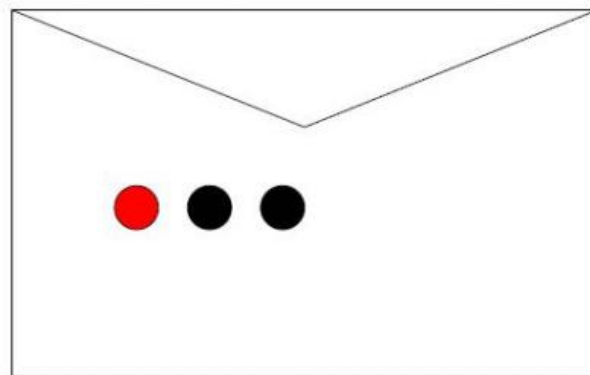
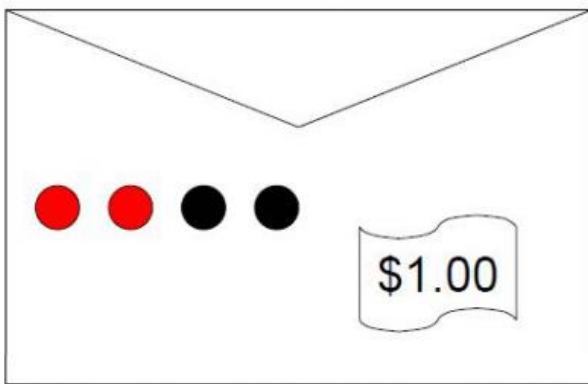
通式： $P(B|A) = \frac{P(B) * P(A|B)}{P(A)}$

第七章：朴素贝叶斯学习

□ 贝叶斯学习

贝叶斯公式：

$$P(h|D) = \frac{P(h) * P(D|h)}{P(D)}$$



- 如果摸到一个红球，那么，这个信封有1美元的概率是？
- 如果摸到一个黑球，那么，这个信封有1美元的概率是？

逆概问题



第七章：朴素贝叶斯学习

□ 贝叶斯学习

- ◆ c_1 、 c_2 表示左右两个信封。
- ◆ $P(R)$ ， $P(B)$ 表示摸到红球、黑球的概率。
- ◆ $P(R)=P(R|c_1)*P(c_1) + P(R|c_2)*P(c_2)$ ：全概率公式
- ◆ 后验： $P(c_1|R)=P(R|c_1)*P(c_1)/P(R)$
- ◆ $P(R|c_1)=2/4$
- ◆ $P(R|c_2)=1/3$
- ◆ $P(c_1)=P(c_2)=1/2$

如果摸到一个红球，那么，这个信封有1美元的的概率是

$$P(c_1|R)=P(R|c_1)*P(c_1)/P(R)=0.6$$

如果摸到一个黑球，那么，这个信封有1美元的的概率是

$$P(c_1|B)=P(B|c_1)*P(c_1)/P(B)=3/7$$



第七章：朴素贝叶斯学习

□ 贝叶斯学习

贝叶斯公式：

$$P(h|D) = \frac{P(h) * P(D|h)}{P(D)}$$

贝叶斯学习的目的是对于训练样本 D ，找出一个最靠谱的猜测 h (假设)，使得后验概率达到最大。

也就是计算下列模型的最优解。

$$h_{MAP} = \operatorname{argmax}_{h \in H} P(h|D)$$

其中 H 是各种“猜测”的集合。 h_{MAP} 是最佳的“最靠谱”的猜测，满足后验最大。

将上面的模型写完整：

$$h_{MAP} = \operatorname{argmax}_{h \in H} P(h|D) = \operatorname{argmax}_{h \in H} \frac{P(h) * P(D|h)}{P(D)} = \operatorname{argmax}_{h \in H} P(h) * P(D|h)$$

注： $P(D)$ 是不依赖于 h 的常量。



第七章：朴素贝叶斯学习

□ 贝叶斯学习

最大后验假设：在许多学习场景中，学习器考虑候选假设集合 H ，并在其中寻找给定数据 D 时，**可能性**最大的假设 $h \in H$ ，即为最大后验假设（Maximum a Posteriori, MAP），也称为MAP假设，定义为 h_{MAP} 。

确定MAP假设 h_{MAP} 的方法，是利用贝叶斯公式计算集合 H 中每个候选假设的后验概率。即

$$h_{MAP} = \operatorname{argmax}_{h \in H} P(h|D) = \operatorname{argmax}_{h \in H} \frac{P(h) * P(D|h)}{P(D)}$$



第七章：朴素贝叶斯学习

□ 贝叶斯学习

贝叶斯模型：

$$h_{MAP} = \operatorname{argmax}_{h \in H} P(h) * P(D|h)$$

- ◆ 当设计者不清楚先验时，可将所有假设的先验 $P(h)$ 均设为相同的值，即 $P(h) = \frac{1}{H}$ ，此时，模型变成了 $h_{MAP} = \operatorname{argmax}_{h \in H} P(D|h)$ ， $P(D|h)$ 常被称为给定 h 时，数据 D 的似然，而使 $P(D|h)$ 最大的假设，称为极大似然假设。
- ◆ 可以看出，贝叶斯模型实际上是先验与数据似然函数的乘积。如何理解先验？先验在很多时候，能够提供有力的“人为”帮助，提升模型的分类能力和鲁棒。

模型的抽象含义是：对于给定观测数据 D ，一个猜测 h 是好是坏，取决于“这个猜测本身的可能性大小（先验概率 $P(h)$ ，**Prior**）”和“这个猜测生成我们观测到的数据的可能性大小”（似然 $P(D|h)$ ，**Likelihood**）的乘积



第七章：朴素贝叶斯学习

□ 贝叶斯学习

先验的理解

例：用户输入一个错误单词**Thet**，请判断用户实际真正想输入的单词：“**They**” or “**That**”？

分析：假设“**They**”的后验概率 $P(\text{They}|\text{Thet})=P(\text{Thet}|\text{They}) * P(\text{They})/P(\text{Thet})$;

假设“**That**”的后验概率 $P(\text{That}|\text{Thet})=P(\text{Thet}|\text{That}) * P(\text{That})/P(\text{Thet})$;

其中， $P(\text{They})$ 和 $P(\text{That})$ 为假设的先验概率。

1. 若根据实际使用频率，单词**That**的使用频率要高于**They**，也就是说，用户实际想输入**That**的可能性更高，即 $P(\text{They}) < P(\text{That})$
2. 若根据键盘中字母的排列，字母**t**和字母**y**是紧密靠近的，因此，用户实际想输入**they**的可能性更高，即 $P(\text{They}) > P(\text{That})$

根据不同的先验知识，先验概率的值不同，因此先验知识的形式是多样的。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯学习

朴素贝叶斯到底“朴素”在哪？也就是朴素贝叶斯的假设是，当类假设 h 给定时，特征之间相互条件独立。

换句话说，在给定假设 h 的情况下，观察到联合的 $d_1, d_2, \dots, d_n$ 的概率等于单独特征（属性）的概率乘积：

$$P(d_1, d_2, \dots, d_n|h) = \prod_i P(d_i|h)$$

代入到贝叶斯模型中，有

$$h_{MAP} = \operatorname{argmax}_{h \in H} P(h|D) = \operatorname{argmax}_{h \in H} \frac{P(h) * \prod_i P(d_i|h)}{P(D)} = \operatorname{argmax}_{h \in H} P(h) * \prod_i P(d_i|h)$$

“朴素”即“特征独立假设”的好处就是大大降低了估计带来计算复杂度。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯学习

目标值PlayTennis的14个训练样例

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

问题：结合朴素贝叶斯方法，
对下面的新实例（4个特征属性）
进行分类：

Outlook=**Sunny**
Temperature=**Cool**
Humidity=**High**
Wind=**Strong**

PlayTennis=Yes or No?



第七章：朴素贝叶斯学习

□ 朴素贝叶斯学习

$$h_{MAP} = \operatorname{argmax}_{h \in H = \{Yes, No\}} P(h) * \prod_i P(d_i|h)$$

分析：结合朴素贝叶斯分类器模型，分别把 $h=Yes$ 和 $h=No$ 两种假设下的后验概率计算出来，最大值对应的假设就是我们的答案。

◆ 当 $h=Yes$ 时，

$$\begin{aligned} P(h) * \prod_i P(d_i|h) &= P(Yes) * P(Sunny|Yes) * P(Cool|Yes) * P(High|Yes) * p(Strong|Yes) \\ &= \frac{9}{14} * \frac{2}{9} * \frac{3}{9} * \frac{3}{9} * \frac{3}{9} = 0.0053 \end{aligned}$$

◆ 当 $h=No$ 时，

$$\begin{aligned} P(h) * \prod_i P(d_i|h) &= P(No) * P(Sunny|No) * P(Cool|No) * P(High|No) * p(Strong|No) \\ &= \frac{5}{14} * \frac{3}{5} * \frac{1}{5} * \frac{4}{5} * \frac{3}{5} = 0.0206 \end{aligned}$$

显然， $P(No) * \prod_i P(d_i|No) = 0.0206 > P(Yes) * \prod_i P(d_i|Yes) = 0.0053$ ，所以，
PlayTennis=No, 今天的天气不适合打网球



第七章：朴素贝叶斯学习

□ 贝叶斯学习方法的特性

1. 贝叶斯方法允许假设做出不确定性的预测（如：某一癌症病人有85%的机会康复）。
2. 先验知识可以与观察数据一起决定假设的最终概率。在贝叶斯学习中，先验知识的形式可以是：1）每个候选假设的先验概率；2）每个候选假设在观察数据上的概率分布。
3. 新的样本分类可以由多个假设一起作出预测，用它们的概率来加权（最优贝叶斯）。



第七章：朴素贝叶斯学习

□ 采用朴素贝叶斯对垃圾邮件进行分类

1. 样本：1000封邮件，每个邮件被标记为垃圾邮件或者正常邮件。设有400封垃圾邮件，600封正常邮件。
2. 分类目标（问题）：给定第1001封邮件，确定它是垃圾邮件还是非垃圾邮件？
3. 方法：朴素贝叶斯。

该如何建模和实现？

（一）分析：令待分类的这封邮件为 $D=[d_1, d_2, \dots, d_n]$ ，由 n 个单词组成。垃圾邮件定义为 h_1 ，正常邮件定义为 h_2 。设400封垃圾邮件中的单词总数为 N_1 ，600封正常邮件中的单词总数为 N_2 。



第七章：朴素贝叶斯学习

□ 采用朴素贝叶斯对垃圾邮件进行分类

（二）如果能够计算出垃圾邮件 h_1 假设的后验概率和正常邮件 h_2 假设的后验概率，两者概率最大的假设，即为该封邮件的类别。采用贝叶斯公式。

数学描述：
$$P(h_1|D) = \frac{P(D|h_1)P(h_1)}{P(D)}, P(h_2|D) = \frac{P(D|h_2)P(h_2)}{P(D)}$$

$P(D)$ 为与 h 无关的常量，因此，只需计算 $P(D|h_1)P(h_1)$ 和 $P(D|h_2)P(h_2)$ 即可。



第七章：朴素贝叶斯学习

□ 采用朴素贝叶斯对垃圾邮件进行分类

（三）为了计算 $P(D|h_1)P(h_1)$ 和 $P(D|h_2)P(h_2)$ ，可以先计算先验概率 $P(h_1)$ 和 $P(h_2)$ 。

$$P(h_1) = \frac{400}{1000} = 0.4; \quad P(h_2) = \frac{600}{1000} = 0.6$$

那如何计算 $P(D|h_1)$ 和 $P(D|h_2)$ ？

$$P(D|h_1) = P(d_1, d_2, \dots, d_n|h_1)$$

然后呢？问题变得复杂。由于数据的稀疏性以及语言的多样性，完全出现两封完全相同的邮件（文本）是几乎不可能的。也就是说， n 个单词 $(d_1, d_2, \dots, d_n)$ 在所有邮件中同时出现的概率几乎为0。那怎么办？



第七章：朴素贝叶斯学习

□ 采用朴素贝叶斯对垃圾邮件进行分类

（四）当给定某假设时，采用特征的条件独立性假设，即朴素贝叶斯。

那如何计算 $P(D|h_1)$ 和 $P(D|h_2)$ ？

$$\begin{aligned} P(D|h_1) &= P(d_1, d_2, \dots, d_n|h_1) = P(d_1|h_1) * P(d_2|h_1) * \dots * P(d_n|h_1) \\ &= \prod_{i=1}^n P(d_i|h_1) \end{aligned}$$

现在问题变得非常简单，计算 $P(d_i|h_1)$ 的概率，只需要统计单词 d_i 在400封垃圾邮件中出现的频次，然后相乘。同理， $P(D|h_2)$ 的概率也是这么计算。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

方法的基本定义：

1. 设 $\mathbf{x} = \{x_1, x_2, x_3, \dots, x_d\}$ 为一个待分类的样本，由 d 个特征属性构成；
2. 有类别集合 $C = \{y_1, y_2, \dots, y_C\}$ ；
3. 计算 $P(y_1|\mathbf{x}), P(y_2|\mathbf{x}), \dots, P(y_C|\mathbf{x})$ ；
4. 如果 $P(y_k|\mathbf{x}) = \max\{P(y_1|\mathbf{x}), P(y_2|\mathbf{x}), \dots, P(y_C|\mathbf{x})\}$, 则 $\mathbf{x}$ 属于 y_k 这个类别。

在以上4个步骤中，只要成功完成第3步，即可实现样本 $\mathbf{x}$ 的分类。

如何实现第3步？

利用前面所学习的朴素贝叶斯学习算法！



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

第3步计算 $P(y_1|\mathbf{x})$, $P(y_2|\mathbf{x})$, ..., $P(y_c|\mathbf{x})$ 条件概率的具体步骤：

1. 已知类别的训练样本集（如14天的天气数据，以及是否打了网球）
2. 统计每个类别下，各个特征属性的条件概率（出现频次），即

$$P(x_1|y_1), P(x_2|y_1), \dots, P(x_d|y_1);$$

$$P(x_1|y_2), P(x_2|y_2), \dots, P(x_d|y_2);$$

$$\vdots$$

$$P(x_1|y_c), P(x_2|y_c), \dots, P(x_d|y_c);$$

3. 计算各类别的先验概率 $P(y_1), P(y_2), \dots, P(y_c)$ （统计各类别出现的频率）
4. 根据朴素贝叶斯方法，各个特征属性是条件独立的。利用贝叶斯定理(公式)，

$$\text{我们要计算的 } P(y_i|\mathbf{x}) = \frac{P(\mathbf{x}|y_i)P(y_i)}{P(\mathbf{x})} = \frac{P(x_1|y_i) \cdot P(x_2|y_i) \cdots P(x_d|y_i) \cdot P(y_i)}{P(\mathbf{x})} = \frac{P(y_i) \cdot \prod_{j=1}^d P(x_j|y_i)}{P(\mathbf{x})},$$

其中， $P(\mathbf{x})$ 是数据的先验，可以当做常数处理，不影响后验概率的比较。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

✓ Laplacian校验(零概率问题)

当特征属性为离散值时，计算条件概率 $P(x_j|y_i)$ 即在类 y_i 条件下统计属性 x_j 出现的次数，可能会遇到出现次数为0，从而导致该条件概率为0，会影响分类器的性能（训练样本较少时会遇到该问题）。

这种情况下，可以利用Laplacian校验，即统计每一类下的某特征属性出现的次数时，全部计数加1，可避免0概率的出现。

当训练样本数量充分大时，不影响分类器的结果。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

前面的例子中，考虑了特征值属性为离散值时的情况，在实际问题中，通常是特征值属性为连续值的情况。

那该如何计算完成上述条件概率的计算？即

$$P(x_1|y_i), P(x_2|y_i), \dots, P(x_d|y_i), \forall i = 1, \dots, C$$

➤ 当特征值为连续值时，通常假设特征属性服从高斯分布，即

$$P(\mathbf{x}|y_i) = N(\mu_i, \sigma_i)$$

因此，
$$P(x_j|y_i) = \frac{1}{\sqrt{2\pi}\sigma_i} e^{-\frac{(x_j - \mu_i)^2}{2\sigma_i^2}}$$

➤ 如何计算每一类各特征属性的均值以及方差？即 μ_i 和 σ_i ($i = 1, \dots, C$) 直接利用期望和方差公式即可。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

➤ 最小错误率贝叶斯分类器

利用后验概率 $P(y_i|\mathbf{x})$ ，当 $P(y_k|\mathbf{x})$ 为所有后验概率中最大值时，待分类样本 $\mathbf{x}$ 属于 y_k 类；即 $P(y_k|\mathbf{x}) = \max_{i=1,\dots,C} P(y_i|\mathbf{x})$

➤ 最大似然比贝叶斯分类器

对于两类问题，当 $P(\mathbf{x}|y_i)P(y_i) > P(\mathbf{x}|y_j)P(y_j)$ 时，判决 $\mathbf{x}$ 属于 y_i 类；也就是当

$\frac{P(\mathbf{x}|y_i)}{P(\mathbf{x}|y_j)} > \frac{P(y_j)}{P(y_i)}$ 时，判决 $\mathbf{x}$ 属于 y_i 类。其中， $\frac{P(\mathbf{x}|y_i)}{P(\mathbf{x}|y_j)}$ 为似然比。 $\frac{P(y_j)}{P(y_i)}$ 为判决门限。

➤ 最小风险贝叶斯分类器



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

➤ 最小风险贝叶斯分类器

顾名思义，对于待分类样本，有多种决策，具有最小风险的决策（分类）即为最终的分类。作如下定义：

1. 决策 ω_i ：把待识别样本 $\mathbf{x}$ 分类到 y_i 类中；
2. 损失 ρ_{ij} ：把真实属于 y_i 类的样本 $\mathbf{x}$ 错误分类到 y_j 类中；
3. 条件风险 $R(\omega_i|\mathbf{x})$ ：对待识别样本 $\mathbf{x}$ 采取决策 ω_i ，产生的可能风险；条件风险的计算公式如下： $R(\omega_i|\mathbf{x}) = \sum_{j=1}^C \rho_{ij} P(y_j|\mathbf{x})$
4. 最小风险贝叶斯分类器就是计算具有最小风险的决策 ω_k

$$R(\omega_k|\mathbf{x}) = \min_{i=1,\dots,C} R(\omega_i|\mathbf{x})$$

那么待识别样本 $\mathbf{x}$ 采取决策 ω_k ，也就是将被分类到 y_k 类中。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

➤ 最小风险贝叶斯分类器

$$\text{当 } \rho_{ij} = \begin{cases} 0, i = j \\ 1, i \neq j \end{cases} \text{ 时,}$$

有

$$\min_{i=1,\dots,c} R(\omega_i|\mathbf{x}) = \min_{i=1,\dots,c} \sum_{j=1}^c \rho_{ij} P(y_j|\mathbf{x}) = \min_{i=1,\dots,c} \sum_{j=1, j \neq i}^c P(y_j|\mathbf{x})$$

$$= \min_{i=1,\dots,c} 1 - P(y_i|\mathbf{x}) \quad \text{注: } \sum_{j=1}^c P(y_j|\mathbf{x}) = 1$$

$$= \max_{i=1,\dots,c} P(y_i|\mathbf{x}) \quad \text{注: 最小错误率贝叶斯分类器}$$

可见，最小错误率分类器是最小风险贝叶斯分类器的一种特例。



第七章：朴素贝叶斯学习

□ 朴素贝叶斯分类器

➤ 分类器性能评价

在机器学习中，基本上分类器的评价是通过分类器正确率(识别率)进行衡量。

分类器正确率：指被正确分类的样本数占有所有样本数的比率。

分类正确率的计算通常是基于一部分新的测试样本集，而非训练样本集。

因为训练集往往会因为过拟合，而得出较高的正确率，但不能作为分类器的评价指标。



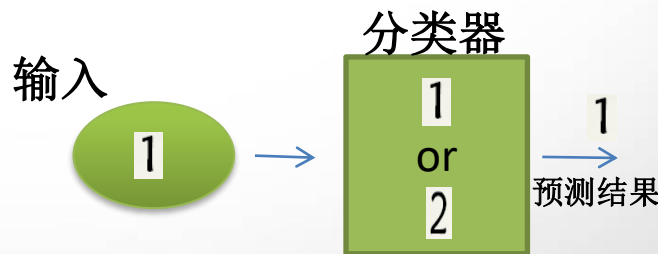
第八章： 分类器

第八章：分类器

□ 定义

- 对于一组训练样本集 $x_1, x_2, \dots, x_N$, 每个样本由 D 个属性组成, 形成 D -维矢量。其中, 每个样本分别采用一个标签 $y_1, y_2, \dots, y_N$ 描述所属类别, 样本集由 C 个类别组成。对于给定的新样本 x_{new} , 判定其所属的类别, 被称为**分类**。
- 利用训练集生成的分类模型/方法/算法, 被称为**分类器(Classifier)**。

分类问题是机器学习的核心问题, 也是人工智能、模式识别、计算机视觉、图像理解、文本分类、数据挖掘等应用领域要解决的关键问题。





第八章：分类器

□ 分类器种类

机器学习发展至今，衍生出大量的分类器模型，本课程主要介绍以下几类分类器：

➤ 概率分类器（参见前面的章节）

贝叶斯分类器

朴素贝叶斯分类器

Logistic回归

➤ 非概率分类器

K-近邻分类器（欧式距离）

大间隔分类器（支持向量机）

线性判别分类器（投影）

神经网络分类器（多层感知器、BP网络、深度卷积网络—第九章讲）



第八章：分类器

不同于概率分类器，对于每个样本 $\mathbf{x}$ ，计算 $P(y=c|\mathbf{x})$ ，提供类别 $y=c$ 的可能性，非概率分类器输出的是一个指定的类别，即 $y(\mathbf{x})=c$ 。

□ K-近邻分类器（K-nearest neighbors, KNN）

K-近邻分类器是基于欧式距离提出的较为直观的分类器方法，思想极其简单，不需要对训练集进行模型训练，完全无参数。

假设有 N 个训练样本，每个样本分别由属性 $\mathbf{x}$ 和标记 y 进行描述。对于一个新样本 $\mathbf{x}_{\text{new}}$ ，如何利用KNN进行分类？

Step 1: 分别计算 $\mathbf{x}_{\text{new}}$ 与训练集中 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ 之间的距离；

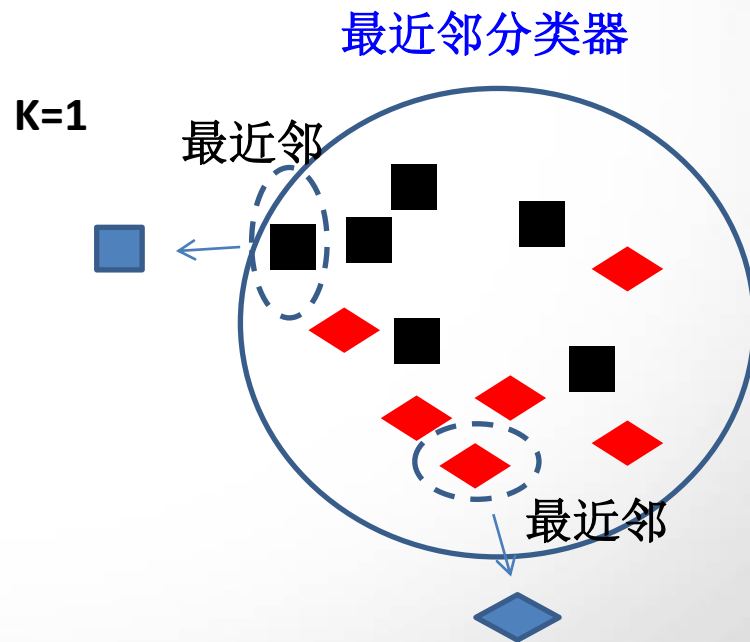
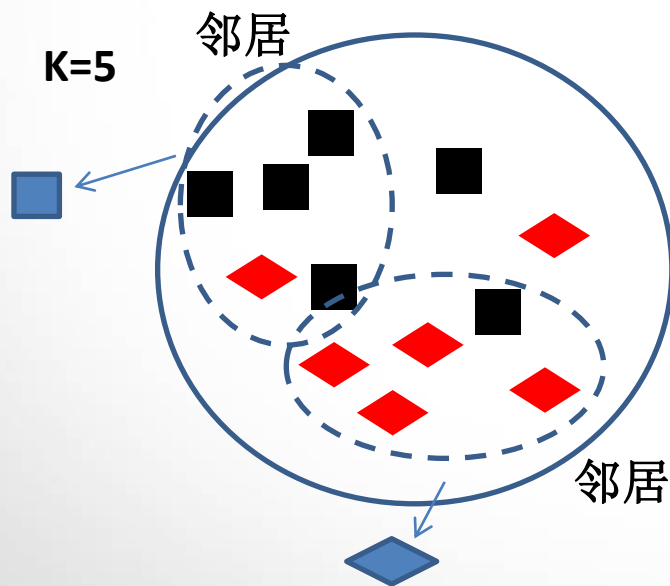
Step 2: 找出与 $\mathbf{x}_{\text{new}}$ 最近的 K 个样本 $\mathbf{x}_i$ ；

Step 3: 根据“服从多数”原则， $\mathbf{x}_{\text{new}}$ 的类别即为 K 个样本中最多的类。

第八章：分类器

□ K-近邻分类器 (K-nearest neighbors, KNN)

KNN分类器的思想用一个图进行描述





第八章：分类器

□ K-近邻分类器

距离计算

样本 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 之间的欧式距离表达式

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{\sum_{l=1}^d (x_i^l - x_j^l)^2}$$

K值的选择

- K值通常是一个奇数，即选取奇数个邻居点，从而可以通过投票确定类别；
- 如果K太小，分类器容易被噪声干扰；比如K=1时，最近邻点如果被噪声污染，则分类错误。
- 如果K太大，可能会永远无法进行正确的分类（各类样本不均衡问题），比如两类问题，第I类有10个样本，第II类有30个样本，如果K>21，则新样本将会一直被识别为第II类。



第八章：分类器

□ 其他距离

距离通常用于度量两个样本之间的相似性，在分类器中经常用到。

- ✓ 马氏距离（协方差）
- ✓ 曼哈顿距离（城市间街道）
- ✓ 切比雪夫距离（ $\max_l |x_i^l - x_j^l|$ ）
- ✓ 余弦距离（向量夹角 $\cos\theta = \mathbf{x}_i \cdot \mathbf{x}_j / |\mathbf{x}_i| |\mathbf{x}_j|$ ）
- ✓ 汉明距离（信息编码）
- ✓ 相关性（统计协方差 $\rho_{\mathbf{x}_i, \mathbf{x}_j}$ ）

□ 度量学习

度量学习不同于上述“已知”的距离表达式，而是通过机器学习建模的方式，学习一个最佳的距离度量，使得对特定的问题，具有最佳的识别性能。

Re-ID要解决的问题:



Re-ID要解决的问题:

Camera #1:



Camera #2:

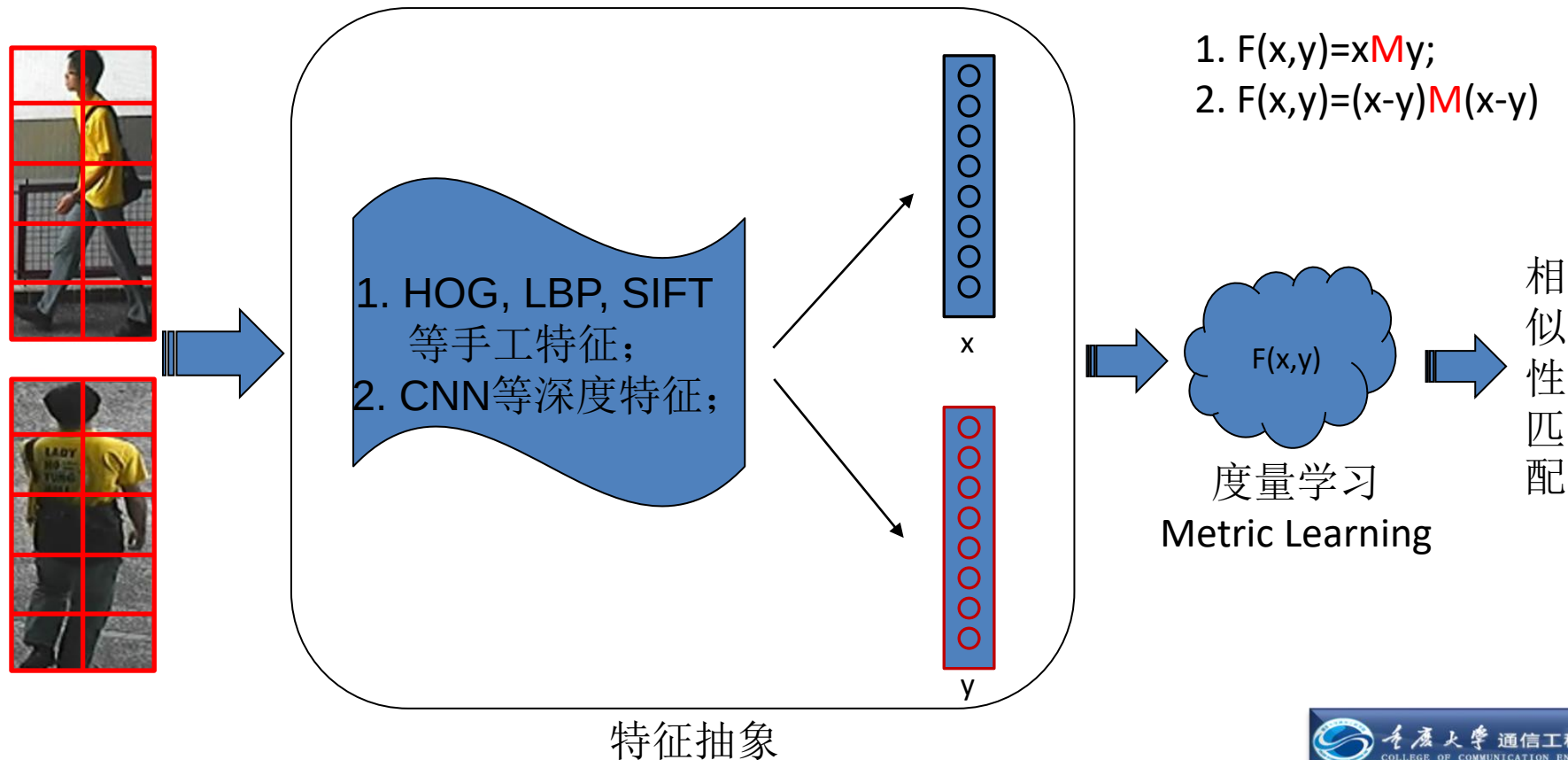


特征抽象

距离判定



Re-ID要解决的问题:



Re-ID的度量学习原理:

三元组样本集构造: x, x_p, x_n



x

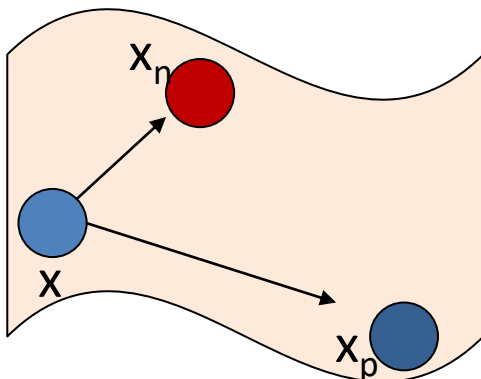


x_p

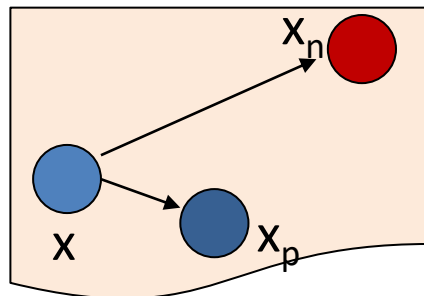


x_n

Metric learning



差的度量



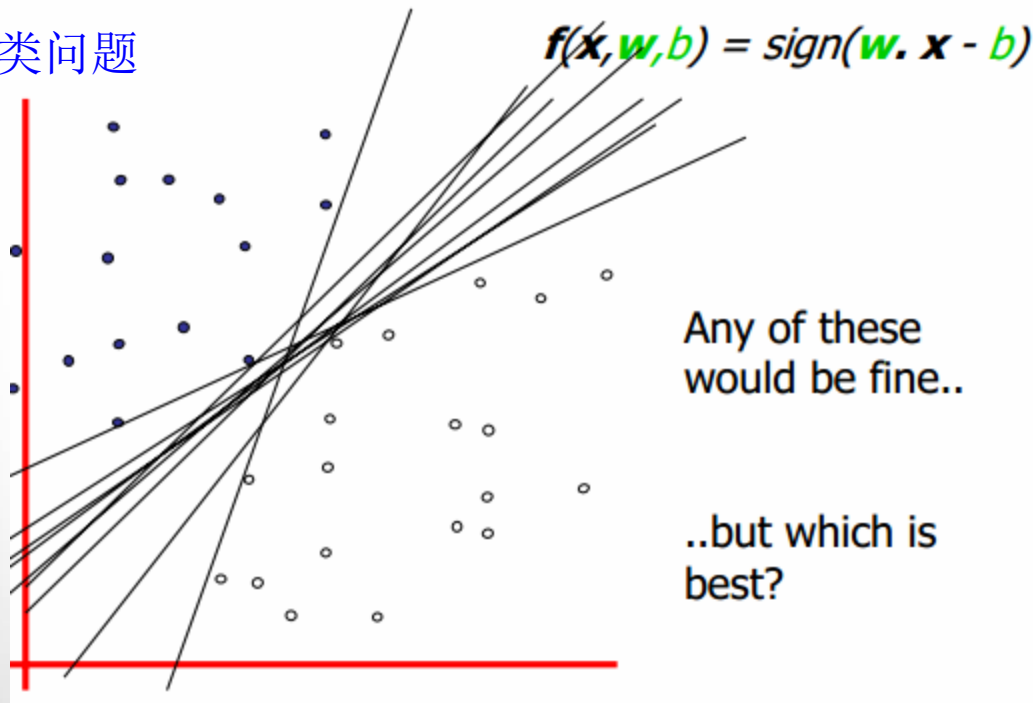
好的度量

Triplet Loss(三元组): $\min \sum_i^N \left[\|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha \right]_+$

第八章：分类器

□ 支持向量机（Support Vector Machine, SVM, Vapnik'95）

考虑线性分类问题



假设给定一个特征空间上的训练数据集 $T = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2) \dots (\mathbf{x}_N, y_N)\}$, 其中, $\mathbf{x}_i \in \mathbb{R}^n$, $y_i \in \{+1, -1\}$, $i=1, 2, \dots, N$ 。 $\mathbf{x}_i$ 为第 i 个特征向量, 也称为实例, y_i 为 $\mathbf{x}_i$ 的类标记; 当 $y_i = +1$ 时, 称 $\mathbf{x}_i$ 为正例; 当 $y_i = -1$ 时, 称 $\mathbf{x}_i$ 为负例。 $(\mathbf{x}_i, y_i)$ 称为样本点。



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机

定义：给定线性可分训练数据集，通过间隔最大化或等价的求解相应的**凸二次规划问题**学习得到的分离超平面为

$$\mathbf{w}\mathbf{x} + b = 0,$$

相应的分类决策函数

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}\mathbf{x} + b)$$

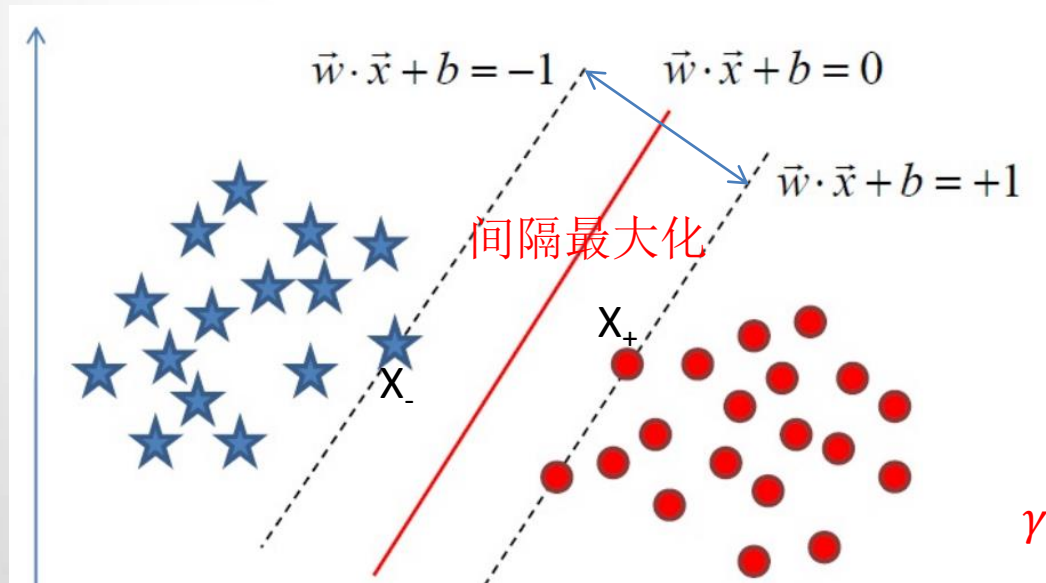
该决策函数称为线性可分支持向量机。

第八章：分类器



□ 支持向量机 (Support Vector Machine, SVM)

➤ 线性可分支持向量机



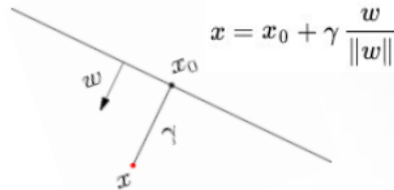
定义：函数间隔和几何间隔

函数间隔：给定的训练数据集 T 和超平面 (w, b) ，定义超平面 (w, b) 关于样本点 (x_i, y_i) 的函数间隔为：

$$\hat{\gamma} = y(w^T x + b) = y f(x)$$

几何间隔：两个异类支持向量之差在法向量上的投影，即

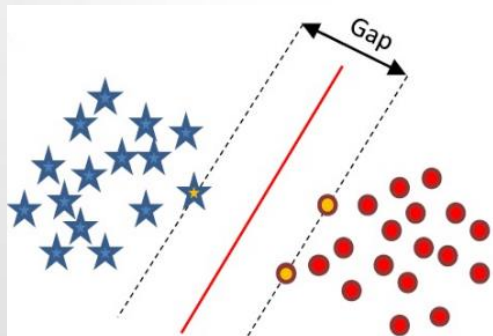
$$\gamma = \frac{|w^T x + b|}{\|w\|} = \frac{y(w^T x + b)}{\|w\|} = \frac{1}{\|w\|}$$



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机



最大间隔分类超平面

$$\begin{aligned} & \max_{w,b} \frac{1}{\|w\|} \\ \text{s.t. } & y_i(w \cdot x_i + b) \geq 1, \quad i = 1, 2, \dots, N \end{aligned}$$



$$\begin{aligned} & \min_{w,b} \frac{1}{2} \|w\|^2 \\ \text{s.t. } & y_i(w \cdot x_i + b) \geq 1, \quad i = 1, 2, \dots, N \end{aligned}$$



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机

线性可分支持向量机模型

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y_i (w \cdot x_i + b) \geq 1, \quad i = 1, 2, \dots, N \end{aligned}$$

带有约束的凸二次规划问题。如何求解？



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机（硬间隔最大化）

拉格朗日乘子法

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1)$$

原问题的对偶问题：极大极小问题

$$\min_{\mathbf{w}, b} \max_{\alpha} L(\mathbf{w}, b, \alpha)$$

$$\max_{\alpha} \min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha)$$

→ 先求内层最小值

将拉格朗日函数 $L(\mathbf{w}, b, \alpha)$ 分别对 $\mathbf{w}$, b 求偏导并令其为0:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_{i=1}^n \alpha_i y_i = 0$$

补充:

$$\min f(\mathbf{x})$$

$$\text{s.t. } g(\mathbf{x}) \leq 0$$

$$L(\mathbf{x}, u) = f(\mathbf{x}) + u g(\mathbf{x}), u > 0$$

$$\text{所以 } f(\mathbf{x}) = \max L(\mathbf{x}, u)$$

依据：拉格朗日的对偶性，即广义拉格朗日的极小化问题，可以拆分成两步。



第八章：分类器

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_{i=1}^n \alpha_i y_i = 0$$

□ 支持向量机 (Support Vector Machine, SVM)

➤ 线性可分支持向量机 (硬间隔最大化)

将上式代入拉格朗日函数 $L(\mathbf{w}, b, \alpha)$ 中，得到

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b, \alpha) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] \\ &= \frac{1}{2} \mathbf{w}^T \mathbf{w} - \mathbf{w}^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \\ &= \frac{1}{2} \mathbf{w}^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i - \mathbf{w}^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i - b \cdot 0 + \sum_{i=1}^n \alpha_i \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right)^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \end{aligned}$$

$$\max_{\alpha} \min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha)$$



$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$s.t. \alpha_i \geq 0, i = 1, 2, \dots, n$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$



$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \sum_{i=1}^N \alpha_i$$

$$s.t. \sum_{i=1}^N \alpha_i y_i = 0$$

$$\alpha_i \geq 0, \quad i = 1, 2, \dots, N$$

KKT条件



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机（硬间隔最大化）

KKT条件（Karush[1939], Kuhn和Tucker[1951]），即**确定某点为极值的必要条件**。

$$\begin{aligned} \min f(x) \\ \text{s.t. } g(x) \leq 0 \end{aligned}$$

KKT条件包括**原可行性** $g(x^*) \leq 0$ ，**对偶可行性** $\alpha \geq 0$ 和**互补松弛性** $\alpha \cdot g(x^*) = 0$

SVM中的KKT条件：

$$y_i(wx_i + b) - 1 \geq 0,$$

$$\alpha_i \geq 0,$$

$$\alpha_i[y_i(wx_i + b) - 1] = 0 \longrightarrow \text{支持向量}$$



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机（硬间隔最大化）

1.模型：
$$\min_{\alpha} \quad \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i$$

$$\text{s.t.} \quad \sum_{i=1}^N \alpha_i y_i = 0$$

$$\alpha_i \geq 0, \quad i=1,2,\dots,N$$

2.求得最优解 α^*

3. 计算

$$w^* = \sum_{i=1}^N \alpha_i^* y_i x_i$$

$$b^* = y_i - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j) \xrightarrow{\text{通常计算}} b^* = - \frac{\max_{y_i=-1} w^T x_i + \min_{y_i=1} w^T x_i}{2}$$

4.分离超平面

$$w^* x + b^* = 0$$

5.分类决策函数

$$f(x) = \text{sign}(w^* x + b^*)$$



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机（硬间隔最大化）

举例：给定3个数据点：正样本点 $x_1=(3,3)^T$ ， $x_2=(4,3)^T$ ，负样本点 $x_3=(1,1)^T$ ，利用线性可分支持向量机，求线性分类决策函数？

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i \\ = & \frac{1}{2} (18\alpha_1^2 + 25\alpha_2^2 + 2\alpha_3^2 + 42\alpha_1\alpha_2 - 12\alpha_1\alpha_3 - 14\alpha_2\alpha_3) - \alpha_1 - \alpha_2 - \alpha_3 \\ \text{s.t.} \quad & \alpha_1 + \alpha_2 - \alpha_3 = 0 \\ & \alpha_i \geq 0, \quad i = 1, 2, 3 \end{aligned}$$



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机（硬间隔最大化）

举例：给定3个数据点：正样本点 $x_1=(3,3)^T$ ， $x_2=(4,3)^T$ ，负样本点 $x_3=(1,1)^T$ ，利用线性可分支持向量机，求线性分类超平面？

✓ 将 $\alpha_1 + \alpha_2 = \alpha_3$ 带入目标函数，得到关于 α_1, α_2 的函数：

$$s(\alpha_1, \alpha_2) = 4\alpha_1^2 + \frac{13}{2}\alpha_2^2 + 10\alpha_1\alpha_2 - 2\alpha_1 - 2\alpha_2$$

✓ 对 α_1, α_2 求偏导并令其为0，易知 $s(\alpha_1, \alpha_2)$ 在点(1.5, -1)处取极值。

✓ 然而，该点不满足条件 $\alpha_2 \geq 0$ 。

当 $\alpha_1=0$ 时，最小值 $s(0, 2/13)=-2/13$

当 $\alpha_2=0$ 时，最小值 $s(1/4, 0)=-1/4$

✓ 于是， $s(\alpha_1, \alpha_2)$ 在 $\alpha_1=1/4$ ， $\alpha_2=0$ 时达到最小，此时， $\alpha_3 = \alpha_1 + \alpha_2 = 1/4$



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性可分支持向量机（硬间隔最大化）

举例：给定3个数据点：正样本点 $x_1=(3,3)^T$ ， $x_2=(4,3)^T$ ，负样本点 $x_3=(1,1)^T$ ，利用线性可分支持向量机，求线性分类超平面？

- ✓ 可见， $\alpha_1=\alpha_3=1/4$ 对应的点 x_1, x_3 是支持向量。
- ✓ 带入公式，求 w 和 b ：

$$w^* = \sum_{i=1}^N \alpha_i^* y_i x_i$$

$$b^* = y_i - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j)$$

✓ 分离决策函数为 $f(x) = \text{sign}\left(\frac{1}{2}x_1 + \frac{1}{2}x_2 - 2\right)$

得到 $w_1=w_2=0.5$ ， $b=-2$

- ✓ 因此，分离超平面为 $\frac{1}{2}x_1 + \frac{1}{2}x_2 - 2 = 0$

提问：对于 $\alpha > 0$ 时， $y(wx+b)-1=0$?



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性支持向量机（软间隔最大化）

若数据近似线性可分，则增加松弛因子 $\xi_i \geq 0$ ，使函数间隔加上松弛变量大于等于1 (软间隔)。这样，约束条件变成

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

SVM的模型变成

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i$$

惩罚项

$$\text{s.t. } y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \dots, N$$

$$\xi_i \geq 0, \quad i = 1, 2, \dots, N$$

$$\xi_i = \max(0, 1 - y_i(w \cdot x_i + b))$$



第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 线性支持向量机（软间隔最大化）

1. 构造SVM模型

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i$$

$$\text{s.t. } \sum_{i=1}^N \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, N$$

2. 求解 α^*

SMO(Sequential Minimal Optimization)

算法求解, 参见文献

John C. Platt, Sequential Minimal Optimization:

A Fast Algorithm for Training Support Vector machines

3. 计算 w 和 b

$$w^* = \sum_{i=1}^N \alpha_i^* y_i x_i$$

$$b^* = y_i - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j)$$

4. 求得分离超平面

$$w^* x + b^* = 0$$

5. 分类决策函数

$$f(x) = \text{sign}(w^* x + b^*)$$

$$= \text{sign}\left(\sum_{i=1}^N \alpha_i^* y_i x_i x + b^*\right)$$

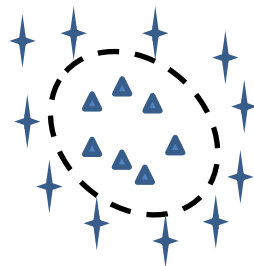
第八章：分类器

□ 支持向量机（Support Vector Machine, SVM）

➤ 非线性支持向量机（核函数+软间隔最大化）

目的：将线性不可分的数据(信号)，通过一个非线性函数 $\varphi(\cdot)$ ，映射到高维特征空间，使得线性可分。

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, N \end{aligned}$$



$$\varphi(x_i) \cdot \varphi(x_j)$$

$$k(x_i, x_j)$$

$$e^{-\sigma \|x_i - x_j\|^2}$$



第八章：分类器

□ 子空间学习（Subspace Learning）

➤ 概念

- ✓ 子空间学习是指对于一给定的分类任务，如何为原数据寻找一个低维子空间，使得在该低维子空间内实现更好的分类，而计算复杂度降低；
- ✓ 子空间学习的方法，一般通过投影的方式，实现高维特征（原数据空间）向低维子空间的映射。这个过程也被称为“降维”；

➤ 种类

- ✓ 线性子空间学习
- ✓ 非线性子空间学习

通常情况下，非线性子空间学习方法是通过对“核函数”进行建模实现。整个过程，是在线性子空间学习的方法上进行扩展。



第八章：分类器

□ 子空间学习（Subspace Learning）

► 线性子空间学习

定义原始高维特征集为 $\mathbf{x}$ ，子空间学习从数学上是为了学习一个投影（或变换 $\mathbf{W}$ ），使得特征集 $\mathbf{x}$ 在低维子空间的表达式为

$$\mathbf{Y} = \mathbf{W}^T \mathbf{X}$$

线性子空间学习的目的是为了获得变换矩阵 $\mathbf{W}$ ，使得低维子空间表达 $\mathbf{Y}$ 能更好的实现分类。

► 种类

- ✓ 主成分分析(PCA);
- ✓ Fisher判别分析（线性判别分析，LDA）；
- ✓ 流形学习（局部保持投影, LPP）；



第八章：分类器

□ 子空间学习 (Subspace Learning)

► 主成分分析(PCA)

主成分分析是一种无监督的降维方法，在整个过程中，不需要知道样本集 $\mathbf{X}$ 的标签。数学上，**PCA**是一种均方误差最小意义下的投影方法。

定义高维特征集 $\mathbf{X}$ ，存在一个投影（或变换 $\mathbf{W}$ ），即特征集 $\mathbf{X}$ 在低维子空间的表达形式为

$$\mathbf{Y} = \mathbf{W}^T \mathbf{X}$$

使得 $\mathbf{Y}$ 保留高维特征集 $\mathbf{X}$ 中的大部分有效信息（能量最大或方差最大），即满足：

$$\max_{\mathbf{W}} \|\mathbf{Y} - \boldsymbol{\mu}\|_F^2 = \max_{\mathbf{W}} \|\mathbf{W}^T \mathbf{X} - \mathbf{W}^T \mathbf{m}\|_F^2$$

$$= \max_{\mathbf{W}} \text{Tr}(\mathbf{W}^T (\mathbf{X} - \mathbf{m})(\mathbf{X} - \mathbf{m})^T \mathbf{W})$$

$$= \max_{\mathbf{W}} \text{Tr}(\mathbf{W}^T \boldsymbol{\Sigma} \mathbf{W}), s. t. \mathbf{W}^T \mathbf{W} = \mathbf{I}$$

W的求解采用特征值分解

$\boldsymbol{\Sigma} = (\mathbf{X} - \mathbf{m})(\mathbf{X} - \mathbf{m})^T$ 为协方差矩阵



第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ K-L (Karhunen-Loève) 变换

与主成分分析原理相同，KL变换的本质是通过线性变换（正交变换）降低维度，也是均方误差最小意义下的投影方法。

定义一组向量 $\mathbf{x}$ ，如何求一个投影矩阵（或变换矩阵 $\mathbf{U}$ ），满足上述条件？

假设：设 $\mathbf{U}$ 是一个完备正交基的集合，则向量 $\mathbf{x}$ 可以由该**完备正交基**集合线性表示

$$\mathbf{x} = \sum_{i=1}^{\infty} \alpha_i \mathbf{u}_i, \text{ 则 } \alpha_i = \mathbf{u}_i^T \mathbf{x}$$

此时，如果我们只考虑有限估计，即采用有限的 d 个正交基近似表示 $\mathbf{x}$ ，有

$$\hat{\mathbf{x}} = \sum_{i=1}^d \alpha_i \mathbf{u}_i$$



第八章：分类器

□ 完备正交基（知识回顾）

1. 连续函数集合的正交性：

设正交函数集合 $U = \{u_0(t), u_1(t), \dots\}$ ，则有

$$\int_{t_0}^{t_0+T} u_m(t)u_n(t)dt = \begin{cases} C, & \text{if } m = n \\ 0, & \text{otherwise} \end{cases}$$

当 $C=1$ 时，则该正交函数集合为**归一化**的正交函数集合。

多维空间坐标系的基轴方向互相正交。



第八章：分类器

□ 完备正交基（知识回顾）

2. 离散情况

对于n个正交向量集合 $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_n]$ ，其中，

$$\begin{aligned}\mathbf{a}_1 &= [a_{11}, a_{21}, \dots, a_{n1}]^T \\ \mathbf{a}_n &= [a_{1n}, a_{2n}, \dots, a_{nn}]^T\end{aligned}$$

$$\text{则有 } \sum_{k=1}^n a_{k,i} a_{k,j} = \begin{cases} C, & \text{if } i = j \\ 0, & \text{if } i \neq j \end{cases}$$

若 $C=1$ ，则该集合为归一化的正交向量集合。

那么，矩阵 $\mathbf{A}$ 是一个正交矩阵，并满足 $\mathbf{A}\mathbf{A}^T = \mathbf{A}^T\mathbf{A} = \mathbf{I}$



第八章：分类器

□ 完备正交基（知识回顾）

正交函数集合U的完备性

若 $f(x)$ 是定义在 t_0 和 $t_0 + T$ 区间的实值信号，平方可积。可以表示为

$$f(x) = \sum_{n=0}^{\infty} a_n u_n(x)$$

对任意小的 $\varepsilon > 0$ ，存在充分大的 N ，用 N 个有限展开式估计 $f(x)$ ，

$$\tilde{f}(x) = \sum_{n=0}^{N-1} a_n u_n(x)$$

并有 $\int_{t_0}^{t_0+T} |f(x) - \tilde{f}(x)|^2 dx < \varepsilon$ ，

那么称函数集合U是完备的。（集合足够大）

傅里叶变换即是一种正交变换

$$F(u) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) e^{-\frac{j2\pi ux}{N}}, u = 0, 1, \dots, N-1$$
$$f(x) = \sum_{u=0}^{N-1} F(u) e^{\frac{j2\pi ux}{N}}, x = 0, 1, \dots, N-1$$



第八章：分类器

□ 子空间学习 (Subspace Learning)

均方误差:

$$\begin{aligned}\varepsilon &= E[(x - \hat{x})^T(x - \hat{x})] = E\left[\sum_{i=d+1}^{\infty} \alpha_i^2\right] = E\left[\sum_{i=d+1}^{\infty} u_i^T x x^T u_i\right] \\ &= \sum_{i=d+1}^{\infty} u_i^T E(x x^T) u_i = \sum_{i=d+1}^{\infty} u_i^T \Sigma_x u_i\end{aligned}$$

均方误差最小化原则: $\min_U \varepsilon = \sum_{i=d+1}^{\infty} u_i^T \Sigma_x u_i$

仅当 $\Sigma_x u_i = \lambda_i u_i$ 时, 取得最小值: $\varepsilon = \sum_{i=d+1}^{\infty} \lambda_i$

结论: 前 d 个最大特征值所对应的特征向量 u_i 所构成的矩阵 U 即是最终解。



第八章：分类器

□ 子空间学习（Subspace Learning）

➤ Fisher判别分析(LDA)

与PCA不同，LDA是一种有监督的降维方法，需要利用样本集的标签信息，从而使得获得的低维子空间表达，能够更好地实现分类任务。

两者区别在于，PCA是为了找到一种原始数据的最佳表达，而LDA是为了找到一种有利于分类的最佳表达。

比如：在字符识别中，区分字母O和字母Q。PCA方法会保留两个字母最相似的部分即O，而抛弃字母Q的“尾巴”，然而对于LDA则会尽力保住这个“尾巴”，因为更易于区分。

定义高维特征集 $\mathbf{x}$ ，存在一个投影（或变换 $\mathbf{W}$ ），即特征集 $\mathbf{x}$ 在低维子空间的表达形式为

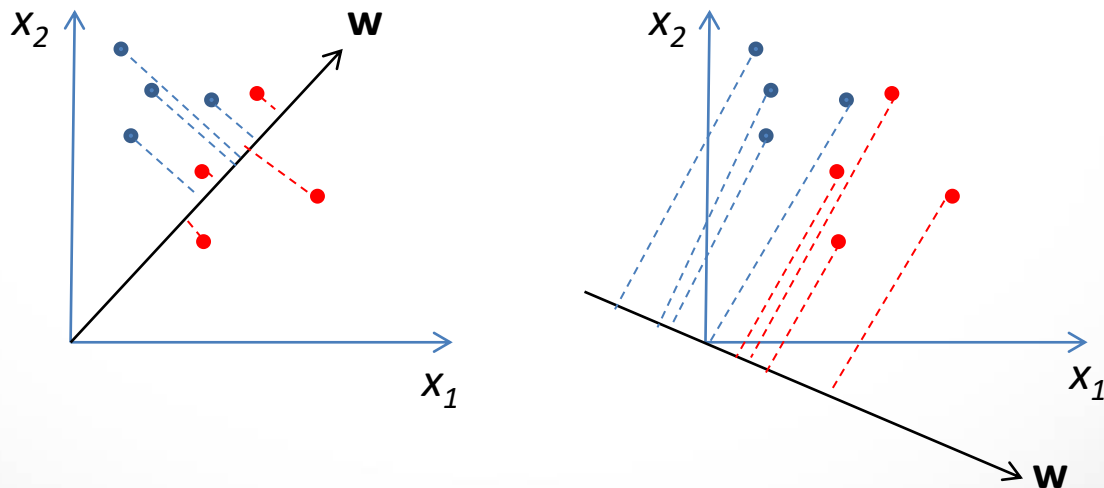
$$\mathbf{Y} = \mathbf{W}^T \mathbf{X}$$

使得 $\mathbf{Y}$ 更加可分类。

第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ Fisher判别分析(LDA)



不同方向的投影，投影后的数据点可分能力不同；对于给定的一组特征集，我们是有可能找到能够最大限度的区分各类数据点的投影方向。



第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ Fisher判别分析(LDA)

- ✓ 定义高维特征集 $X=\{x_1, x_2, \dots, x_n\}$ ，其中 n_1 个样本属于第一类 ω_1 ，剩余的 n_2 个样本属于第二类 ω_2 。那么存在一个投影（或变换 W ），即特征集 X 在低维子空间的表达形式为

$$Y = W^T X$$

使得 Y 更加可分类。

那么该如何建模，进而求出我们期望的 W 呢？

✓ LDA建模过程：

思想：我们希望经过 W 映射后，两类数据的中心间的距离越远越好（异类分离），即类间散度 J_1 ；同时，相同类的样本越往中心靠拢越好（同类紧凑），即类内散度 J_2 。

Fisher准则：满足 $\frac{J_1}{J_2}$ 最大化！



第八章：分类器

$$Y = W^T X$$

□ 子空间学习 (Subspace Learning)

➤ Fisher判别分析(LDA)

✓ LDA建模过程:

思想：我们希望经过 W 映射后，两类数据的中心间的距离越远越好（异类分离），即类间散度 J_1 ；同时，相同类的样本越往中心靠拢越好（同类紧凑），即类内散度 J_2 。

Fisher准则：满足 $\frac{J_1}{J_2}$ 最大化！

1. 投影后，类间散度 J_1 的构造

$$\text{第1类均值（中心）： } \mu_1 = \frac{1}{n_1} \sum_{y \in \omega_1} y = \frac{1}{n_1} \sum_{x \in \omega_1} W^T x = W^T \frac{1}{n_1} \sum_{x \in \omega_1} x = W^T m_1$$

$$\text{第2类均值（中心）： } \mu_2 = \frac{1}{n_2} \sum_{y \in \omega_2} y = \frac{1}{n_2} \sum_{x \in \omega_2} W^T x = W^T \frac{1}{n_2} \sum_{x \in \omega_2} x = W^T m_2$$

$$\text{中心间的距离： } J_1 = \|\mu_1 - \mu\|^2 + \|\mu_2 - \mu\|^2 = \|W^T m_1 - W^T m\|^2 + \|W^T m_2 - W^T m\|^2$$



第八章：分类器

$$\mathbf{Y} = \mathbf{W}^T \mathbf{X}$$

□ 子空间学习 (Subspace Learning)

➤ Fisher判别分析(LDA)

✓ LDA建模过程:

2. 投影后，类内散度J2的构造

$$\text{第1类均值 (中心): } \mu_1 = \frac{1}{n_1} \sum_{\mathbf{y} \in \omega_1} \mathbf{y} = \frac{1}{n_1} \sum_{\mathbf{x} \in \omega_1} \mathbf{W}^T \mathbf{x} = \mathbf{W}^T \frac{1}{n_1} \sum_{\mathbf{x} \in \omega_1} \mathbf{x} = \mathbf{W}^T \mathbf{m}_1$$

$$\text{第2类均值 (中心): } \mu_2 = \frac{1}{n_2} \sum_{\mathbf{y} \in \omega_2} \mathbf{y} = \frac{1}{n_2} \sum_{\mathbf{x} \in \omega_2} \mathbf{W}^T \mathbf{x} = \mathbf{W}^T \frac{1}{n_2} \sum_{\mathbf{x} \in \omega_2} \mathbf{x} = \mathbf{W}^T \mathbf{m}_2$$

$$\text{第1类类样本点与其中心间的距离: } J2_1 = \sum_{\mathbf{y} \in \omega_1} \|\mathbf{y} - \mu_1\|^2 = \sum_{\mathbf{x} \in \omega_1} \|\mathbf{W}^T \mathbf{x} - \mathbf{W}^T \mathbf{m}_1\|^2$$

$$\text{第2类类样本点与其中心间的距离: } J2_2 = \sum_{\mathbf{y} \in \omega_2} \|\mathbf{y} - \mu_2\|^2 = \sum_{\mathbf{x} \in \omega_2} \|\mathbf{W}^T \mathbf{x} - \mathbf{W}^T \mathbf{m}_2\|^2$$

$$\text{因此, } J2 = J2_1 + J2_2 = \sum_{\mathbf{x} \in \omega_1} \|\mathbf{W}^T \mathbf{x} - \mathbf{W}^T \mathbf{m}_1\|^2 + \sum_{\mathbf{x} \in \omega_2} \|\mathbf{W}^T \mathbf{x} - \mathbf{W}^T \mathbf{m}_2\|^2$$



第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ Fisher判别分析(LDA)

✓ LDA建模过程:

3. 利用Fisher准则（即满足 $\frac{J1}{J2}$ 最大化），构造LDA模型

$$\max_W \frac{J1}{J2} = \max_W \frac{\|W^T m_1 - W^T m\|^2 + \|W^T m_2 - W^T m\|^2}{\sum_{x \in \omega_1} \|W^T x - W^T m_1\|^2 + \sum_{x \in \omega_2} \|W^T x - W^T m_2\|^2}$$

模型化简:

$$\max_W \frac{\text{Tr} \left(W^T \left((m_1 - m)(m_1 - m)^T + (m_2 - m)(m_2 - m)^T \right) W \right)}{\text{Tr} \left(W^T \left(\sum_{x \in \omega_1} (x - m_1)(x - m_1)^T + \sum_{x \in \omega_2} (x - m_2)(x - m_2)^T \right) W \right)}$$



$$\max_W \frac{\text{Tr}(W^T S_B W)}{\text{Tr}(W^T S_W W)}$$

特征值分解法求该优化问题

其中, $S_B = (m_1 - m)(m_1 - m)^T + (m_2 - m)(m_2 - m)^T$ 为类间散度矩阵 (between-class scatter);
 $S_W = \sum_{x \in \omega_1} (x - m_1)(x - m_1)^T + \sum_{x \in \omega_2} (x - m_2)(x - m_2)^T$ 为类内散度矩阵 (within-class scatter)



第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ 多重判别分析(MDA)

MDA实质上是LDA的多类推广：

A. 对于多类问题(假设C个类)，J2（类内散度）的构造可以由LDA直接推广，即

$$\begin{aligned} J2 &= J2_1 + J2_2 + \cdots + J2_c \\ &= \sum_{\mathbf{x} \in \omega_1} \|\mathbf{W}^T \mathbf{x} - \mathbf{W}^T \mathbf{m}_1\|^2 + \cdots + \sum_{\mathbf{x} \in \omega_c} \|\mathbf{W}^T \mathbf{x} - \mathbf{W}^T \mathbf{m}_c\|^2 \\ &= \sum_{c=1}^C \sum_{\mathbf{x} \in \omega_c} \|\mathbf{W}^T \mathbf{x} - \mathbf{W}^T \mathbf{m}_c\|^2 = \text{Tr}(\mathbf{W}^T \mathbf{S}_W \mathbf{W}) \end{aligned}$$

其中， $\mathbf{S}_W = \sum_{c=1}^C \sum_{\mathbf{x} \in \omega_c} (\mathbf{x} - \mathbf{m}_c)(\mathbf{x} - \mathbf{m}_c)^T$ 为类内散度矩阵（within-class scatter）



第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ 多重判别分析(MDA)

MDA实质上是LDA的多类推广：

B. 对于多类问题(假设C个类)，J1（类间散度）的构造：

第1类均值（中心）： $\mu_1 = \frac{1}{n_1} \sum_{y \in \omega_1} y = \frac{1}{n_1} \sum_{x \in \omega_1} W^T x = W^T \frac{1}{n_1} \sum_{x \in \omega_1} x = W^T m_1$

第2类均值（中心）： $\mu_2 = \frac{1}{n_2} \sum_{y \in \omega_2} y = \frac{1}{n_2} \sum_{x \in \omega_2} W^T x = W^T \frac{1}{n_2} \sum_{x \in \omega_2} x = W^T m_2$

.

.

第C类均值（中心）： $\mu_C = \frac{1}{n_C} \sum_{y \in \omega_C} y = \frac{1}{n_C} \sum_{x \in \omega_C} W^T x = W^T \frac{1}{n_C} \sum_{x \in \omega_C} x = W^T m_C$

总体均值 $\mu = \frac{1}{N} \sum y = \frac{1}{N} \sum W^T x = W^T \frac{1}{N} \sum x = W^T m$



第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ 多重判别分析(MDA)

MDA实质上是LDA的多类推广：

B. 对于多类问题(假设C个类)，J1（类间散度）的构造：

$$J1 = \sum_{c=1}^C n_c \|\mu_c - \mu\|^2 = \sum_{c=1}^C n_c \|\mathbf{W}^T \mathbf{m}_c - \mathbf{W}^T \mathbf{m}\|^2 = \text{Tr}(\mathbf{W}^T \mathbf{S}_B \mathbf{W})$$

其中， $\mathbf{S}_B = \sum_{c=1}^C n_c (\mathbf{m}_c - \mathbf{m})(\mathbf{m}_c - \mathbf{m})^T$

利用Fisher 准则，MDA模型如下：

$$\max_W \frac{\text{Tr}(\mathbf{W}^T \mathbf{S}_B \mathbf{W})}{\text{Tr}(\mathbf{W}^T \mathbf{S}_W \mathbf{W})}$$

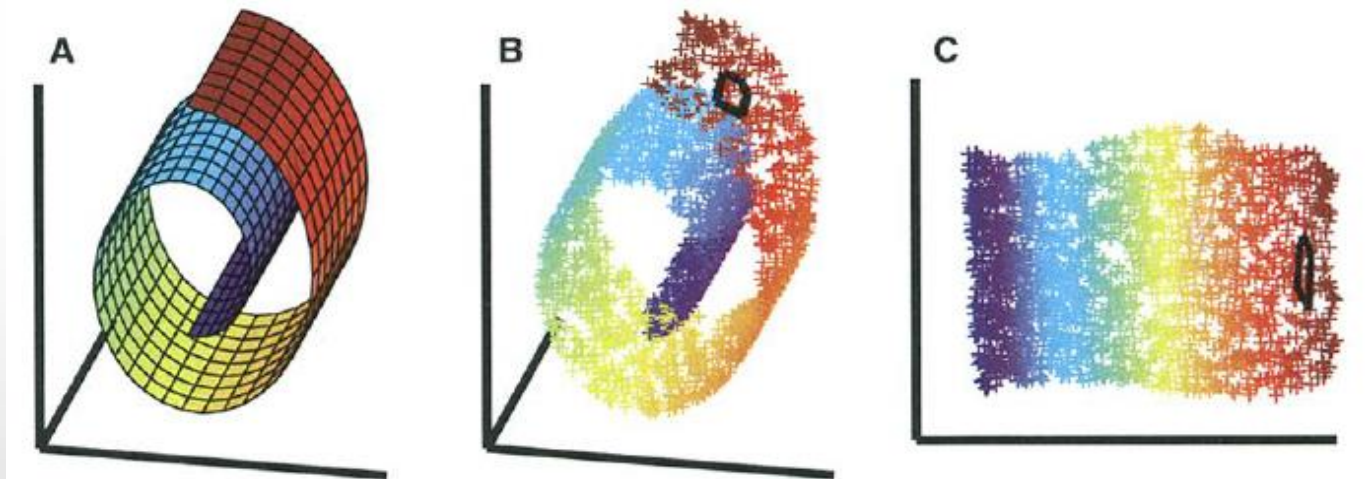
与LDA相同，特征值分解法求该优化问题

第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ 局部保持投影(LPP)

LPP是一种流形思想下的子空间降维方法，前提假设了高维特征数据 $\mathbf{X}$ ，实际上是一种低维的流形结构 $\mathbf{Y}$ 嵌入在高维空间。流形学习的目的映射到低维中，揭示高维数据的本质（局部特性）。





第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ 局部保持投影(LPP)

LPP是一种流形思想下的子空间降维方法，前提假设了高维特征数据 $\mathbf{x}$ ，实际上是一种低维的流形结构 $\mathbf{Y}$ 嵌入在高维空间。流形学习的目的映射到低维中，揭示高维数据的本质（局部特性）。

局部保持特性是指，在高维空间相近的 k 个样本点簇，经过 $\mathbf{W}$ 投影以后，这个 k 个样本点簇依然保持相近！

数学模型为

$$\min_{\mathbf{W}} \sum_i \sum_{j, \forall \mathbf{x}_j \in N_k(\mathbf{x}_i)} A_{i,j} \|\mathbf{y}_i - \mathbf{y}_j\|^2$$

$$\text{其中, } A_{i,j} = \begin{cases} 1, & \text{if } \mathbf{x}_j \in N_k(\mathbf{x}_i) \text{ || } \mathbf{x}_i \in N_k(\mathbf{x}_j) \\ 0, & \text{otherwise} \end{cases}$$



第八章：分类器

□ 子空间学习 (Subspace Learning)

➤ 局部保持投影(LPP)

LPP是一种流形思想下的子空间降维方法，前提假设了高维特征数据 $\mathbf{x}$ ，实际上是一种低维的流形结构 $\mathbf{Y}$ 嵌入在高维空间。流形学习的目的映射到低维中，揭示高维数据的本质（局部特性）。

局部保持特性是指，在高维空间相近的 k 个样本点簇，经过 $\mathbf{W}$ 投影以后，这个 k 个样本点簇依然保持相近！

数学模型为

$$\begin{aligned} \min_W \sum_i \sum_{j, \forall \mathbf{x}_j \in N_k(\mathbf{x}_i)} A_{i,j} \|\mathbf{y}_i - \mathbf{y}_j\|^2 &= \min_W \sum_i \sum_{j, \forall \mathbf{x}_j \in N_k(\mathbf{x}_i)} A_{i,j} \|\mathbf{W}^T \mathbf{x}_i - \mathbf{W}^T \mathbf{x}_j\|^2 \\ &= \min_W \text{Tr}(\mathbf{W}^T \mathbf{X} \mathbf{L} \mathbf{X}^T \mathbf{W}) \end{aligned}$$

其中， $\mathbf{L}$ 是拉普拉斯矩阵， $\mathbf{L} = \mathbf{D} - \mathbf{A}$ ， $\mathbf{D}$ 是一个对角阵， $D_{ii} = \sum_i A_{i,:}$



第八章：分类器

□ 子空间学习与分类器

前面介绍了几种不同的子空间学习方法，其目的是在低维空间内实现分类任务，从而降低计算复杂度，提升分类效率和准确率。

如何实现分类？ 结合本章的前两节，讲述的KNN和SVM方法，可以实现分类，其实现方法基本形式为：

A. PCA+KNN & PCA+SVM

B. LDA+KNN & LDA+SVM

C. MDA+KNN & MDA+SVM

D. LPP+KNN & LPP+SVM

由于子空间模型的灵活性，可提出不同的子空间改进方案，实现高效的分类/识别任务。



第九章： 集群智能仿生

第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

- 群智能作为一种新兴的演化计算技术，已成为研究焦点，它与生命进化、昆虫集体行为、鸟群集体行为，有着极为特殊的关系。
- 所谓群体智能，即**本无智能**的个体，通过**合作**表现出智能行为的特性，在没有集中控制，而且不提供全局模型的前提下，为寻找最优方案或最优解提供了基础。
- 人们把群居昆虫或鸟类的集体行为称作“群智能”或“群体智能”或“集群智能”。
- 主要特点在于：个体的行为很简单，但当它们一起协同工作时，却能够突现出复杂的智能的行为特征。



第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

✓ 优点

灵活性：群体可以适应灵活变化的特性

稳健性：某个体失败，不影响整个群体的行为

自组织行为：不受外部因素的控制

✓ 典型算法

遗传算法（起源于生物进化过程，演化算法）：人类对大自然生物进化过程的计算模拟而抽象出的进化算法；

粒子群优化（启发于鸟群觅食）：基于社会系统中群体智能而产生的启发式仿生算法；

蚁群优化（启发于蚂蚁群体活动）

鱼群、蜂群算法、免疫算法、混沌搜索算法等



第九章：集群智能仿生

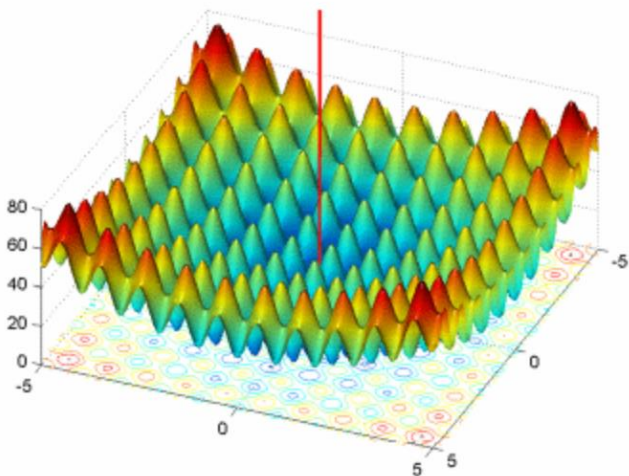
□ 应用

函数优化、神经网络学习

例：以Ras函数（Rastrigin's Function）为目标函数，求其在 $x_1, x_2 \in [-5, 5]$ 上的最小值。

$$Ras(x) = 20 + x_1^2 + x_2^2 - 10(\cos 2\pi x_1 + \cos 2\pi x_2).$$

Global minimum at [0 0]



从优化角度，该函数有太多局部解，很难利用极值理论去求解。

智能仿生算法！

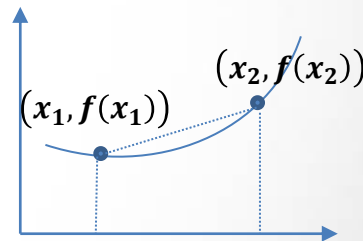
第九章：集群智能仿生

□ 基础概念：凸优化/非凸优化问题

凸优化的局部最优解即是全局最优解，容易求解。非凸优化难以求解。

凸函数定义：定义在某个向量空间的凸集 C 的实值函数 f ，在其定义域 C 内的任意两点 x_1, x_2 ， $0 \leq \lambda \leq 1$ 有：

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$



凸集的定义：集合中的任意两点间 $x_1, x_2 \in C$ ，有 $\lambda x_1 + (1 - \lambda)x_2 \in C$ ，也就是任意两点间的线段都在 C 内，则 C 是凸集。

凸优化问题判定：是解决一类**凸函数**的**凸集**优化问题。





第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

➤ 遗传算法（GA, 起源于生物进化过程，演化算法）：人类对大自然生物进化过程的计算模拟而抽象出的进化算法。

基本思想：每一个物质个体的基本特征会被后代（子代）继承，但后代又会产生一些不同于父代的新特征，在进化过程中，最适应环境的个体特征会被保留（适者生存）。

个体表现形式：个体特征以基因的形式包含在染色体内，通过基因突变或基因杂交，可以产生更适合环境的后代（即更好的解）。

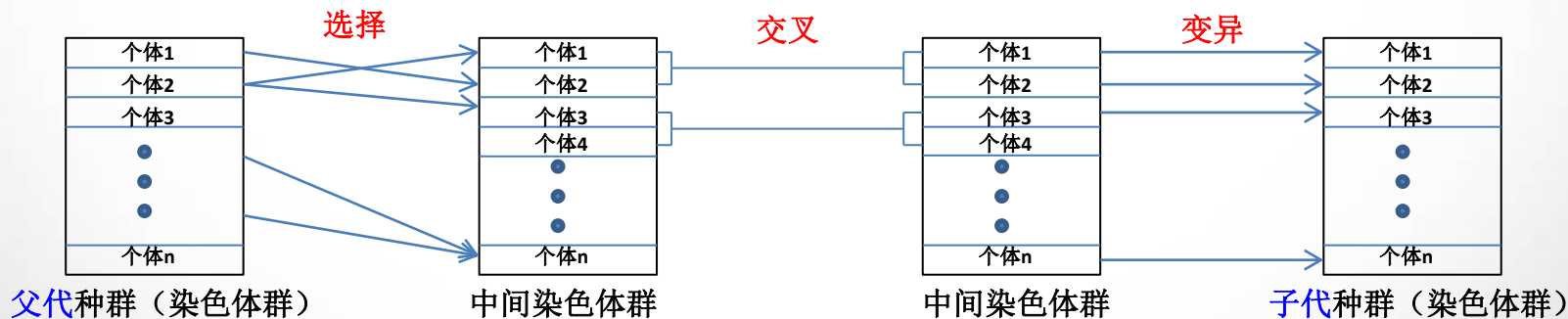
实现：基于种群的演化，这里种群是指一群染色体。通过设定某“**适应度函数**”，从中选择最适应环境的染色体，并进行**复制、交叉、变异**，产生新一代染色体种群，通过一代代进化，直到染色体不再发生变化，即可找到最优的染色体（最优解）。

第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

➤ 遗传算法（GA）

遗传算法主要由三个算子组成，即**选择算子**、**交叉算子**和**变异算子**，用于产生新的一群染色体。



遗传算法的一次进化过程

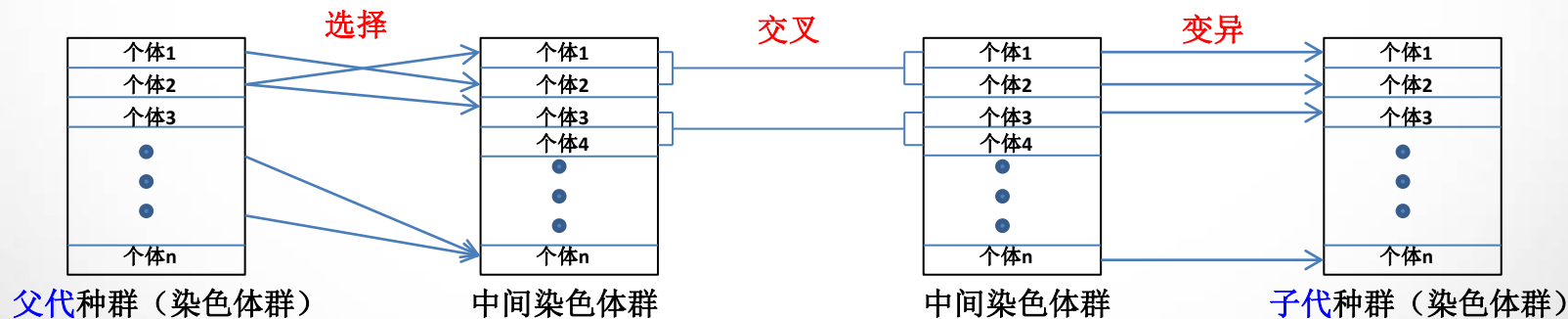
遗传算法如何多次进化呢？ 不断反馈、迭代！

第九章：集群智能仿生

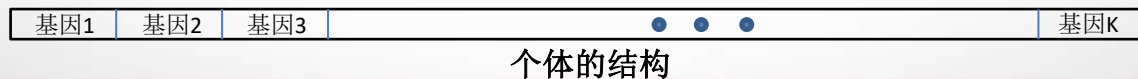
□ 群体智能（Swarm Intelligence）

➤ 遗传算法（GA）

遗传算法主要由三个算子组成，即选择算子、交叉算子和变异算子，用于产生新的一群染色体。



遗传算法的一次进化过程





第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

➤ 遗传算法（GA）

遗传算法主要由三个算子组成，即选择算子、交叉算子和变异算子，用于产生新的一群染色体。

遗传算法的解搜索过程：

1. **编码**：在演化前，先对问题的解空间表示成基因串结构数据（个体的编码）；
2. **生成初始种群**：随机产生 n 个基因串结构数据，每个基因串为一个个体； n 个个体组成一个群体，作为初始迭代点；（注：每个个体代表一个可行解）
3. **适应度评价**：根据适应度函数，计算每个个体的适应度，评估个体的优劣，即解的优劣；根据具体问题，定义能够客观反映染色体优劣的目标函数；
4. **选择**：从当前群体中选择优良的（适应度高的）个体，使它们有机会被选中，并进入到下一次迭代，舍弃适应度低的个体（**适者生存之原则**）；



第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

➤ 遗传算法（GA）

遗传算法主要由三个算子组成，即选择算子、交叉算子和变异算子，用于产生新的一群染色体。

遗传算法的解搜索过程：

5. **交叉**：遗传操作，信息的交换，交叉方法以某交叉概率 P_c 通过第 i 个个体和第 $i+1$ 个个体之间进行交叉，其中交叉算子为

$$\begin{cases} X_i^{t+1} = c_i \cdot X_i^t + (1 - c_i) \cdot X_{i+1}^t \\ X_{i+1}^{t+1} = (1 - c_i) \cdot X_i^t + c_i \cdot X_{i+1}^t \end{cases}$$

6. **变异**：遗传操作，以变异概率 P_m 进行基因变异，变异算子为

$$X_i^{t+1} = X_i^t + c_i$$

交叉和变异算子不唯一



第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

➤ 遗传算法（GA）

控制参数的选择：

（1）基因串长度 L （个体的长度）

个体的长度取决于特定问题，即需要的可行解的长度（精度）；

（2）群体规模 n

群体规模越大，找到全局最优解的可能性也越大，改善搜索效果。但增加了个体评价的计算，收敛速度较慢；

（3）交叉概率 P_c

交叉概率控制着交叉算子使用的频率，交叉概率越高，群体中个体更新速度越快，但会造成优良基因的丢失。但若交叉概率越小，可能导致搜索陷入局部最优解，从而早熟（收敛停滞）

（4）变异概率 P_m

变异是针对某个基因而发生的。如果变异概率较大，则基因变异的数量也较高，会导致随机搜索现象；若变异概率较小，会造成基因丢失而无法恢复。

第九章：集群智能仿生



□ 群体智能 (Swarm Intelligence)

➤ 粒子群算法(PSO)

最早由Eberhart和Kennedy于1995年提出，通过模拟鸟群的觅食行为，建立的一种全局优化方法，利用粒子间的合作与竞争，实现最优解的搜索。

基本思想：每个粒子会根据自身经验的积累和群体知识的积累，根据当前最佳的个体，更新每个粒子。搜索效率较高，不存在交叉、变异操作。

个体表现形式：每一只鸟被视为一个粒子，粒子的表现形式只有两个变量：速度和位置，即鸟飞行的速度和位置。

实现：基于种群的演化，这里种群是指一群鸟(粒子)。从随机的粒子群体出发，通过设定某“适应度函数”来评价每个粒子（解或个体）的优劣，通过速度和位置的不断更新，直到搜索结束。在搜索过程中，只有当前最佳的个体能提供信息给其他粒子（这与遗传算法不同）。

第九章：集群智能仿生



□ 群体智能 (Swarm Intelligence)

➤ 粒子群算法(PSO)

PSO算法描述了大量的粒子通过模拟鸟群搜寻食物的运动方式，相互协作，完成最优解的搜索过程。每个粒子以一定的速度飞行，在迭代一定次数后，找到全局最优的位置，那么这个“位置”即为全局最优解。

定义一个**D**维的搜索空间（即解的长度为**D**），粒子的数量为**N**(种群大小)，第**i**个粒子的位置表达为 $\mathbf{X}_i = (x_{i1}, x_{i2}, \dots, x_{iD})^T$

第**i**个粒子的速度表达为 $\mathbf{V}_i = (v_{i1}, v_{i2}, \dots, v_{iD})^T$

经过搜索后，第**i**个粒子的最佳位置为 $\mathbf{P}_i = (p_{i1}, p_{i2}, \dots, p_{iD})$

整个粒子群的最佳位置为 $\mathbf{P}_g = (p_{g1}, p_{g2}, \dots, p_{gD})$

第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

➤ 粒子群算法(PSO)

粒子速度的直接更新（不需要交叉变异），因此收敛更快：

$$v_{id}(t+1) = w \cdot v_{id}(t) + c_1 \cdot r_{1i}(t) \cdot [p_{id}(t) - x_{id}(t)] + c_2 \cdot r_{2i}(t) \cdot [p_{gd}(t) - x_{id}(t)]$$

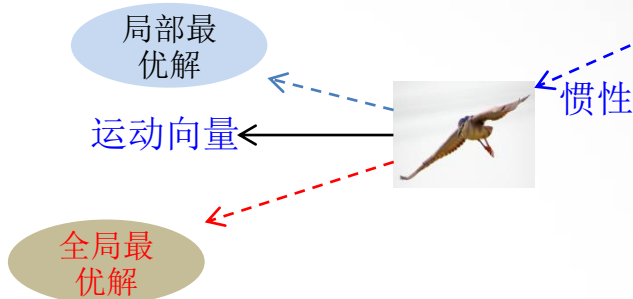
更新方程中涉及三个部分：

第一部分为粒子先前的速度（惯性）；

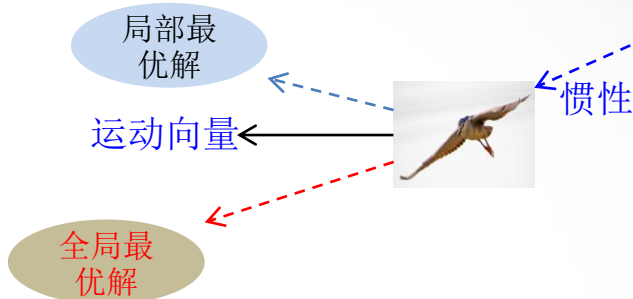
第二部分为“自我认知”部分，即第*i*个粒子**自身的思考**，可理解为第*i*个粒子当前位置与自己最好位置之间的距离；

第三部分为“社会经验”部分，表示粒子间的**信息共享与协作**，可理解为第*i*个粒子当前位置与群体中最好位置之间的距离；

位置更新： $x_{id}(t+1) = x_{id}(t) + v_{id}(t+1)$, $1 \leq i \leq N, 1 \leq d \leq D$



第九章：集群智能仿生



□ 群体智能 (Swarm Intelligence)

➤ 粒子群算法(PSO)

粒子速度的更新:

$$v_{id}(t+1) = w \cdot v_{id}(t) + c_1 \cdot r_{1i}(t) \cdot [p_{id}(t) - x_{id}(t)] + c_2 \cdot r_{2i}(t) \cdot [p_{gd}(t) - x_{id}(t)]$$

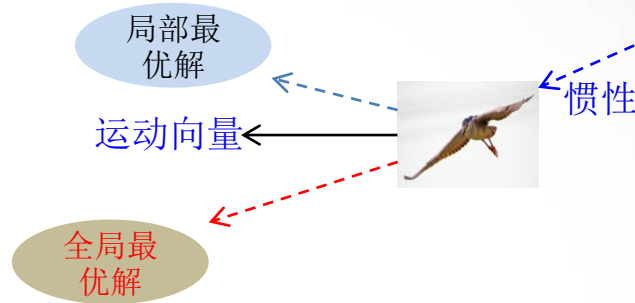
粒子位置更新:

$$x_{id}(t+1) = x_{id}(t) + v_{id}(t+1), 1 \leq i \leq N, 1 \leq d \leq D$$

其中，**c1**和**c2**是正的加速常数，**w**为惯性权重因子，**r1,i**和**r2,i**为区间[0,1]内的随机数

➤ 在实际应用中，可以设计**w**的不同表达形式，产生不同的启发式算法。

第九章：集群智能仿生



□ 群体智能 (Swarm Intelligence)

➤ 粒子群算法(PSO)

PSO实现过程：

- (1) 初始化粒子群体；规模为 n , 包括位置 $\mathbf{x}$ 和速度 $\mathbf{v}$ 。
- (2) 适应度评价；评价每个粒子的优劣。
- (3) 找每个粒子的最佳位置 $\mathbf{pbest}$ 。对于每个粒子 $\mathbf{x}$ ，其当前适应度值如果高于其个体的历史最佳为 $\mathbf{pbest}$ ，则更新 $\mathbf{pbest}=\mathbf{x}$ ；
- (4) 找全局最佳位置 $\mathbf{gbest}$ ；找出粒子群适应度最高的粒子位置 $\mathbf{x}$ ，如果该粒子适应度比当前全局最佳位置 $\mathbf{gbest}$ 高，那么更新当前全局最佳位置 $\mathbf{gbest}=\mathbf{x}$ ；
- (5) 利用公式更新粒子（即速度 $\mathbf{v}$ 和位置 $\mathbf{x}$ ）
- (6) 检验收敛条件是否满足。

第九章：集群智能仿生

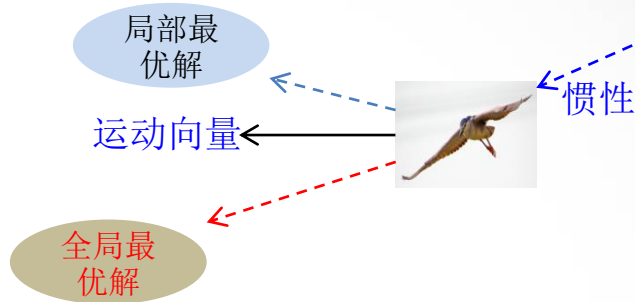
□ 群体智能（Swarm Intelligence）

➤ 参数控制与分析

如果 $w=0$, 则粒子没有记忆性;

如果 $c1=0$, 则粒子没有“认知”，只有社会经验，收敛速度会更快，但易陷入局部最优;

如果 $c2=0$, 则粒子只有“认知”能力，没有“社会”性，则粒子之间没有信息共享和合作，独立搜索，也就没有了群体智能，不可能达到最优解



$$v_{id}(t+1) = w \cdot v_{id}(t) + c_1 \cdot r_{1i}(t) \cdot [p_{id}(t) - x_{id}(t)] + c_2 \cdot r_{2i}(t) \cdot [p_{gd}(t) - x_{id}(t)]$$

粒子间信息共享的部分，仅利用最优的粒子信息



第九章：集群智能仿生

□ 群体智能（Swarm Intelligence）

➤ 蚁群算法(ACO)

蚂蚁是自然界中常见的一种生物，随着现代仿生学的发展，Dorigo等人首次将蚁群系统发展成了有用的优化和控制算法，即蚁群算法。

基本思想：虽然单个蚂蚁的行为极其简单，但群体行为具有自组织性，相互协作的蚂蚁群体能够通过自身释放的“**信息素**”传递信息，很容易找到巢穴到食物源的最短路径，当出现障碍物时，蚂蚁还能够根据对环境的自适应变化，找到最优路径。

实现：选择机制、信息素更新机制、协作机制。信息素越多的路径被选中的概率越大（选择机制）；路径越短，信息素增加越快（信息素更新机制）；个体间通过信息素进行交流（协作机制）。蚂蚁算法已成功应用于解决复杂的组合优化问题，如旅行运营商最优路径搜索问题（TSP问题）。

信息素

蚂蚁之间通信的媒介



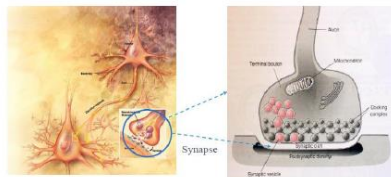
第十章：神经网络与深度学习

第十章：神经网络与深度学习

□ 神经网络发展史

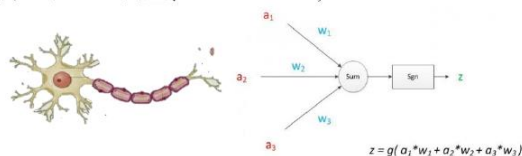
1906

- 神经元学说
- 神经元的连接方式被发现



1943

神经元MP模型, McCulloch, Pitts



Hebb

1949

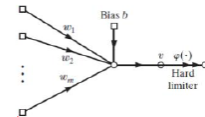
- Hebb学习规则
- $\Delta W_j = k \times f(W_j^T X) X$



1958

- 感知机(Perceptron)模型

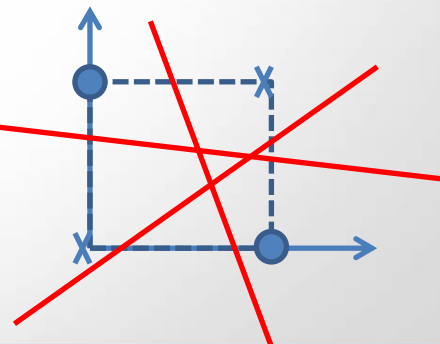
$$w = w + \eta(y - \hat{y})x$$



Marvin Minsky

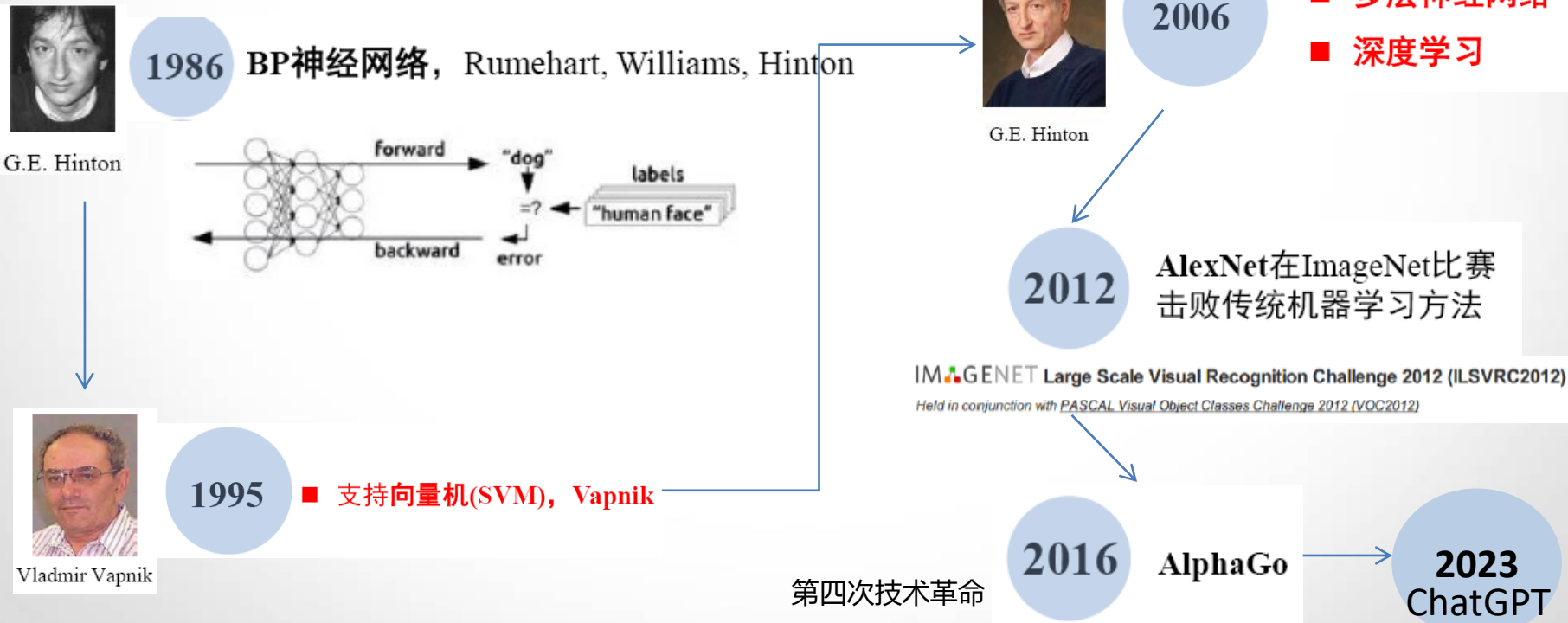
1969

- XOR (异或) 问题
- 计算能力的限制



第十章：神经网络与深度学习

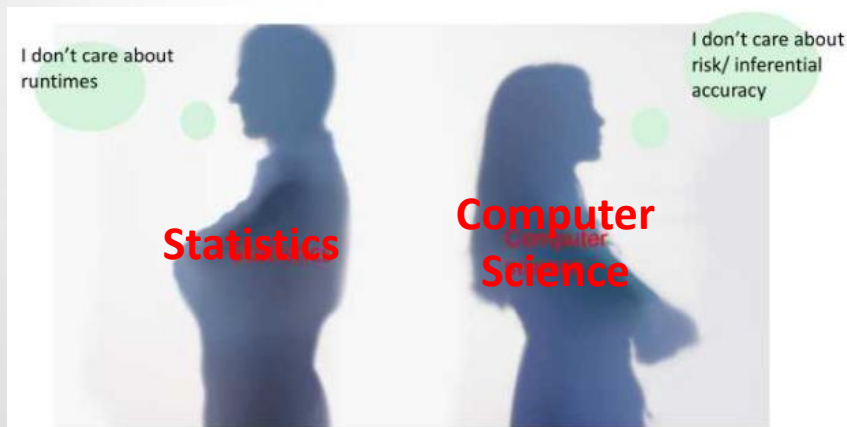
□ 神经网络发展史



第十章：神经网络与深度学习

□ 神经网络发展史

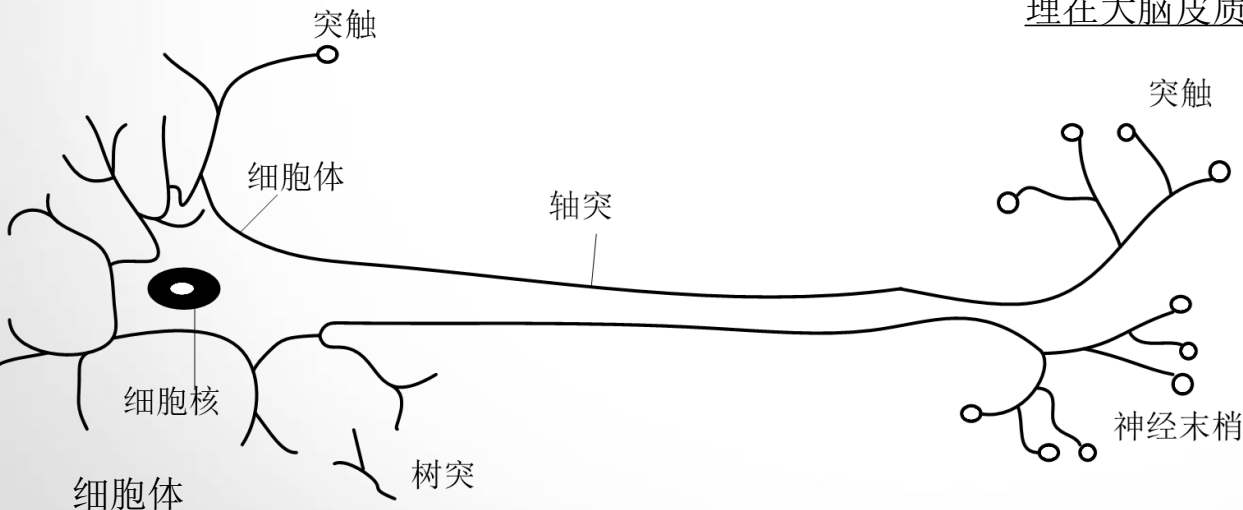
统计学与机器学习的关系



第十章：神经网络与深度学习

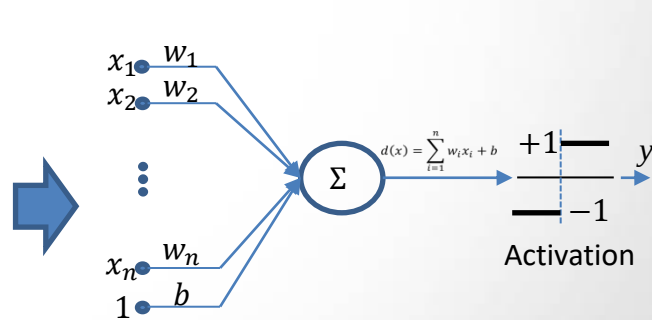
□ 感知器(Frank Rosenblatt, 1958)

生物神经元（神经细胞）结构：



生物神经元结构

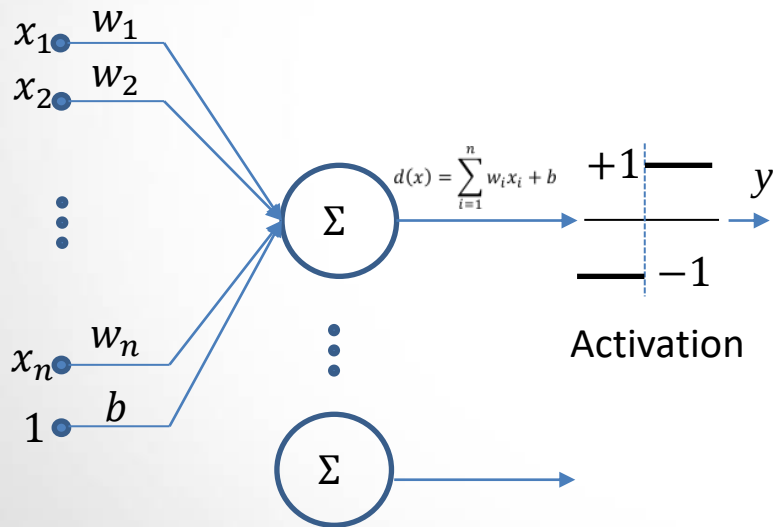
每个神经元由一个细胞体组成，其包含一个细胞核。从细胞体分支扩展出许多为树突的神经纤维和一根长的轴突。一个神经元与10到10万个其他神经元相连，连接处称为突触。信号通过复杂的化学反应从神经元传播到神经元。研究发现，大多数的信息处理在大脑皮质，即大脑外层进行。



人工神经元结构

第十章：神经网络与深度学习

□ 感知器



如何计算权重 w 和 b 呢？

✓ 感知器学习规则

采用类梯度下降方法：

1. 如果第 j 个神经元输出是正确的，则 w_{ij} 和偏差 b_j 不变；
2. 如果第 j 个神经元输出是0，而期望值是1，则 $w_{ij}=w_{ij}+x_i$
 $b_j=b_j+1$ ；
3. 如果第 j 个神经元输出是1，而期望值是0，则 $w_{ij}=w_{ij}-x_i$
 $b_j=b_j-1$ ；



w 和 b 的学习算法：

设第 j 个神经元输出为 y_j ，期望值为 t_j ，则学习规则为

$$w_{ij}=w_{ij}+(t_j-y_j)x_i$$

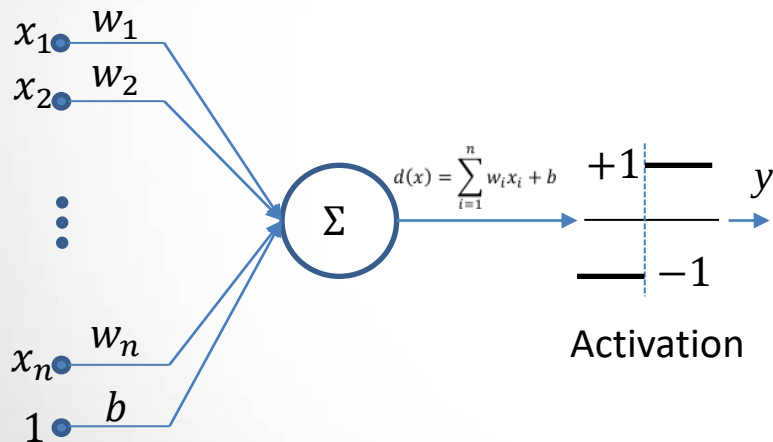
$$b_j=b_j+(t_j-y_j)1;$$

$Y=T$

→ 目标任务

第十章：神经网络与深度学习

□ 感知器



单个神经元的感知器结构

✓ 感知器的局限性

1. 感知器的激活函数为阈值函数，输出只能是两个值，因此只能用于简单的分类问题；
2. 感知器是一种线性分类方法，适用于线性可分问题；
3. 收敛速度较慢。

✓ 例：利用感知器，对4个二维样本进行分类：

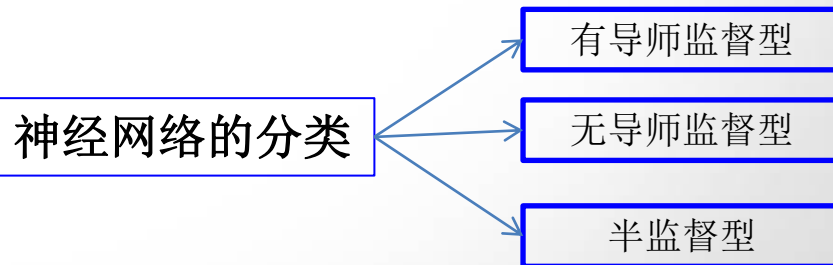
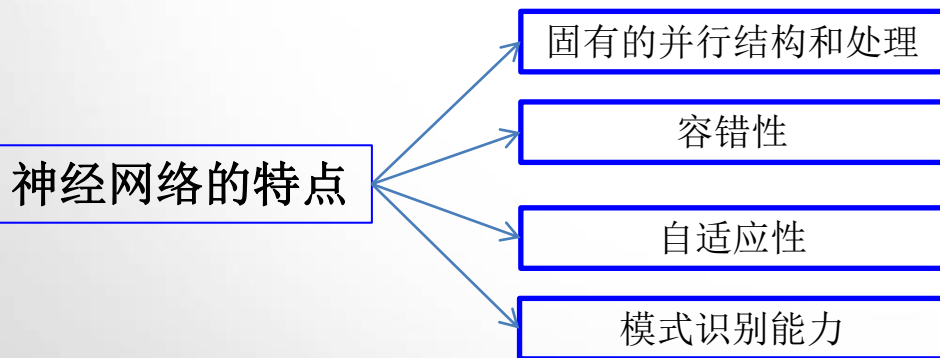
$$X = \begin{bmatrix} -2 & -1 & 1 & 0 \\ -1 & -2 & 1 & 2 \end{bmatrix}, \text{ 目标为 } T = [0 \ 0 \ 1 \ 1];$$

初始权重为 $w = [0, 0]$, $b = 0$

第十章：神经网络与深度学习

□ 人工神经网络（Artificial Neural Network, ANN）

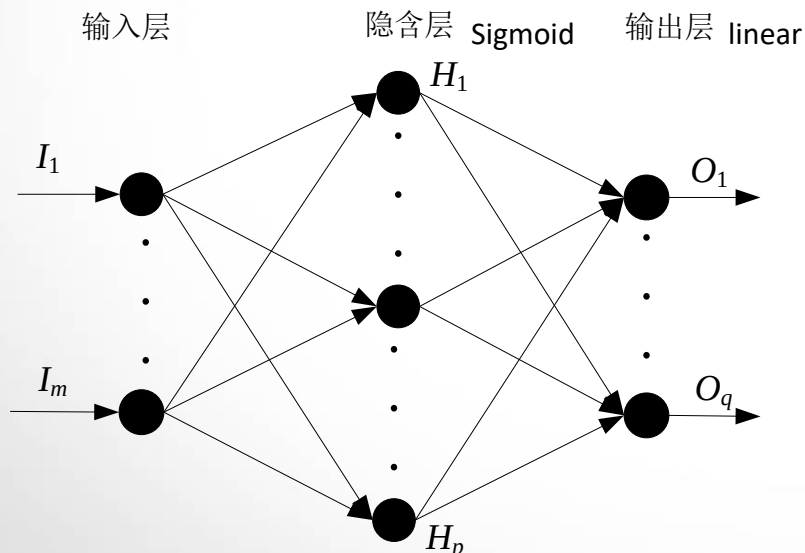
定义：ANN是对人脑或自然神经网络若干基本特性的抽象和模拟，是一种基于连接学说构造的智能仿生模型，是由大量神经元组成的非线性动力系统。神经网络可以看作是输入空间到输出空间的非线性映射，通过调整权重和阈值来“学习”和发现变量间的关系，实现分类或回归任务。



第十章：神经网络与深度学习

□ BP神经网络（Back-propagation Neural Network）

定义：BP网络是一种多层前馈网络，采用最小均方误差为学习函数，梯度下降算法为学习规则，是最广泛使用的网络。



单隐层的BP网络结构

■ 数学理论基础—万能逼近理论



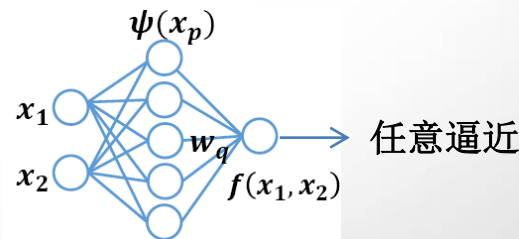
俄国数学家安德雷·柯尔莫哥洛夫, 1956

Kolmogorov定理

$$f(x_1, \dots, x_n) = \sum_{q=1}^{q=2n+1} w_q \left[\sum_{p=1}^n \psi(x_p) \right]$$

st. ψ is a sigmoidal function

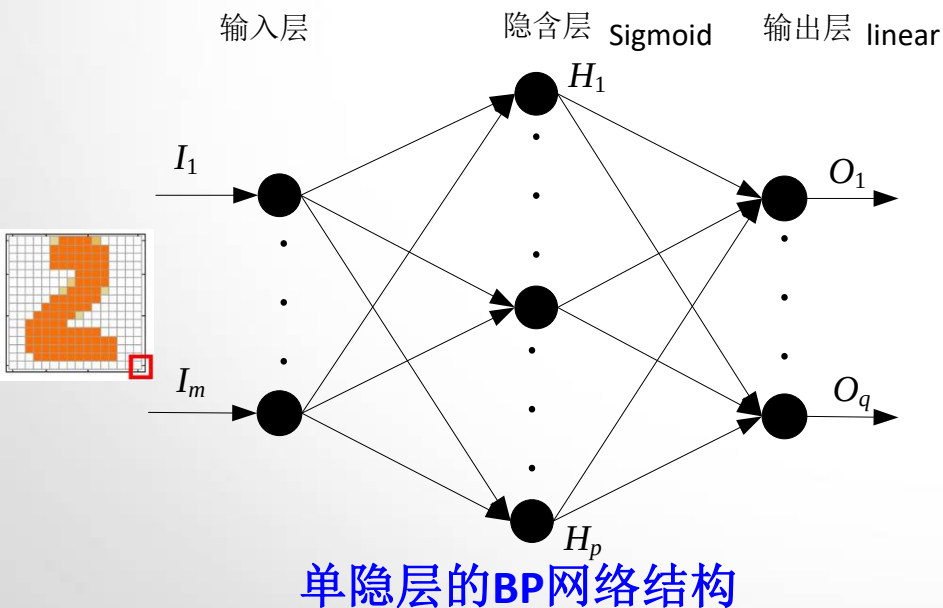
隐层大小为 $2n+1$ ，神经网络以任意精度逼近任意一个 n 元的连续实值函数。



第十章：神经网络与深度学习

□ BP神经网络（Back-propagation Neural Network）

BP训练算法（链式求导法则）：



◆ 输入层到隐含层的权重更新

$$w_{j,i}(t+1) = w_{j,i}(t) + \alpha \delta_j I_i$$

δ_j 为隐含层第 j 个节点的偏差信号；

◆ 隐含层到输出层的权重更新

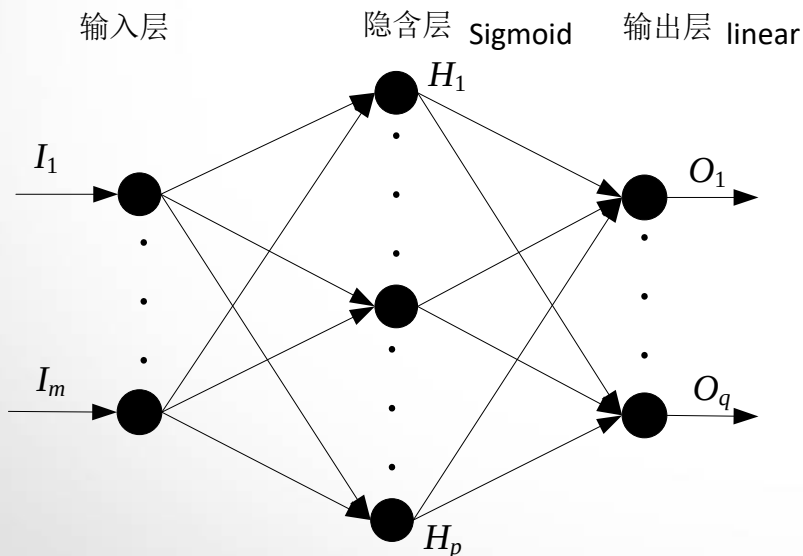
$$w_{k,j}(t+1) = w_{k,j}(t) + \alpha \delta_k H_j$$

H_j 为隐含层第 j 个节点的输出信号；
 δ_k 为输出层第 k 个节点的偏差信号；

第十章：神经网络与深度学习

□ BP神经网络（Back-propagation Neural Network）

BP训练算法：



单隐层的BP网络结构

◆ 隐含层每个节点阈值的更新

$$b_j(t+1) = b_j(t) + \beta \delta_j$$

δ_j 为隐含层第 j 个节点的偏差信号；

$$\delta_j = \sum_k \delta_k w_{k,j} H_j (1 - H_j)$$

◆ 输出层每个节点阈值的更新

$$b_k(t+1) = b_k(t) + \beta \delta_k$$

δ_k 为输出层第 k 个节点的偏差信号；

$$\delta_k = (T_k - O_k) O_k (1 - O_k)$$

第十章：神经网络与深度学习

□ 深度学习（Deep learning）

自2012年开始，深度学习的突破性进展



文本

谷歌NMT翻译
智能问答系统
SyntaxNet
语法分析
...

图像

图像内容分类
图像内容描述
目标追踪检测
图片风格迁移
...

语音

语音内容识别
语音情感分析
音乐自动生成
...

...

...

...

...

第十章：神经网络与深度学习

□ 深度学习（Deep learning）

神经网络是深度学习研究的一部分，典型的网络为**卷积神经网络(CNN)**。

主要应用领域：图像或视频 (局部感受野，端到端学习：End to End Learning)

人类视觉

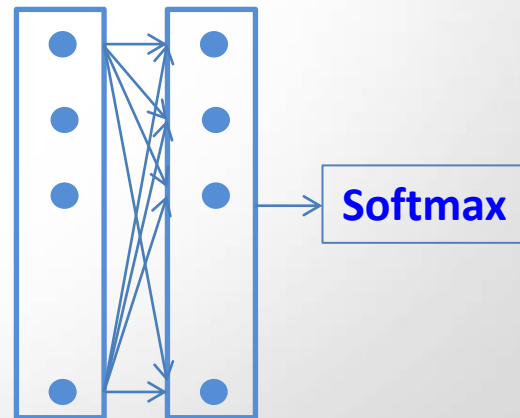
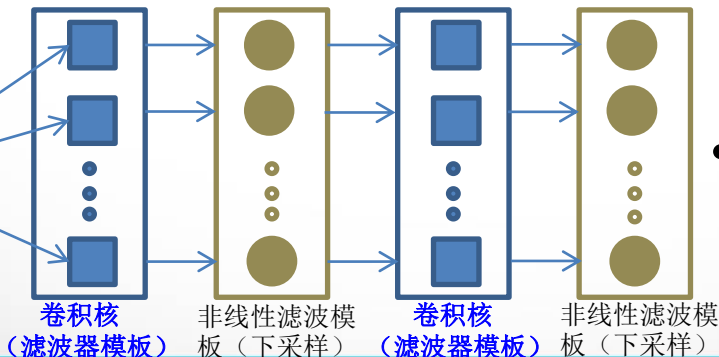


100x100x3



254 255 255 253 249 246 241 241 243 240 239 240 240
255 255 255 251 246 245 242 243 244 240 237 240 241
255 255 255 249 243 243 244 244 240 238 240 241
255 254 253 248 243 243 243 243 242 240 240 239
254 252 251 247 242 242 243 241 243 244 240 240 240
255 254 250 244 240 240 243 243 242 242 241 241 240
253 254 251 245 242 241 241 241 241 241 242 241 241
251 253 252 245 242 242 239 239 241 241 243 243 241
251 251 249 244 241 243 239 238 240 240 241 242 242
250 247 245 243 240 243 240 240 239 237 237 241 242
247 244 244 243 242 243 241 241 240 237 235 239 242
247 244 244 245 243 242 241 241 239 239 237 237 240
250 245 242 243 242 242 241 241 240 240 240 237 238
252 246 242 242 242 242 241 241 240 240 241 238 237
254 248 244 243 241 242 242 241 240 241 243 241 237
254 248 242 242 241 240 240 241 240 240 241 240
253 249 244 242 241 239 239 240 241 241 240 241 241
252 251 247 243 242 241 240 239 240 241 239 240

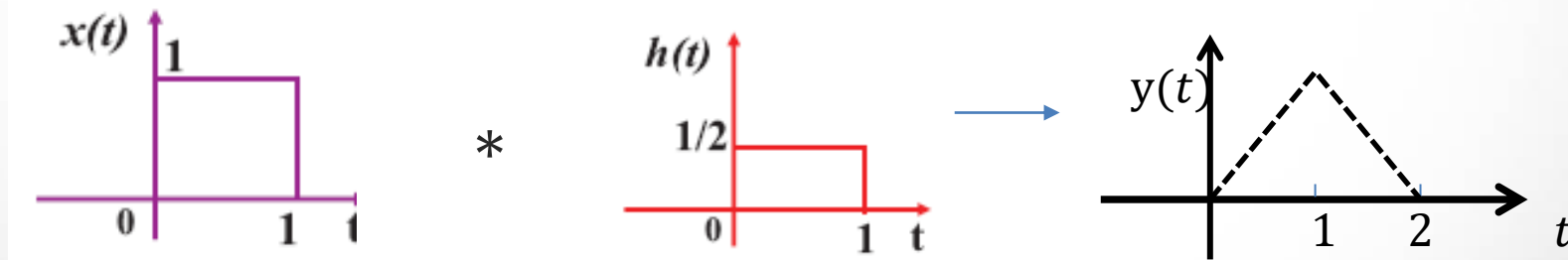
计算机视觉



第十章：神经网络与深度学习

□ 卷积（Convolution）与相关（Correlation）

卷积定义：
$$y(t) = x(t) * h(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau)d\tau$$



相关定义：
$$y(t) = x(t) * h(t) = \int_{-\infty}^{\infty} x(\tau)h(\tau - t)d\tau$$

注：深度学习框架中的“卷积”实际上是“相关”运算，并非真正的卷积。

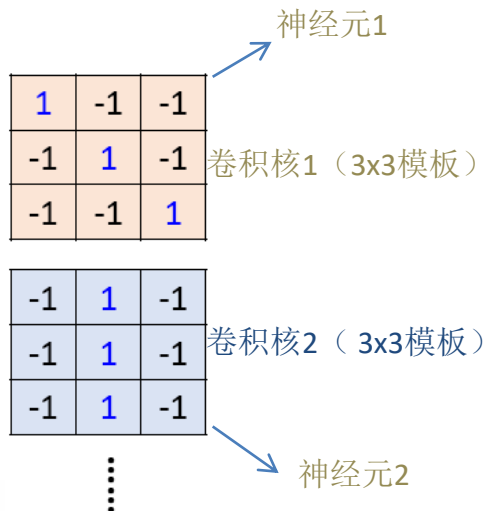
第十章：神经网络与深度学习

深度学习 (Deep learning)

卷积操作:

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

图像



先考虑卷积核1:

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

1	-1	-1
-1	1	-1
-1	-1	1



卷积核在图像上逐步滑动

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

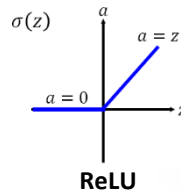
卷积核1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Feature map



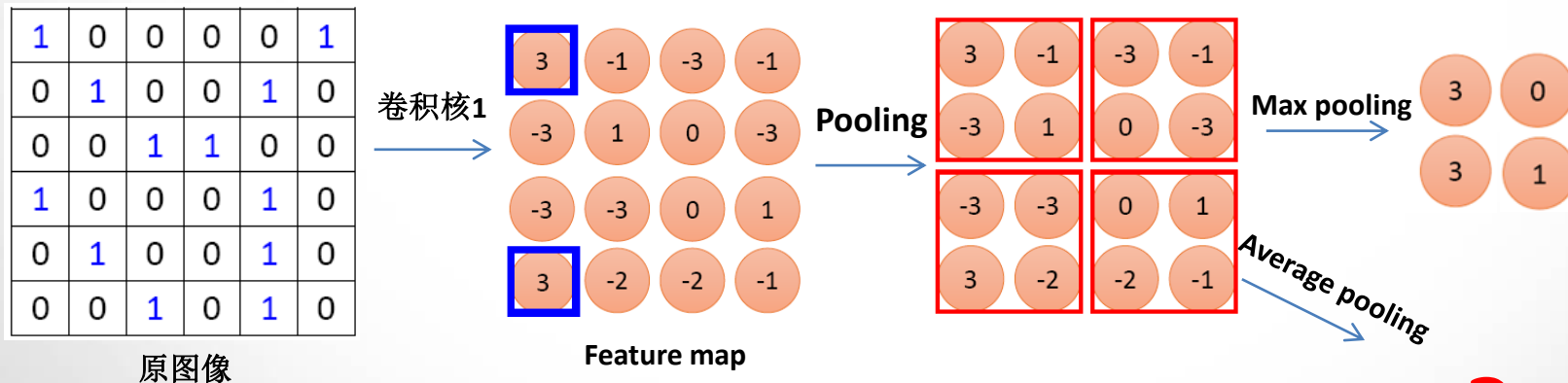
第十章：神经网络与深度学习



深度学习 (Deep learning)

下采样操作(Pooling池化层):

常见方法: **Average pooling**(均值滤波)和**Max pooling**(类似非线性滤波)



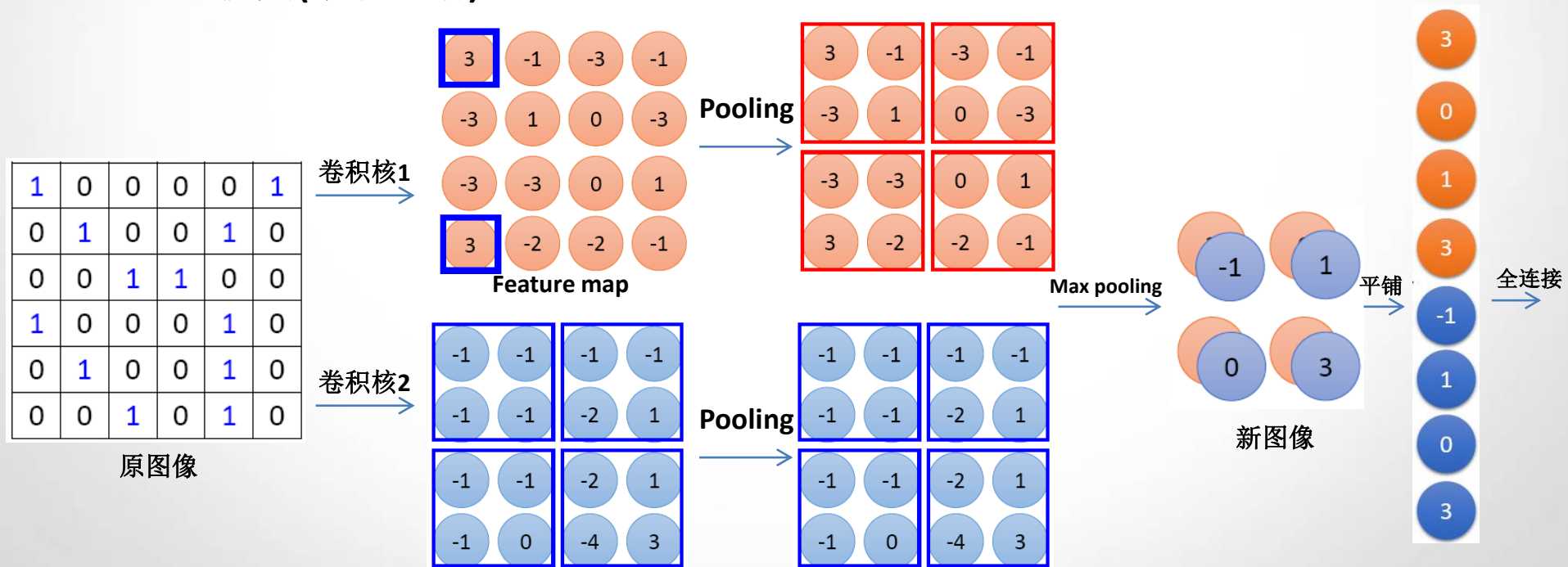
?

注：每一层输出还需要非线性激活函数映射，通常为ReLU: $\max(0, z)$ 激活函数

第十章：神经网络与深度学习

深度学习 (Deep learning)

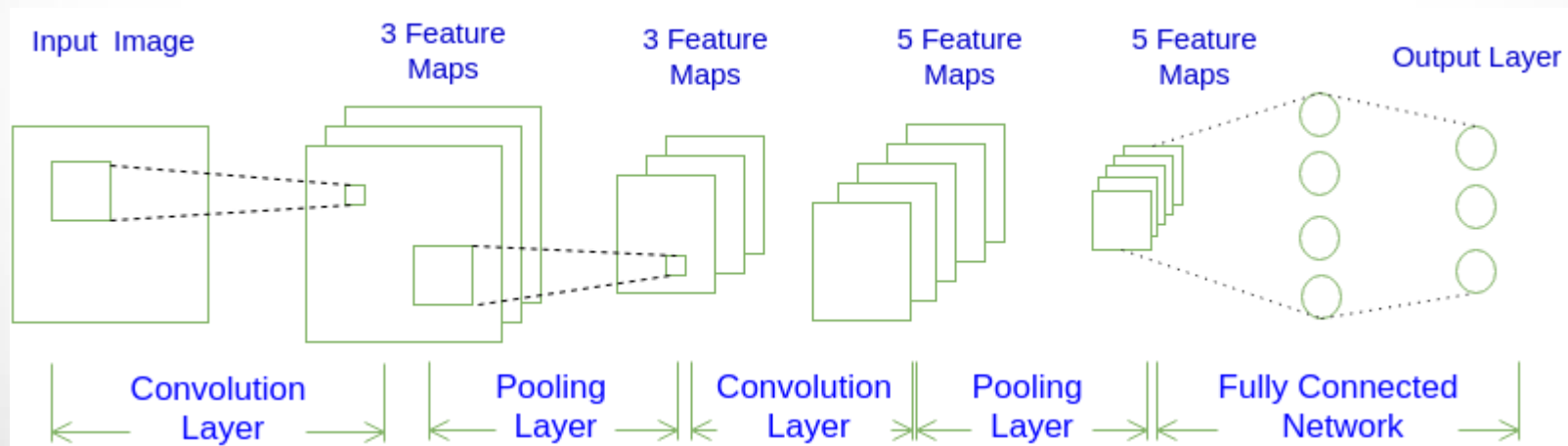
全连接层(串联平铺):



第十章：神经网络与深度学习

□ 深度学习（Deep learning）

➤ 卷积神经网络完整结构：





第十章：神经网络与深度学习

□ 深度学习（Deep learning）

➤ CNN的训练方法

1. 损失函数：

Softmax交叉熵损失： $L = -\sum_{i=1}^m \log P_i = -\sum_{i=1}^m \log \frac{e^{z y_i}}{\sum_{j=1}^K e^{z_j}}$ （交叉熵损失）

2. Mini-batch随机梯度下降

提高计算效率，增强网络泛化能力

3. 误差后向传播BP算法

包括三个部分的反传：卷积层、池化层、输出层

4. 深度学习开发程序

Pytorch, TensorFlow, Caffe, Jittor, MegEngine (Python, GPU)

Logistic Regression?

CNN优点：

局部连接

权重共享

时间/空间
次采样



第十章：神经网络与深度学习

□ 关于交叉熵损失的基础概念回顾

➤ **信息量**：信息消除不确定性的程度。

定义：信息量的大小与信息 x 发生的概率成反比， $I(x) = -\log P(x)$

➤ **信息熵**：信息量的熵，即所有信息量的期望。

定义： $H(X) = -\sum_{i=1}^N P(x_i) \log P(x_i)$

➤ **相对熵**：两个概率分布 $p(x)$ 和 $q(x)$ 之间的差异度量，一般采用KL散度

定义： $D_{KL}(p(X) \parallel q(X)) = \sum_{i=1}^N p(x_i) \log \frac{p(x_i)}{q(x_i)}$ ，KL散度越小，表示两个分布更接近

第十章：神经网络与深度学习

□ 关于交叉熵损失的基础概念回顾

➤ **交叉熵**：相对熵和信息熵的总和。

定义：对KL散度展开， $D_{KL}(p(X) \parallel q(X)) = \sum_{i=1}^N p(x_i) \log \frac{p(x_i)}{q(x_i)} =$
 $\sum_{i=1}^N p(x_i) \log p(x_i) - \sum_{i=1}^N p(x_i) \log q(x_i) = -H(X) + (-\sum_{i=1}^N p(x_i) \log q(x_i))$

其中，**p(x)**为真实分布，**q(x)**为预测分布，**H(X)**可以看作常数。

交叉熵的公式定义为：

$$H(p, q) = - \sum_{i=1}^N p(x_i) \log q(x_i)$$

$$= - \sum_{i=1}^N \sum_{c=1}^C p_c(x_i) \log q_c(x_i) = - \sum_{i=1}^N \log q(x_i)$$



狗	猫	鸟
0	1	0
0.2	0.7	0.1

真实

预测

此处，KL散度和交叉熵等价

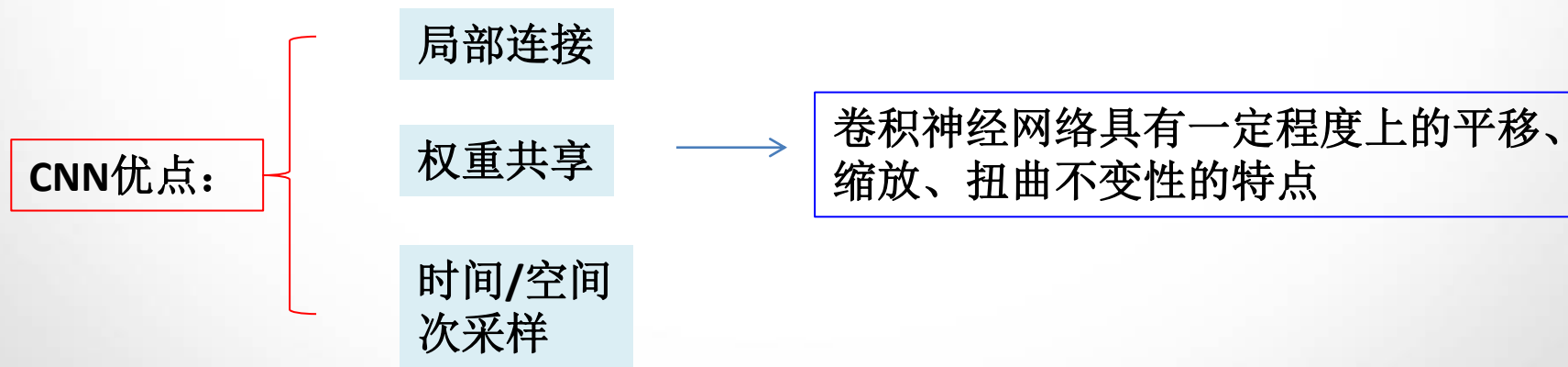


第十章：神经网络与深度学习

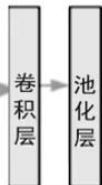
□ 深度学习（Deep learning）

➤ CNN优点

启发于生物上的“感受野”机制，在视觉神经系统中，一个神经元的感受野是指视网膜的特定区域，只有在该区域内的刺激，才能激活该神经元。



输入



output: 56*56*64;

Transition Block

Transition Block

Transition Block

Classification Block

Dense Block 1

Dense Block 2

Dense Block 3

Dense Block 4

预测

“马”

BN
Conv 3*3,32
Relu

cat

BN
Conv 3*3,32
Relu

cat

BN
Conv 3*3,32
Relu

k 2

Bottleneck 3

```
# 设置最后一个Dense Block (最后一个Dense Block不需要Transition Block)
feature_list = [x]
for i in range(nb_layers):
    x = Conv2D(growth_rate, (3, 3), padding="same", activation='relu')(x)
    feature_list.append(x)
    if i < (nb_layers-1):
        x = Concatenate()(feature_list)

# 全局池化
x = GlobalAveragePooling2D()(x)
x = Dense(2, activation='softmax')(x)
output_layer = x
model = Model(input_layer, output_layer)
model.summary()
```

设置 [Dense Block + Transition Block] 多个, 此处以1个为例

```
for j in range(3):
```

```
# 1. Dense Block
```

```
# 用一个列表存放提供特征的层
```

```
feature_list = []
```

```
for i in range(nb_layers):
```

```
    x = BatchNormalization()(x)
```

```
    x = Conv2D(growth_rate, (3, 3), padding="same", activation='relu')(x)
```

```
    feature_list.append(x)
```

```
    if i < (nb_layers-1):
```

```
        x = Concatenate()(feature_list)
```

```
# 2. Transition Block
```

```
x = BatchNormalization()(x)
```

```
x = Conv2D(growth_rate, (1, 1), padding="same", activation='relu')(x)
```

```
x = AveragePooling2D((2, 2), strides=(2, 2))(x)
```

#代码示例: 6-16

```
from keras.optimizers import Adam
```

```
model.compile(loss='categorical_crossentropy', optimizer=Adam(lr=0.001), metrics=['accuracy'])
```

```
model.fit_generator(train_generator, epochs=5, validation_data=validation_generator)
```



第十章：神经网络与深度学习

□ Transformer（变换器）

背景：2017年由谷歌团队首次提出，论文“Attention is All You Need”发表在机器学习顶会NeurIPS，在自然语言处理的多个任务上取得惊艳的效果。其在结构上完全抛弃了CNN和RNN，完全由注意力机制构成。

贡献：大语言模型均基于Transformer构建，如BERT、GPTs、ChatGPT (Chat Generative Pretraining Transformer)

延伸：视觉Transformer（ViT）的提出，开启了计算机视觉从CNN向Transformer的技术转变，成为视觉大模型的核心技术。

大一统：视觉-语言多模态大模型，如Sora、GPT4o

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, et al. “Attention Is All You Need”, NeurIPS, 2017.

第十章：神经网络与深度学习

Transformer（变换器）

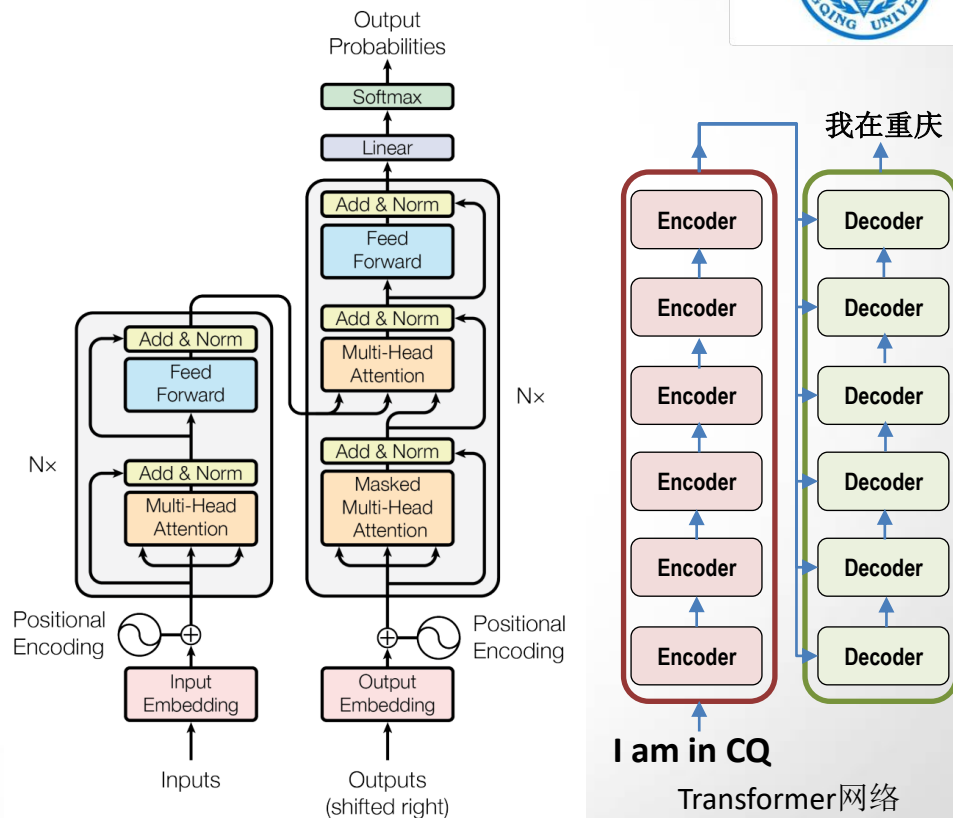
Transformer是一个Encoder-Decoder结构。

输入：一段文本 “I am in CQ”

输出：一段文本 “我在重庆”

Encoder (编码器): 将输入变换到编码空间

Decoder(解码器): 获得预测输出（翻译）



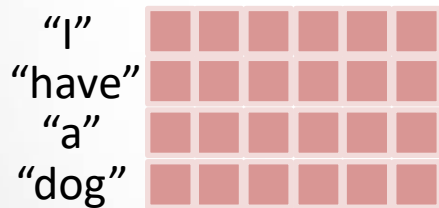
Transformer基本结构 (1个encoder+1个decoder)

第十章：神经网络与深度学习

□ Encoder的输入（embedding）

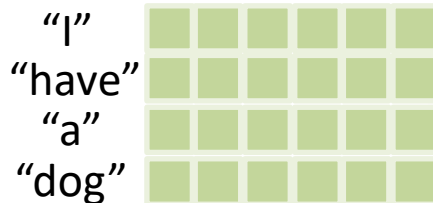
① 输入嵌入 (input embedding)

将每个单词进行word embedding，
形成512维的向量



② 位置编码 (positional encoding)

将每个单词的位置进行编码，
形成512维的向量



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Encoder的输入 $X_{n \times d} = \text{input embedding} + \text{positional encoding}$

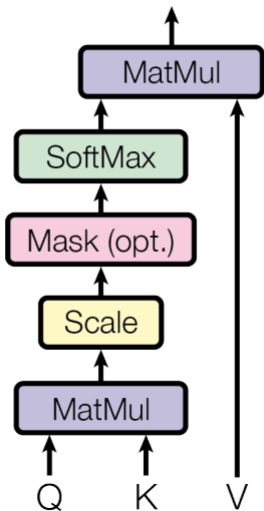
其中，n是单词的数量，d是嵌入维度

第十章：神经网络与深度学习

□ Attention（注意力机制）

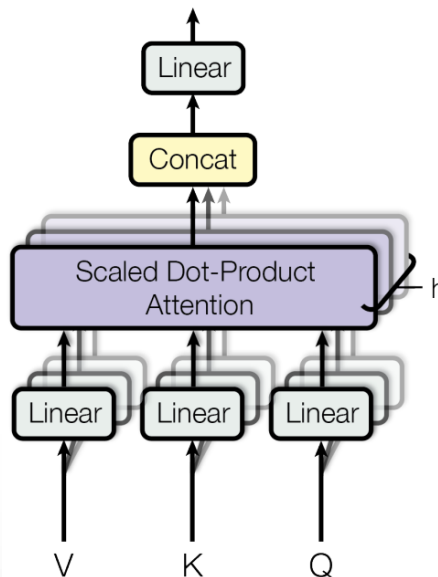
① Self-attention(自注意力)

Scaled Dot-Product Attention



② Multi-head attention (多头注意力)

Multi-Head Attention

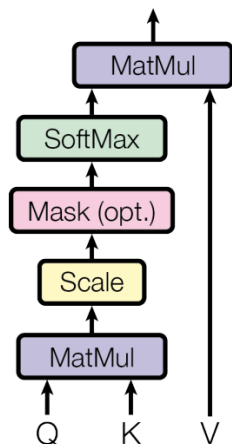


第十章：神经网络与深度学习

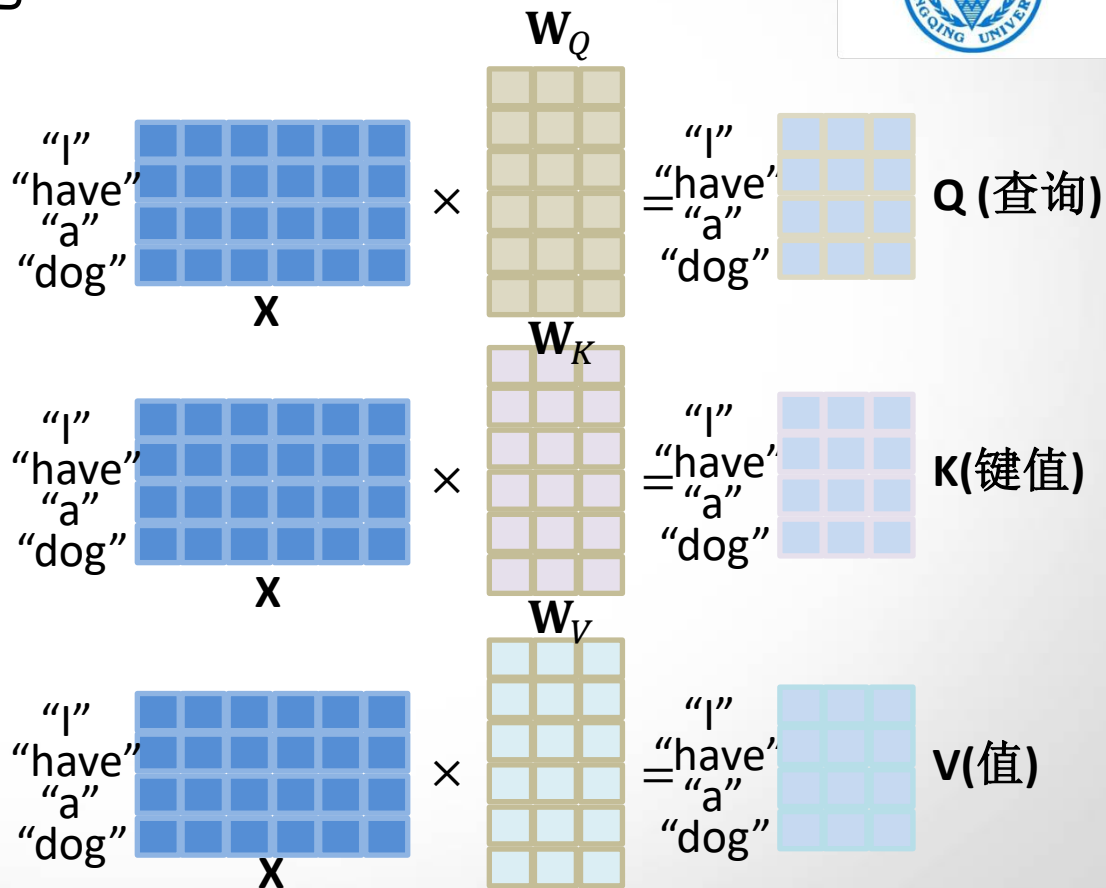
□ Attention（注意力机制）

① Self-attention(自注意力)

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

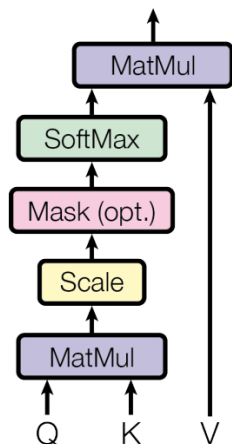


第十章：神经网络与深度学习

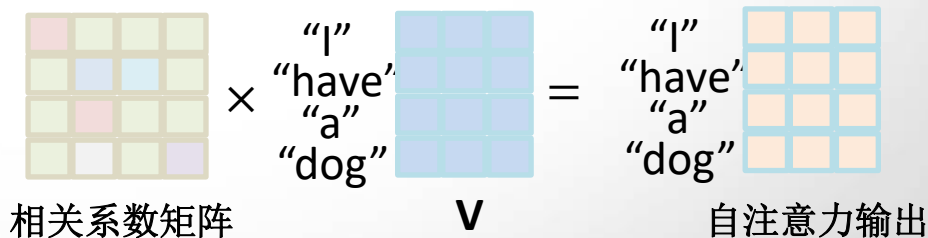
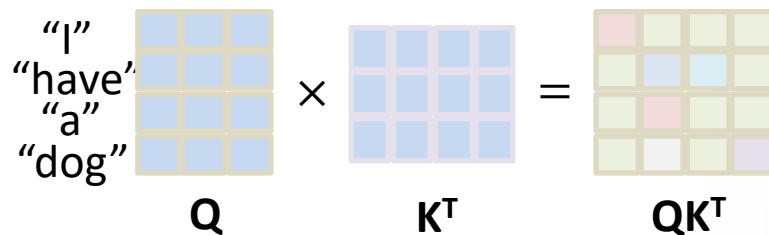
□ Attention（注意力机制）

① Self-attention(自注意力)

Scaled Dot-Product Attention



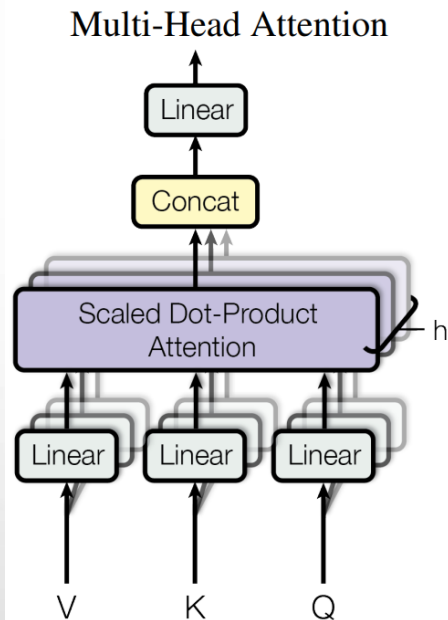
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



第十章：神经网络与深度学习

□ Attention（注意力机制）

② Multi-head attention (多头注意力)

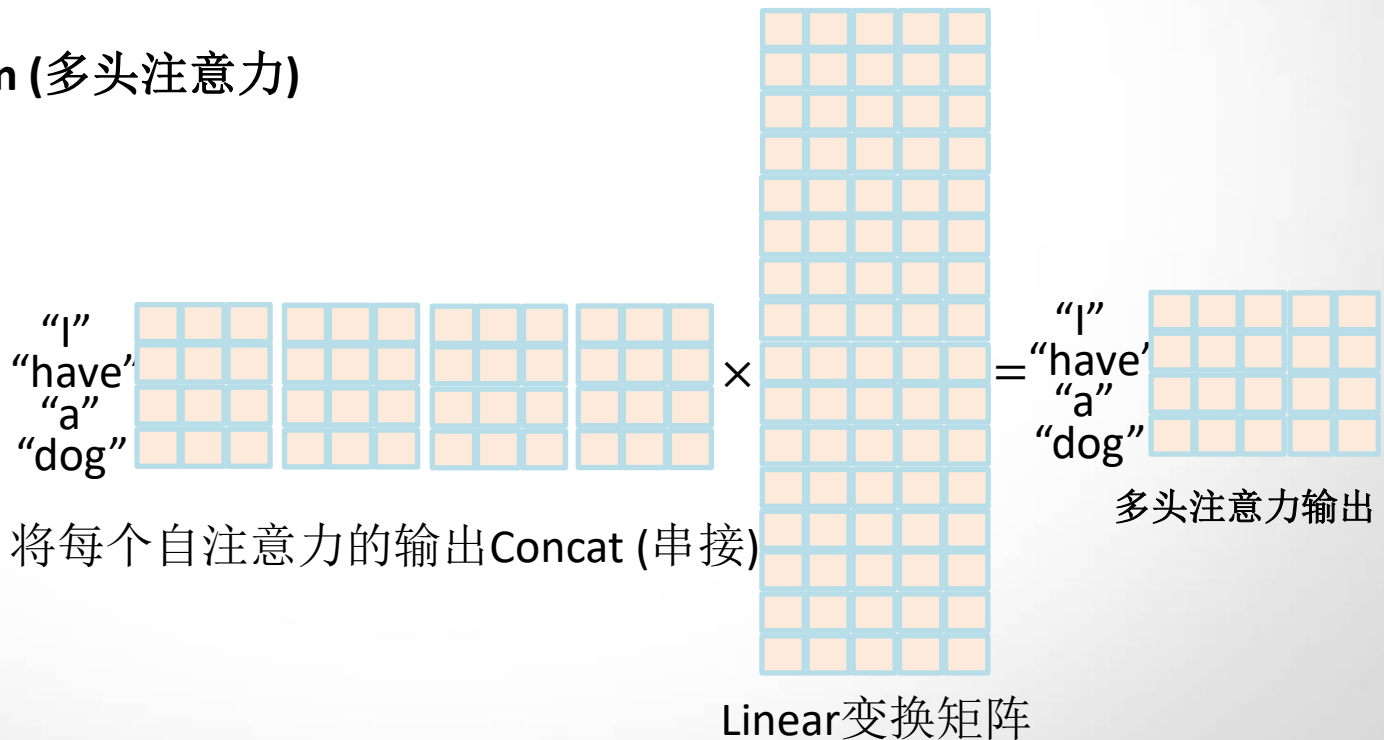
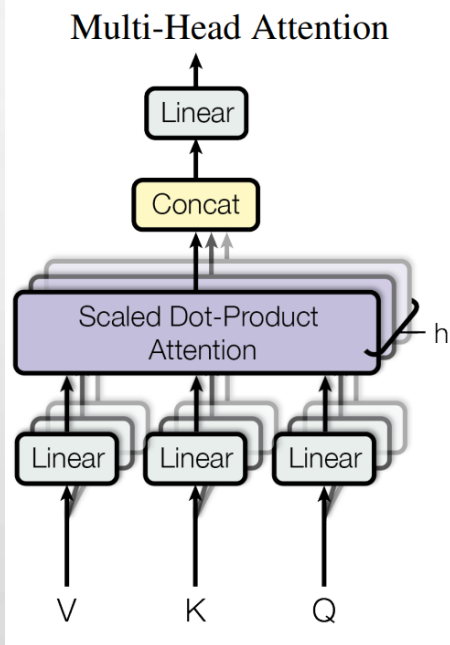


$$\begin{array}{c}
 \begin{array}{ccc}
 \begin{array}{|c|c|c|c|c|} \hline \text{red} & \text{green} & \text{green} & \text{green} & \text{green} \\ \hline \text{green} & \text{blue} & \text{blue} & \text{green} & \text{green} \\ \hline \text{green} & \text{red} & \text{green} & \text{green} & \text{green} \\ \hline \text{green} & \text{green} & \text{green} & \text{green} & \text{purple} \\ \hline \end{array} & \times & \begin{array}{c} \text{"I"} \\ \text{"have"} \\ \text{"a"} \\ \text{"dog"} \end{array} \\
 \text{注意力系数矩阵} & & V \\
 \hline
 \begin{array}{ccc}
 \begin{array}{|c|c|c|c|c|} \hline \text{red} & \text{green} & \text{green} & \text{green} & \text{green} \\ \hline \text{green} & \text{blue} & \text{blue} & \text{green} & \text{green} \\ \hline \text{green} & \text{red} & \text{green} & \text{green} & \text{green} \\ \hline \text{green} & \text{green} & \text{green} & \text{green} & \text{purple} \\ \hline \end{array} & \times & \begin{array}{c} \text{"I"} \\ \text{"have"} \\ \text{"a"} \\ \text{"dog"} \end{array} \\
 \text{注意力系数矩阵} & & V \\
 \hline
 \begin{array}{ccc}
 \begin{array}{|c|c|c|c|c|} \hline \text{red} & \text{green} & \text{green} & \text{green} & \text{green} \\ \hline \text{green} & \text{blue} & \text{blue} & \text{green} & \text{green} \\ \hline \text{green} & \text{red} & \text{green} & \text{green} & \text{green} \\ \hline \text{green} & \text{green} & \text{green} & \text{green} & \text{purple} \\ \hline \end{array} & \times & \begin{array}{c} \text{"I"} \\ \text{"have"} \\ \text{"a"} \\ \text{"dog"} \end{array} \\
 \text{注意力系数矩阵} & & V
 \end{array}
 \end{array}
 =
 \begin{array}{c}
 \begin{array}{|c|c|c|c|c|} \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \end{array} \\
 \text{自注意力输出} \\
 \hline
 \begin{array}{|c|c|c|c|c|} \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \end{array} \\
 \text{自注意力输出} \\
 \hline
 \begin{array}{|c|c|c|c|c|} \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \text{orange} & \text{orange} & \text{orange} & \text{orange} & \text{orange} \\ \hline \end{array} \\
 \text{自注意力输出}
 \end{array}$$

第十章：神经网络与深度学习

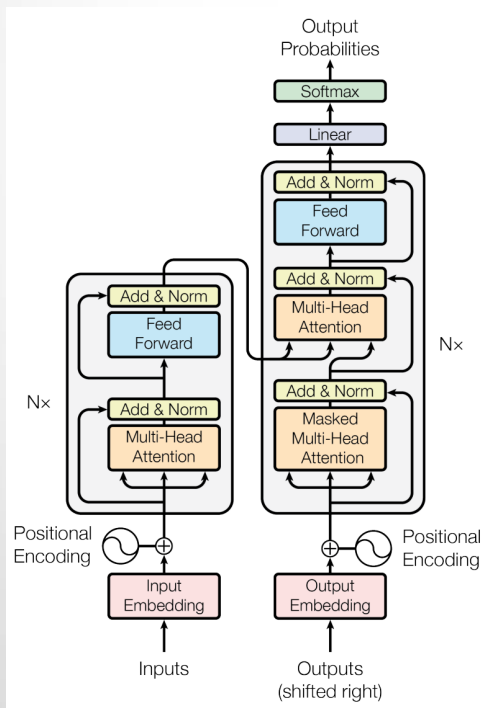
□ Attention（注意力机制）

② Multi-head attention (多头注意力)



第十章：神经网络与深度学习

□ Add、Normalization、Feed Forward（残差连接、归一化和前馈网络）



$$X = \text{LayerNorm}(X + \text{MHA}(X))$$



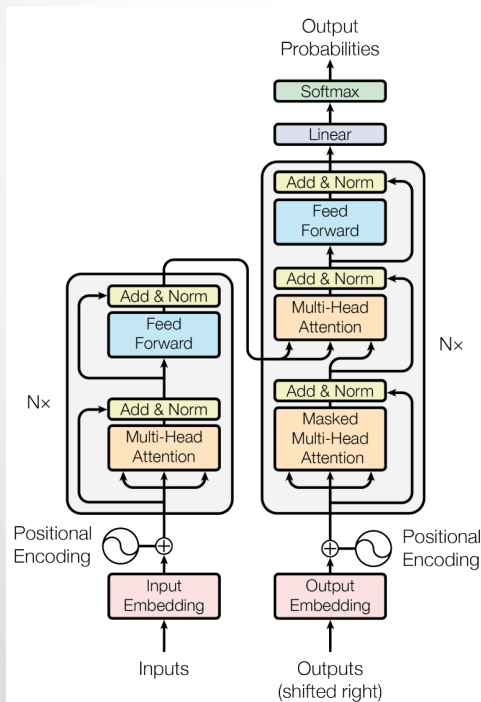
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



Encoder输出: $Y = \text{LayerNorm}(X + \text{FFN}(X))$

第十章：神经网络与深度学习

□ Decoder



与Encoder类似，区别在于：

1. 包括两个MHA和三个Add&Norm
2. 第一个MHA是masked MHA，即预测某个单词时，需要对该单词之后的词汇掩盖。



《论语·述而》中有言：

“志于道，据于德，依于仁，游于艺”

希望本课程可激发大家学习人工智能的兴趣、了解人工智能的知识、乐于掌握人工智能的技能、形成人工智能伦理品行。

谢谢!

实验室源码和数据发布<https://www.leizhang.tk>

《机器学习》

主讲：张磊

E-mail: leizhang@cqu.edu.cn

